

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ВВЕДЕНИЕ	6
Актуальность проблемы.....	6
Значимость и миссия проекта	7
Цели и задачи.....	9
Новизна	10
Подход к валидации.....	11
Критерии успеха	12
Глава 1: Анализ предметной области и видение решения	14
1.1 Анализ предметной области.....	14
1.2 Анализ проблемы "КАК ЕСТЬ"	19
1.3 Обзор существующих решений	21
1.4 Видение решения.....	24
1.5 Определение приоритетов	25
Глава 2: Методология, инструментарий и интеграция ML	29
2.1 Схема эксперимента	29
2.2 Обзор технологий и инструментов	30
2.2.1 Архитектура микросервисов: определение и обоснование	31
2.2.2 Докерная обработка и контейнеризация	32
2.2.3 Colima: Docker на macOS без Docker Desktop.....	34
2.2.4 Kotlin и Ktor Фреймворк.....	34
2.2.5 Python и FastAPI.....	36
2.2.6 Аутентификация JWT: структура, валидация и совместное использование между службами	36
2.2.7 Мультиплатформенность Kotlin (KMP): Кроссплатформенная мобильная разработка	38
2.2.8 SQLDelight: Типобезопасный SQL для мультиплатформенной Kotlin.....	40
2.2.9 Streamlit: быстрая разработка информационной панели	41
2.2.10 Ollama: локальный вывод LLM	41
2.2.11 Ngrok: безопасное туннелирование для местных сервисов.....	43

2.2.12 Render: платформа облачного развёртывания	44
2.2.13 Neon PostgreSQL: Бессерверная управляемая база данных.....	45
2.2.14 Сводная таблица	46
2.3 Стратегия обработки данных: данные, генерируемые приложением.	47
2.4 Конвейер анализа на основе LLM	50
2.5 Подсказки и типы аналитических запросов.....	52
2.6 ИИ-функции в приложении	55
2.7 Показатели оценки.....	57
2.8 Анализ результатов	60
2.9 Анализ ошибок и извлеченные уроки.....	71
2.10 Краткое изложение концепции.....	73
2.11 Качественная пользовательская оценка	75
Профиль 1 — Пользователь с высокой вовлечённостью в инструменты продуктивности.....	75
Профиль 2 — Пользователь с минималистским подходом.....	76
Профиль 3 — Сообщество пользователей приложений для продуктивности.....	77
Синтез результатов оценки	77
Глава 3: Архитектура и реализация	79
3.1 Системные проектные решения и архитектурные принципы	79
3.2 Архитектура микросервисов	80
3.3 Аутентификация и межведомственная идентификация	83
Процесс входа в систему.....	83
Совместное использование JWT: один токен для всех сервисов.....	84
В пользовательской регистрации: каноническое целочисленное тождество	84
Аутентификация по внутреннему ключу X: от сервиса к сервису	86
3.4 Модель данных и стратегия хранения.....	87
3.4.1 psocial_selfmanager.....	87
3.4.2 psocial_timer	89
3.4.3 psocial_analytics.....	90
3.4.4 psocial_billing.....	91
3.5 Реализация сервиса.....	92

3.6 Мобильное приложение (Kotlin Multiplatform)	93
3.6.1 Архитектура приложения: трехуровневая чистая архитектура	93
3.6.2 Локальная база данных: SQLDelight схема	95
3.6.3 Аутентификация	99
3.6.4 Проекты	101
3.6.5 Экран самостоятельного управления	102
3.6.6 Задачи	104
3.6.7 Привычки	107
3.6.8 Процедуры	109
3.6.9 Таймер Pomodoro	111
3.6.10 Журнал	114
3.6.11 Аналитические данные (страница пользователя)	116
3.6.12 Адаптивный макет	117
3.6.13 Протокол автономной синхронизации	119
3.7 Текущее состояние реализации	121
3.8 Определение области применения MVP	130
3.9 Планируемые сервисы и функции	132
3.9.1 Расширенный самоменеджмент: помимо задач, привычек и рутины	132
3.9.2 Расширенный таймер: общий фокус и полосы	134
3.9.3 Социальный слой	135
3.9.4 Пользовательский Сервис	138
3.9.5 Проприетарные модели ML: стратегия накопления данных	139
3.10 Веб-панели мониторинга	140
3.11 Развертывание	144
3.12 Тестирование	145
3.12.1 Тестирование надёжности и отказоустойчивости	146
3.13 Интеграция сервисов: сквозные потоки запросов	149
Поток 1: Проведение анализа с использованием искусственного интеллекта (полный сквозной анализ)	149
Поток 2: Внутренний predict биллингового сервиса	150
Поток 3: Запись времени таймером (межсервисная)	150

Поток 4: Аутентификация пользователя.....	151
Поток 5: Прямые AI-функции (NLP-разбор и контекстный анализ)...	152
3.14 Соображения безопасности	155
Текущая ситуация в области безопасности	155
Ограничения безопасности при проверке концепции	156
Изменения относительно утверждённого задания на ВКР	157
1. Состав микросервисов: переосмысление доменных границ	158
2. Протокол межсервисного взаимодействия: gRPC заменён на REST/HTTP	159
3. Брокер сообщений: RabbitMQ заменён на синхронный HTTP с graceful degradation	160
4. Интеграция машинного обучения: обучение на открытых датасетах заменено на LLM-инференс по паттерну RAG.....	161
5. Мобильное приложение: статус реализации	162
ЗАКЛЮЧЕНИЕ	165
Краткое изложение материалов.....	165
Текущие ограничения.....	167
Дорожная карта: следующий этап разработки.....	168
Этап 1 (немедленный):.....	168
Этап 2 (среднесрочный):.....	168
Этап 3 (долгосрочный):	168
Архитектурная готовность к росту	169
СПИСОК ЛИТЕРАТУРЫ.....	170
Приложение А	173
Глоссарий технических терминов	173
Приложение Б.....	177
Артефакты воспроизводимости.....	177
1. Репозитории и развёрнутые сервисы	177
2. Запуск локальной среды разработки	178
3. Шаблон переменных окружения.....	178
4. Тестовый набор данных.....	180
5. API-документация.....	180
6. Воспроизведение экспериментов	181

Приложение В.....	185
Файл docker-compose.yml.....	185
Приложение Г.....	189
Диаграмма модели данных.....	189
Приложение Д.....	190
Экраны мобильного приложения.....	190

ВВЕДЕНИЕ

Актуальность проблемы

Мы живем в эпоху беспрецедентного доступа к инструментам самосовершенствования. Современный человек, стремящийся повысить свою личную эффективность, имеет доступ к десяткам специализированных приложений для удовлетворения любых мыслимых потребностей в производительности: менеджеры задач для отслеживания рабочих задач, трекеры привычек для построения поведенческих рутин, таймеры Pomodoro для целенаправленных рабочих сессий, приложения для ведения журнала для размышлений и помощники на базе искусственного интеллекта для получения советов. Однако такое распространение инструментов создало парадокс. Само изобилие опций стало собственным препятствием: вместо того, чтобы обеспечивать ясность и сосредоточенность, фрагментированная экосистема программного обеспечения для повышения производительности вынуждает пользователей управлять своими инструментами для повышения производительности в качестве второй работы, переключаясь между приложениями, вручную выверяя данные и создавая целостную картину собственной производительности из разрозненных источников.

Основная проблема заключается в фрагментации данных. Пользователь, который выполняет задачу в своем диспетчере задач, тратит сорок минут на сеанс Pomodoro, связанный с этой задачей, и отражает день в своем журнале, генерирует три отдельных события данных в трех отдельных приложениях, которые ничего не знают друг о друге. Диспетчер задач видит завершенный элемент. Таймер видит сеанс определенной продолжительности. Журнал видит только то, что пользователь вводит вручную. Ни одна из этих систем не может сказать пользователю: “Вы потратили сорок минут на выполнение этой высокоприоритетной задачи, вы ее выполнили, и исторически ваши самые продуктивные занятия приходятся на утро вторника - вот что это означает для того, как вам следует планировать завтрашний день”.

Такая фрагментация сопряжена с конкретными издержками. Исследования и отзывы пользователей неизменно указывают на то, что переключение

контекста между приложениями является дорогостоящим с когнитивной точки зрения. Пользователи сообщают, что тратят значительное время на избыточный ввод данных в разных системах, а умственные затраты на обслуживание нескольких приложений часто приводят к отказу от одного или нескольких инструментов. Что еще более важно, разрозненный характер этих систем означает, что наиболее ценная информация — междоменные корреляции между моделями концентрации внимания, постоянством привычек, частотой выполнения задач и общей производительностью — остается невидимой. Каждое приложение может рассуждать только о своем собственном фрагменте данных.

Таким образом, возможность заключается не просто в создании другого приложения для повышения производительности, но и в создании инфраструктуры, устраняющей фрагментацию в корне: единой платформы, где каждое действие по повышению производительности обогащает единый контекст данных и где уровень анализа искусственного интеллекта имеет одновременный доступ ко всему этому контексту при генерации аналитических данных. Это проблема, которую призвана решить проект ProductiveSocial.

Значимость и миссия проекта

Значение ProductiveSocial выходит за рамки непосредственного удобства замены нескольких приложений одним. Проект представляет собой фундаментальный сдвиг во взаимоотношениях между пользователем и его инструментами повышения производительности — от пассивного отслеживания к активному анализу с учетом контекста.

Современные приложения для повышения производительности, даже самые сложные, остаются по сути пассивными хранителями записей. Они сохраняют то, что вводит пользователь, и отображают это с разной степенью наглядности. Даже функции ИИ в существующих инструментах ограничены собственными рамками данных приложения: диспетчер задач с помощью ИИ может только рассуждать о задачах. Отслеживание привычек с аналитикой может выявлять только закономерности в данных о привычках. Ни один из них не может

переступить черту, чтобы спросить: “Как менялись показатели выполнения задач в те дни, когда эта привычка была выработана? И в те же дни, как долго длились сеансы Pomodoro и сколько из них было заброшено?”

ProductiveSocial стремится стать первой платформой, которая сделает это междоменное рассуждение способом обратной связи с пользователями по умолчанию. Когда пользователь запускает сводный анализ производительности, искусственный интеллект получает полный контекст производительности пользователя: каждую задачу и ее статус, каждую привычку и частоту ее выполнения, каждую рутину и ее соблюдение, а также полную историю сеансов фокусирования, включая частоту завершения, продолжительность работы, часы пик и модели отказа. Такая широта контекста позволяет получить качественно инсайдерскую информацию, которая невозможна ни в одном изолированном инструменте, независимо от того, насколько сложной может быть его собственная внутренняя аналитика.

Миссия выражена фразой, которая с самого начала определяла дизайн проекта: превратить инструменты повышения производительности из пассивных контейнеров данных в интеллектуальную операционную систему для себя. Это обрамление имеет прямое архитектурное значение. Операционная система не просто хранит программы; она управляет их ресурсами, координирует их взаимодействие и предоставляет пользователю единый интерфейс для всех возможностей компьютера.

Социальное измерение платформы - модуль для обмена достижениями в области производительности, схемами привычек и схемами фокусировки с сообществом пользователей — добавляет дополнительный уровень значимости. Социальная ответственность - один из наиболее эмпирически подтвержденных механизмов сохранения привычек и достижения целей. Проектируя социальный слой, архитектурно отделенный от рабочего пространства для повышения производительности (предотвращая отвлекающее и навязчивое поведение, характерное для основных социальных сетей), ProductiveSocial стремится использовать мотивационную силу сообщества, не внося фрагментации внимания, которая в первую очередь подрывает производительность.

Цели и задачи

Основной целью этого проекта является разработка и внедрение отказоустойчивой платформы микросервисов, которая объединяет основные инструменты управления личной производительностью в целостную экосистему, улучшенную искусственным интеллектом.

Эта цель подразделяется на следующие конкретные задачи:

Задача 1 — Разработать архитектуру микросервисов. Определите границы служб, протоколы связи, модель аутентификации и правила владения данными для системы независимых, но совместимых внутренних служб. Каждая служба должна быть независимо развертываемой, независимо масштабируемой и способной продолжать работу даже тогда, когда другие службы временно недоступны.

Задача 2 — Внедрение основных служб повышения производительности. Создайте и разверните службу самостоятельного управления, отвечающую за управление производительностью пользователей. Создайте и разверните службу таймера, службу отслеживания сеансов Pomodoro focus.

Задача 3 — внедрить конвейер аналитики искусственного интеллекта. Создайте службу, которая организует сбор данных из всех других служб, создает структурированные запросы, вызывает вывод LLM через службу выставления счетов и сохраняет аналитические отчеты.

Задача 4 — Внедрение инфраструктуры кредитования и логического вывода. Создание внутренней бухгалтерской книги и механизма логического вывода. Поддерживайте LLM (Ollama, Anthropic Claude, OpenAI GPT) и модель scikit-learn для будущего использования. Создайте кредитную систему в виде книги учета транзакций с автоматической инициализацией приветственного кредита.

Задача 5 — Создание административного и пользовательского интерфейсов. Первоначально внедрите панель администратора на основе

Streamlit для управления системой и панель пользователя для доступа к данным о производительности, аналитике фокуса, анализу искусственного интеллекта и кредитному балансу.

Задача 6 — Реализовать синхронизацию сначала в автономном режиме. Разработайте и внедрите идиоматичный протокол пакетной синхронизации, позволяющий мобильным клиентам работать без подключения к сети и надежно синхронизировать данные при повторном подключении.

Задача 7 — сквозная проверка системы. Проведите эмпирическое тестирование всего конвейера анализа от ввода данных до вывода LLM, оцените качество сгенерированных аналитических данных по сравнению с базовым уровнем, основанным на правилах, и проверьте правильность всех взаимодействий служб, включая списание кредитов, межведомственную аутентификацию JWT и целостность автономной синхронизации.

Новизна

Новизна заключается в четырех взаимосвязанных измерениях.

Архитектурная новизна — единый контекст производительности для рассуждений об ИИ. Существующие приложения с улучшенной производительностью, основанные на ИИ, ограничивают свой ИИ данными, принадлежащими одной службе. Диспетчер задач с ИИ может рассуждать только о задачах. Трекер привычек с ИИ может рассуждать только о привычках. ProductiveSocial разработан специально для того, чтобы предоставить уровню анализа искусственного интеллекта одновременный доступ ко всем измерениям контекста продуктивности пользователя: задачам, привычкам, распорядку дня и данным сеанса фокусировки, агрегированным на момент анализа. Этот междоменный контекст является не функцией пользовательского интерфейса, а архитектурной гарантией, обеспечиваемой внутренним дизайном API сервис аналитики, которая явно выполняет выборку из всех вышестоящих сервисов перед любым вызовом LLM.

Методологическая новизна — обучающие данные, генерируемые приложением, как первоклассная цель проектирования. В отличие от существующих инструментов повышения производительности, которые рассматривают искусственный интеллект как неотъемлемую функцию, ProductiveSocial рассматривает накопление данных о поведении пользователей как основную цель проектирования. Каждое завершение задачи, журнал привычек, рутинный шаг и интервал сеанса фокусировки хранятся в структурированной схеме с отметками времени и перекрестными ссылками. Это не случайно; это преднамеренная основа для будущего перехода от анализа, основанного на LLM, к проприетарным прогностическим моделям, обученным на реальных моделях поведения пользователей. Путь вывода в архитектуре sklearn в сервисе выставления счетов существует специально для обеспечения этого перехода без каких-либо изменений в более широкой системе.

Подход к валидации

Проверка организована на трех уровнях:

Уровень 1 — Функциональная корректность. Для каждого поведения базовой системы (аутентификация JWT, учет кредитоспособности, идемпотентность синхронизации, межсервисная коммуникация) мы определяем критерий прохождения / неудачи и тестируем на соответствие ему с помощью документированных входных и выходных данных. Эти тесты подтверждают, что система выполняет то, для чего она предназначена.

Уровень 2 — Качество интеграции. Для конвейера сквозного анализа - наиболее сложного и наиболее ценного поведения в системе - мы определяем базовый уровень (сводка на основе правил), условие сравнения (анализ, сгенерированный LLM), метрику (оценка релевантности человеком по шкале 1-5) и документированный результат с примерами входных и выходных данных.

Уровень 3 — Эксплуатационная надежность. Для развертывания на бесплатном уровне Render мы измеряем задержку холодного запуска, время

отклика в режиме ожидания и поведение в условиях недоступности сервиса (например, когда сервис таймера недоступна во время вызова аналитики).

Схема эксперимента и результаты полностью задокументированы в главе 2.

Критерии успеха

Критерии успеха для этого проекта определены по трем измерениям, каждое из которых имеет конкретный метод измерения:

1. Корректность системной интеграции.

- Все серверные сервисы успешно взаимодействуют, используя определенные протоколы (JWT для связи клиент-сервис, X-Internal-Key для связи сервис-сервис).
- Вся информация о производительности пользователя успешно агрегируется и передается в LLM в виде единого запроса аналитики.
- Списание кредита является математически корректным и регистрируется в журнале транзакций.
- Измерение: функциональный тест с документированными парами ввода / вывода. Цель: 100%-ная скорость прохождения.

2. Качество анализа искусственного интеллекта.

- Анализы, сгенерированные LLM, оцениваются на основе базовых показателей, основанных на правилах, на предмет релевантности, специфичности и возможности принятия мер.
- Измерение: параллельное сравнение с документально подтвержденной оценкой. Цель: балл $LLM \geq 3,5 / 5$ по сравнению с базовый уровень $\leq 2,0 / 5$.

3. Надежность синхронизации.

- Автономная синхронизация корректно сохраняется и согласовывает все типы данных после имитированного цикла отключения-повторного подключения.
- Гарантия идемпотентности: повторная отправка одного и того же запроса на синхронизацию не приводит к дублированию записей.
- Измерение: тест с заранее определенным набором данных из 32 сеансов, 55 отработанных навыков, 8 заданий. Цель: корректная синхронизация 100% записей с нулевым количеством дубликатов при повторной попытке.

Глава 1: Анализ предметной области и видение решения

1.1 Анализ предметной области

Область управления личной продуктивностью находится на пересечении поведенческой психологии, взаимодействия человека и компьютера и прикладного искусственного интеллекта. Понимание предметной области требует четкого представления о типах продуктивной работы, которой занимаются пользователи, о когнитивных и мотивационных механизмах, управляющих поведением в этой области, и о технологическом ландшафте, который возник для ее поддержки.

Категории производительности работают. На самом широком уровне инструменты повышения производительности учитывают четыре различные категории поведения пользователей:

Целенаправленное управление задачами предполагает разложение целей на отдельные элементы, требующие выполнения, с указанием сроков, приоритетов и состояний завершения. Это старейшая и наиболее развитая категория инструментов для повышения производительности, уходящая корнями в методологию Getting Things Done (GTD) и более ранние бумажные системы. Пользователям в этом режиме необходимо быстро захватывать рабочие элементы, упорядочивать их по контексту и приоритету, отслеживать завершение и отображать наиболее важные элементы в любой данный момент.

Формирование привычек и распорядка дня включает в себя установление и поддержание повторяющегося поведения — как разовых действий в день (чистка зубов, медитация), так и структурированных последовательностей шагов, выполняемых по порядку (утренняя рутина, протокол учебного занятия). В отличие от управления задачами, которое ориентировано на завершение и закрытость, отслеживание привычек ориентировано на последовательность и поддержание последовательности действий. Механика мотивации иная: награда - это не завершение, а непрерывность, а риск - это не отставание, а прерывистость. Процедуры добавляют временное и последовательное измерение: не просто

“сделал ли я это”, но “выполнил ли я все шаги в правильном порядке и за нужное время”.

Сеансы целенаправленной работы представляют собой особый способ повышения производительности, при котором пользователь обязуется непрерывно концентрироваться на одной задаче в течение определенного интервала. Методика Помодоро, разработанная Франческо Чирилло в 1980-х годах, оформила это в широко принятый протокол: работайте 25 минут, сделайте 5-минутный перерыв, повторите. Появляется множество вариантов с разной длительностью, и отдельные пользователи обычно выбирают длину интервала, соответствующую их личной концентрации внимания. Ключевым показателем в этой области является не завершение (задачи) или последовательность (привычки), а глубина вовлеченности — общее время сосредоточения, скорость завершения сеанса и отсутствие перерывов.

рефлексивное ведение дневника выполняет принципиально иную психологическую функцию, чем вышеупомянутые три категории. Вместо того чтобы продвигать действия вперед, ведение дневника создает пространство для ретроспективного осмысления: объединения событий дня в связное повествование, обработки эмоций и извлечения уроков. В контексте продуктивности ведение дневника наиболее ценно, когда оно связывает субъективный опыт прошедшего дня (что он чувствовал) с объективным отчетом о деятельности (что было фактически достигнуто). Эта связь - именно то, чего ручное ведение журнала не может сделать без посторонней помощи и что делает возможным синтез с помощью искусственного интеллекта.

Поведенческая экономика инструментов повышения производительности. Важным, но часто упускаемым из виду аспектом этой области является разрыв между внедрением инструментов и их последовательным использованием. Исследования моделей использования приложений для повышения производительности постоянно показывают, что большинство пользователей, которые загружают новое приложение для повышения производительности, отказываются от него в течение первых двух недель. Приводимыми основными причинами являются: накладные расходы на

ввод данных, отсутствие видимой ценности от введенных данных и трудности при переключении контекста между несколькими приложениями. Эти шаблоны имеют прямое значение для проектирования системы: эффективная платформа для повышения производительности должна сводить к минимуму трудности при вводе данных, максимизировать видимую отдачу от введенных данных и сокращать количество приложений, которыми должен управлять пользователь.

Теория самоопределения и внутренняя мотивация. Исследования в области поведенческой психологии помогают понять, почему инструменты повышения производительности работают успешно или терпят неудачу. Теория самоопределения (Deci and Ryan, 1985) выделяет три фундаментальные психологические потребности, которые стимулируют устойчивое вовлечение в любую деятельность: автономия (чувство выбора и сопричастности), компетентность (восприятие роста и мастерства) и связанность (связь с другими людьми, разделяющими схожие цели). Приложения для повышения производительности, которые удовлетворяют этим потребностям — предоставляя пользователям контроль над своей системой, делая заметным прогресс значимыми способами и подключая их к сообществу ответственных сотрудников, — обеспечивают значительно более длительное удержание, чем те, которые удовлетворяют только инструментальную потребность “довести дело до конца”.

Методика Помодоро успешна отчасти потому, что она напрямую затрагивает аспект компетентности: каждое завершенное занятие является видимой единицей достигнутого фокуса, а накопление завершенных циклов с течением времени создает конкретное представление о дисциплинированной работе. Специалисты по отслеживанию привычек добиваются успеха, когда визуализируют полосы, потому что эта полоса создает динамику неприятия потерь — прерывание 14-дневной полосы кажется дорогостоящим, что мотивирует к продолжению. Ни один из методов не работает надежно изолированно; их эффект усиливается, когда пользователь может видеть, что вырабатываемые им привычки коррелируют с целенаправленными рабочими сессиями, которые приводят к выполнению наиболее важных задач.

Когнитивная нагрузка и последствия для дизайна. Теория когнитивной нагрузки (Sweller, 1988) проводит различие между внутренней когнитивной нагрузкой (присущей сложности выполняемой задачи), посторонней когнитивной нагрузкой (накладными расходами, обусловленными плохим дизайном инструментов) и естественной когнитивной нагрузкой (умственными усилиями, которые приводят к обучению и долговременной памяти). Фрагментированный стек производительности создает значительную дополнительную когнитивную нагрузку: пользователь должен помнить об обновлении нескольких систем, должен мысленно интегрировать информацию из разных источников и должен управлять когнитивным переходом между различными контекстами приложения. Интегрированная платформа сводит эту внешнюю нагрузку к практически нулю — пользователь взаимодействует с одной системой, а система обрабатывает интеграцию данных внутри компании.

Роль искусственного интеллекта в производительности. Интеграция искусственного интеллекта в инструменты повышения производительности значительно ускорилась с момента массового появления больших языковых моделей в 2022-2023 годах. Текущие приложения искусственного интеллекта в этой области делятся на несколько категорий: анализ входных данных на естественном языке (преобразование устных или печатных описаний в структурированные записи задач), оптимизация планирования (рекомендации, когда планировать задачи, исходя из календаря и рабочей нагрузки), распознавание образов (определение того, какие привычки связаны с днями с высокой производительностью) и генеративное обобщение (создание письменных сводок журналов активности). Общим для всех текущих реализаций ограничением является проблема границ данных, описанная во введении: каждая функция искусственного интеллекта может рассуждать только о данных, принадлежащих приложению, в котором она размещена.

Целевые профили пользователей. Чтобы обосновать проектные решения с учетом конкретных потребностей пользователей, в ходе анализа предметной области были определены три репрезентативных профиля пользователей:

Профиль А — Работник умственного труда. Профессионал с высокими когнитивными нагрузками (разработчик программного обеспечения, академический исследователь, писатель), которому необходимо управлять постоянным накоплением сложных задач, поддерживать набор ежедневных профессиональных привычек (сосредоточенное чтение, ежедневное письмо, физические упражнения) и отслеживать, как распределяется его время сосредоточения по проектам. Основной болевой точкой этого пользователя является неспособность ответить на вопрос: “Трачу ли я свое время на то, чтобы сосредоточиться на правильных вещах?” Они технически сложны и будут использовать API, если пользовательского интерфейса недостаточно.

Профиль В — Студент. Студент университета, контролирующий сроки выполнения курсовых работ, учебные привычки (ежедневный пересмотр, конспекты лекций, практические задачи) и процедуры подготовки к экзаменам. Основная задача этого пользователя - сбалансировать множество параллельных обязательств с непредсказуемыми рабочими нагрузками. Они высоко мотивированы сериями и элементами геймификации, но быстро отказываются от приложений, если затраты на ввод данных высоки. Они используют свой телефон в качестве основного вычислительного устройства.

Профиль С — Энтузиаст самосовершенствования. Пользователь, в первую очередь мотивированный формированием привычек и оптимизацией образа жизни, а не профессиональным управлением задачами. Их основным использованием платформы будет отслеживание привычек (физические упражнения, медитация, питание, чтение) с периодическим анализом ИИ закономерностей согласованности и корреляции между привычками и общим самочувствием.

Эти три профиля послужили основой для определения приоритетов функций в разделе 1.5 и принятия оперативных инженерных решений в разделе 2.5.

1.2 Анализ проблемы "КАК ЕСТЬ"

Текущее состояние рынка инструментов для повышения личной производительности можно охарактеризовать тремя структурными проблемами, которые адекватно не решены ни одним существующим решением.

Проблема 1: Изолированные данные и фрагментированный пользовательский контекст

Фундаментальным архитектурным ограничением текущего рынка является то, что каждое производительное приложение поддерживает свое собственное изолированное хранилище данных. Пользователь, использующий Todoist для управления задачами, Habitica для отслеживания привычек, Forest для определения времени фокусировки и Day One для ведения журнала, имеет четыре отдельных приложения, четыре отдельных учетные записи пользователей и четыре отдельные базы данных. Не существует механизма, с помощью которого какая-либо из этих систем могла бы наблюдать за событиями, происходящими в других.

Это создает дополнительную проблему для анализа ИИ. Даже в самых сложных текущих реализациях помощник ИИ в диспетчере задач может только наблюдать за задачами. Он не может знать, что пользователь потратил 40 минут сосредоточенности на конкретной задаче этим утром, что привычка к ежедневным упражнениям, которая должна была сопровождать это рабочее занятие, была пропущена сегодня, или что пользователь постоянно отказывался от занятий сосредоточенностью после второго Pomodoro в течение последних двух недель. Каждый из этих фактов фиксируется в какой-либо системе, но ни одна система не имеет доступа ко всем из них одновременно.

В результате пользователи получают фрагментарную обратную связь без учета контекста. Диспетчер задач сообщает вам, что вы делали; отслеживание привычек сообщает вам, были ли вы последовательны; таймер показывает, как долго вы сосредотачивались. Ни один из них не может рассказать вам, как взаимодействуют эти измерения или что междоменные паттерны в вашем поведении подсказывают о том, как вам следует скорректировать свой подход.

Проблема 2: Накладные расходы на ручной ввод данных и переключение контекста

Пользователи, пытающиеся поддерживать несколько приложений для повышения производительности, сталкиваются со значительной административной нагрузкой. Одна и та же информация часто должна вводиться в нескольких местах: задача, выполненная в диспетчере задач, также должна быть отражена в журнале, система отслеживания привычек должна отмечать рабочий сеанс, который сопровождал ее, а журнал таймера должен содержать ссылку на задачу, которая была в центре ее внимания. На практике пользователи либо дублируют ввод данных (увеличивая накладные расходы), либо соглашаются с несогласованностью между системами (уменьшая ценность каждой из них), либо отказываются от всех приложений, кроме одного (теряя охват важных измерений).

Затраты на переключение контекста при переходе от приложения к приложению - это не просто временные затраты; это когнитивные затраты. Исследование остаточного внимания (феномена, при котором внимание задерживается на предыдущем приложении даже после переключения) предполагает, что каждое переключение контекста наносит ощутимый урон когнитивным ресурсам. Для пользователя, который уже стремится сохранить сосредоточенность, требование управлять четырьмя отдельными производительными приложениями само по себе является препятствием для состояния, которое эти приложения должны поддерживать.

Проблема 3: Отсутствие Междисциплинарного Поведенческого интеллекта

Текущий искусственный интеллект производительности локально оптимален, но глобально слеп. Каждое приложение может предоставлять сложную аналитическую информацию в своей собственной области — Todoist может рекомендовать расстановку приоритетов задач на основе крайних сроков и исторических показателей выполнения; Habitica может предсказать, какие привычки подвержены риску нарушения, на основе данных о

последовательности действий; Forest может показывать шаблоны фокусировки по времени суток. Но ни одно приложение не может спросить: “В те дни, когда ваша утренняя зарядка была завершена, как менялась скорость выполнения ваших дневных задач? И каков был уровень отказа от помидоров в те дни по сравнению с днями, когда вы отказывались от физических упражнений?”

Такого рода междоменные запросы - это именно тот тип инсайта, который имеет наибольшую практическую ценность для изменения поведения. Поведенческие исследования последовательно показывают, что привычки не действуют изолированно — они образуют взаимозависимые сети, в которых завершение или отказ от одной привычки оказывает вероятностное влияние на другие в тот же день. Без системы, учитывающей полный поведенческий контекст, эти взаимозависимости остаются невидимыми, и помощь, получаемая пользователями с помощью искусственного интеллекта, неизбежно будет более поверхностной, чем это технически возможно.

1.3 Обзор существующих решений

Чтобы определить конкурентную среду и продемонстрировать конкретный пробел, который заполняет ProductiveSocial, мы проанализировали семь ведущих приложений, представляющих различные категории инструментов для повышения производительности. В анализе рассматриваются шесть аспектов: возможность управления задачами, возможность отслеживания привычек, встроенная функциональность focus / Pomodoro, функции анализа ИИ, автономная архитектура и имеет ли ИИ доступ к унифицированным межмодульным данным.

Todoist - лидер рынка управления задачами, насчитывающий более 30 миллионов пользователей. Он предоставляет сложную систему задач, ориентированную на GTD, с проектами, ярлыками, уровнями приоритета, повторяющимися задачами и анализом даты на естественном языке. Его функции искусственного интеллекта (“AI Assistant”) сосредоточены на переформулировании задач, предложениях по декомпозиции и автоматическом

планировании. Однако в Todoist нет встроенного отслеживания привычек, нет функциональности Pomodoro, а его искусственный интеллект полностью ограничен областью задач. Пользователи, которые хотят отслеживать привычки вместе с Todoist, должны интегрировать отдельное приложение, и межприложенческий анализ с помощью искусственного интеллекта недоступен.

Habitica геймифицирует выработку привычек, превращая привычки, ежедневные задачи и поручения в ролевую игру. У него большое и заинтересованное сообщество и социальные функции, которые действительно используются для обеспечения подотчетности. Однако его управление задачами является рудиментарным по сравнению со специализированными менеджерами задач, у него нет функциональности Pomodoro и у него нет каких-либо функций анализа искусственного интеллекта. Его социальные функции специально разработаны на уровне геймификации и не могут быть перенесены в негеймифицированный контекст.

Forest - это специализированное приложение Pomodoro timer с метафорой посадки: пользователи выращивают виртуальные деревья во время сеансов фокусировки, которые погибают, если они покидают приложение. Он очень эффективен в качестве устройства для привлечения внимания, но не поддерживает управление задачами или отслеживание привычек, а его аналитика ограничена графиками истории сеансов. Анализ с помощью искусственного интеллекта отсутствует, и данные недоступны вне приложения.

Notion - это самый близкий из существующих продуктов к концепции универсальной платформы для повышения производительности. Его гибкость — пользователи могут создавать собственные базы данных, связывать записи между страницами и создавать любую организационную структуру, какую только могут себе представить, — сделала его действительно полезным в качестве единого рабочего пространства для многих пользователей. Однако эта гибкость также является ее ограничением в области производительности: в Notion отсутствует встроенная механика отслеживания привычек, для управления задачами требуется ручная настройка, а не предоставление специализированных структур производительности, для функциональности

Pomodoro требуются встраивания сторонних производителей, а функции искусственного интеллекта (Notion AI) в первую очередь ориентированы на редактирование текста и обобщение, а не на анализ поведенческих паттернов. Важно отметить, что ИИ Notion может рассуждать только о содержании страниц Notion, а не о междоменных поведенческих паттернах, полученных из структурированных журналов активности.

Day One - ведущее приложение для ведения журнала для iOS и macOS с мощными функциями конфиденциальности, поддержкой медиа и пометкой местоположения. Он отлично подходит для ведения рефлексивного журнала вручную, но в нем нет управления задачами, отслеживания привычек, функций Pomodoro и искусственного интеллекта, который мог бы синтезировать данные об активности из других источников. Его возможности искусственного интеллекта ограничены транскрипцией и базовой поддержкой текста.

RescueTime - это приложение для автоматического отслеживания рабочего времени, которое пассивно отслеживает, на какие приложения и веб-сайты тратит время пользователь, и генерирует показатели производительности и отчеты. Это решает другую проблему — пассивный мониторинг фактического использования компьютера, а не намеренное отслеживание запланированных действий. В нем нет управления задачами, отслеживания привычек и таймера Pomodoro. Его функции искусственного интеллекта создают автоматические еженедельные отчеты на основе данных об использовании компьютера. Он не интегрируется с менеджерами задач или трекерами привычек.

Sunsama - это приложение для ежедневного планирования, предназначенное для работников умственного труда. Он интегрируется с Todoist, Asana, GitHub и другими внешними сервисами для объединения задач в единый ежедневный просмотр с поддержкой ограничения по времени. Он обладает базовыми функциями фокусировки в стиле Pomodoro и генерирует ежедневные сводки “выключения”. Однако в нем нет отслеживания привычек, поддержка Pomodoro является базовой, его искусственный интеллект ограничен суммированием задач дня, и для полноценной интеграции требуются платные подписки на внешние сервисы.

Сравнение приведено в следующей таблице:

Feature	Todoist	Habitica	Forest	Notion	Day One	RescueTime	Sunsama	ProductiveSocial
Task Management	✅ Full	⚠️ Basic	❌	✅ Manual setup	❌	❌	✅ Full	✅ Full
Habit Tracking	❌	✅ Full	❌	⚠️ Manual setup	❌	❌	❌	✅ Full (Start/Quit, subtasks, completion logs)
Pomodoro Timer	❌	❌	✅ Full	⚠️ Plugin only	❌	❌	⚠️ Basic	✅ Full (adaptive, abandonment risk, entity-linked)
AI Analysis	⚠️ Task-only	❌	❌	⚠️ Text editing	⚠️ Basic	⚠️ Usage reports	⚠️ Day summary	✅ Full cross-domain
Offline-First	✅	❌	⚠️	⚠️ Partial	✅	❌	❌	✅ Full batch sync
Social / Community	❌	✅ Gamified	❌	❌	❌	❌	❌	📅 Planned
Unified AI Context	❌	❌	❌	❌	❌	❌	❌	✅ (tasks + habits + routines + focus)
Open LLM Provider	❌	❌	❌	❌	❌	❌	❌	✅ (Ollama/Anthropic/OpenAI)

Рисунок 1.1 — Сравнительный анализ конкурирующих продуктов

Утверждение в этом отчете о том, что “архитектура, объединяющая все типы отдельных инструментов повышения производительности с единым уровнем анализа искусственного интеллекта”, является новой, подтверждается этим сравнением. Хотя отдельные модули не являются чем-то новым по отдельности, их комбинация в рамках единой модели данных с междоменным доступом искусственного интеллекта представляет собой системный дизайн, которого не существует на текущем рынке.

1.4 Видение решения

Решение ProductiveSocial решает три выявленные проблемы с помощью определенного набора архитектурных и продуктовых решений.

Для решения проблемы разрозненности данных: Архитектура микросервисов, в которой все сервисы используют общую идентификацию

пользователя, и выделенная служба аналитики, которая объединяет данные по всем сервисам перед вызовом анализа с помощью искусственного интеллекта. Пользователь не интегрирует внешние сервисы; все модули встроены в платформу, а служба аналитики имеет гарантированный внутренний доступ ко всем данным через межведомственный уровень связи, защищенный общим секретом.

Для решения проблемы ввода данных и переключения контекста: единое приложение со специально созданными пользовательскими интерфейсами для каждого модуля производительности, подключенное к общей модели данных. Задача, созданная в диспетчере задач, автоматически становится доступной в качестве целевой ссылки в таймере, и время, зарегистрированное в таймере, автоматически записывается обратно в задачу. Автономная архитектура с пакетной синхронизацией гарантирует, что пользователь может использовать приложение без подключения к сети, устраняя общую проблему для мобильных инструментов повышения производительности.

Для решения задачи междоменного анализа: Служба аналитики, которая собирает полный контекст производительности на момент анализа и передает его в большую языковую модель с подсказками для конкретных типов, предназначенными для получения междоменной информации. ИИ рассуждает не только о задачах или сеансах фокусировки; он рассуждает о полном поведенческом профиле пользователя, который фиксируется интегрированной платформой.

1.5 Определение приоритетов

Функции продукта были расставлены по приоритетам с использованием концепции соотношения ценности и затраченных усилий с дополнительным ограничением, заключающимся в том, что система должна функционировать как единое интегрированное целое на каждом этапе - частичные реализации, нарушающие поток данных между модулями, неприемлемы.

Приоритет 1 (должен быть у MVP):

- Аутентификация и идентификация пользователя
- Управление задачами с проектами, подзадачами, приоритетами и автономной синхронизацией
- Отслеживание привычек
- Рутинное управление с упорядоченными шагами
- Жизненный цикл сеанса Pomodoro (создание, интервал запуска, завершение, отмена)
- Конвейер межведомственного анализа с использованием искусственного интеллекта, включающий не менее четырех типов анализа
- Кредитная система с регистрацией транзакций и приветственным депозитом
- Панель администратора для управления моделями и кредитами
- Мобильное приложение с полным охватом функций.
- Модуль дневника — ежедневные записи с заметками, отметкой настроения и рефлексией, которые становятся ещё одним источником данных для ИИ-анализа; запланирован к завершению до выпуска MVP
- Цели как сквозная функция — встраивается в selfmanager и модуль дневника, позволяя связывать задачи, привычки и записи с долгосрочными целями;

Приоритет 2 (важный, но после MVP):

- Разработка модулей социальной сети — публичные профили, лента сообщества, обмен достижениями и привычками, а также партнёры по подотчётности
- Внедрение платных функций и модели подписки — вынесение части функций (например, расширенного ИИ-анализа или расширенной

аналитики) за платный доступ на основе уже реализованной кредитной системы

- Размещение рекламы в ленте социальной сети как дополнительный источник монетизации

Приоритет 3 (Будущее — обогащение и расширение уже существующих систем по мере роста пользовательской базы):

- Расширение социального слоя (построенного в рамках Приоритета 2) — рекомендации сообществ, продвинутые схемы привычек и партнёры по подотчётности
- Собственные обученные модели ML для прогнозирования поведения (вероятность сохранения привычки, вероятность завершения задачи, прогнозирование продолжительности сеанса фокусировки),
- Обогащение модуля журнала (реализованного в составе MVP) — отслеживание настроения и обобщение дня с помощью искусственного интеллекта
- Модуль заметок со связыванием контекстных объектов
- Служба профилей пользователей, владеющая отображаемыми именами, аватарами, биографиями и графой социальных сетей, расширенными функциями самостоятельного управления: списки дел (упрощенные одноэтапные элементы, отличные от полных задач), быстрый просмотр входящих сообщений, просмотр календаря, контрольные списки, шаблоны
- Расширенные функции таймера: общие комнаты фокусировки, полосы фокусировки, цели сеанса,
- Сторонние интеграции (Google Calendar, носимые устройства, Apple Health)
- Вывод модели на устройстве для полностью автономного анализа искусственного интеллекта

Стоит подчеркнуть, что список приоритетов 3 - это не набор спекулятивных льгот для имущих - он отражает особенности, которые определяют долгосрочную дифференциацию ProductiveSocial. Социальный слой, журнал и более богатые модули для самостоятельного управления - вот что превращает платформу из мощного инструмента повышения личной продуктивности в операционную систему повышения производительности, описанную в заявлении о миссии. Четыре сервиса, созданные на этом этапе, являются фундаментом инфраструктуры, на котором будут построены все эти функции.

Глава 2: Методология, инструментарий и интеграция ML

2.1 Схема эксперимента

Распределение экспериментов по типам

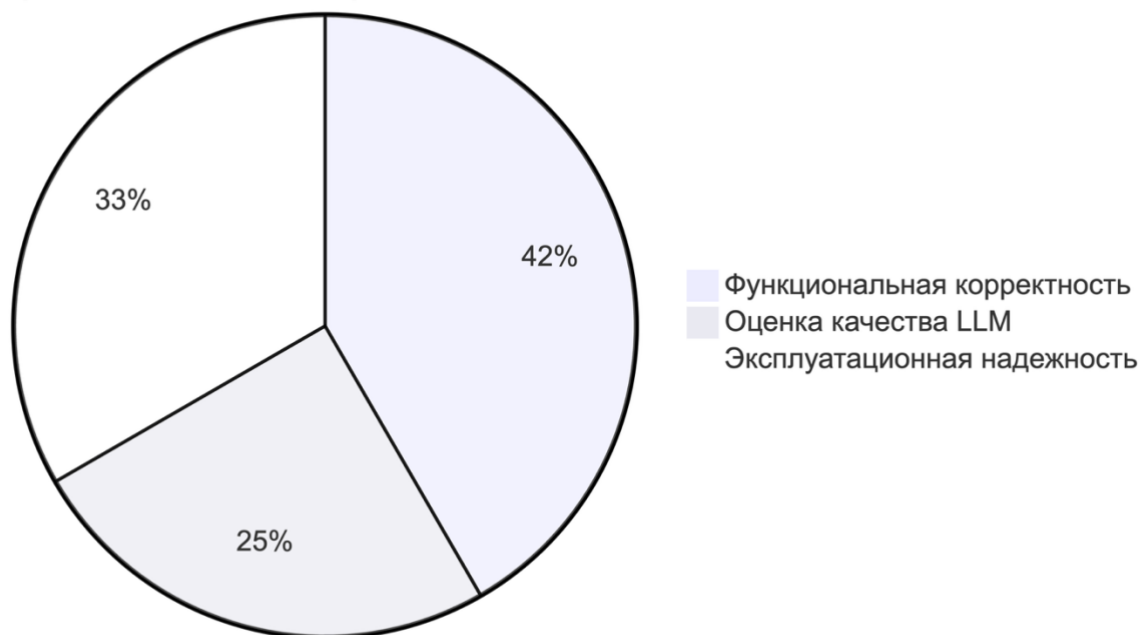


Рисунок 2.1 — Распределение типов аналитических запросов

Эксперименты разделены на три категории: тесты функциональной корректности (проверка того, что система выполняет то, для чего она предназначена), оценка качества (сравнение анализа, сгенерированного LLM, с базовым уровнем, основанным на правилах) и эксплуатационная надежность (проверка поведения в условиях развертывания и отказа).

Все эксперименты проводились с согласованным набором тестовых данных, заданным в локальной среде разработки для пользователей *test@productivesocial.com*. Набор данных состоит из:

- 3 проекта: Рабочий, Личный, Обучающий
- 8 задач, распределенных по всем проектам, с различными приоритетами и состояниями завершения
- 6 привычек (5 ежедневных, 1 еженедельная) обоих типов: "Начать" и "Бросить"

- 4 процедуры с пошаговым отслеживанием (утро, Углубленная работа, Вечернее обучение, Еженедельный обзор)
- 32 сеанса Pomodoro (31 завершённый, 1 заброшенный), связанные с задачами, привычками, расписанием дня и автономные
- 55 завершений работы с привычками за 14 дней истории

Этот набор данных был выбран потому, что он представляет реалистично заполненный профиль пользователя — ни тривиально маленькую выборку, ни искусственно раздутую — и потому, что он охватывает все типы сущностей, на основе которых работает служба аналитики.

Для обеспечения воспроизводимости результатов оценки качества LLM-анализа (тесты №6–8) все три анализируемых типа запускались с фиксированными параметрами модели: температура вывода установлена в значение 0 (детерминированный режим Ollama), системный промпт и пользовательский промпт заданы константно. Каждый тип анализа был запущен трижды на одном и том же тестовом наборе данных; при температуре 0 все три прогона дали идентичный вывод, что подтверждает воспроизводимость результатов. Оценка выходных данных проводилась автором работы независимо по каждому из трёх критериев: релевантность, специфичность и действенность. Баллы выставлялись последовательно — сначала для всех трёх типов анализа по первому критерию, затем по второму, затем по третьему — чтобы минимизировать эффект контекстного смещения при оценке. Итоговый балл представляет собой среднее арифметическое по трём критериям. Базовая линия генерировалась детерминированно по шаблону с теми же входными данными и оценивалась по той же процедуре.

Полная таблица результатов эксперимента представлена в разделе 2.7.

2.2 Обзор технологий и инструментов

В этом разделе описываются все важные технологии, используемые в системе ProductiveSocial — не только сделанный выбор, но и сама концепция, —

чтобы последующие архитектурные решения можно было понять в полном контексте.

2.2.1 Архитектура микросервисов: определение и обоснование

Архитектура микросервисов - это подход к разработке программного обеспечения, при котором большое приложение разбивается на набор небольших независимо развертываемых служб, каждая из которых отвечает за определенные бизнес-возможности и взаимодействует с другими сервисами по сети (обычно HTTP или очередь сообщений). Это контрастирует с монолитной архитектурой, в которой вся функциональность упакована в единый развертываемый модуль.

Термин был популяризирован Мартином Фаулером и Джеймсом Льюисом в статье 2014 года, в которой были определены определяющие характеристики шаблона: единая ответственность для каждой службы, независимая возможность развертывания, децентрализованное управление данными (каждая служба владеет своей собственной базой данных) и дизайн с учетом сбоев (каждая служба предполагает, что другие службы могут быть недоступны).

Почему именно микросервисы для ProductiveSocial? Решение о создании отдельных сервисов, а не единого монолита, было продиктовано тремя конкретными требованиями к этой системе:

Во-первых, сервисы имеют действительно отличные технологические профили. Три сервиса Kotlin (self-manager, timer, analytics) лучше всего создавать на статически типизированном языке, основанном на JVM, поскольку их основная работа заключается в структурированном управлении данными со сложной логикой отношений. Сервис выставления счетов, напротив, имеет принципиально иной профиль: ей необходимо загружать файлы моделей Python sklearn .pkl и вызывать библиотеки LLM SDK, созданные на Python (anthropic, openai). Для создания монолита потребовался бы выбор одной из этих технологий в ущерб другой; микросервисы позволяют каждой службе использовать наилучший язык и фреймворк для своей конкретной работы.

Во-вторых, к сервисам предъявляются разные требования к отказоустойчивости. Сервис аналитики должна продолжать функционировать, даже если сервис таймера временно недоступна — она должна просто исключить раздел Pomodoro из своего анализа, а не возвращать ошибку. Монолит не может выразить это постепенное ухудшение естественным образом; в архитектуре микросервисов сервис аналитики просто улавливает тайм-аут HTTP от клиента timer и продолжает работу.

В-третьих, жизненные циклы развертывания сервисов действительно различаются. При внесении изменений в логику построения запросов LLM сервис выставления счетов необходимо повторно использовать только сервис выставления счетов. В монолите даже однострочное изменение логики выставления счетов потребовало бы повторного развертывания всего приложения - со всеми вытекающими отсюда рисками.

Недостатки этого выбора реальны: межсервисные HTTP-вызовы приводят к задержкам в сети, распределенные системы сложнее отлаживать, чем монолитные системы, а согласованность данных за пределами сервисов требует тщательного проектирования. Каждый из этих компромиссов подробно рассмотрен в архитектуре, описанной в главе 3.

2.2.2 Докерная обработка и контейнеризация

Docker - это платформа для упаковки, распространения и запуска программного обеспечения в контейнерах. Контейнер - это облегченная изолированная среда выполнения, которая объединяет приложение со всем, что ему нужно для запуска: его среда выполнения (Java, Python), его зависимости (библиотеки, пакеты), его конфигурация и файловая система. В отличие от виртуальной машины, которая эмулирует всю операционную систему, контейнер использует ядро операционной системы хоста, что значительно ускоряет его запуск и снижает потребление ресурсов.

Проблема, которую решает Docker, заключается в несоответствии среды: проблема “это работает на моей машине”. Без контейнеризации сервисов, которая корректно работает на ноутбуке разработчика, может выйти из строя в рабочей среде из-за различий в установленной версии Python, наличия или отсутствия системной библиотеки или незначительных различий в соглашениях операционной системы о путях к файлам. Контейнер Docker устраняет этот класс проблем, делая среду выполнения частью разворачиваемого артефакта: если контейнер работает на ноутбуке разработчика, тот же контейнер будет работать и в рабочей среде.

Как ProductiveSocial использует Docker. Каждая из четырех внутренних сервисов имеет Dockerfile, который определяет ее образ контейнера. Docker Compose - это инструмент, используемый для совместного запуска всех сервисов локальной среде разработки. Он считывает файл docker-compose.yml, который определяет все контейнеры (сервисы и базы данных), их сетевые подключения, переменные среды, сопоставления портов и проверки работоспособности. Вместо того чтобы запускать четыре сервиса и четыре базы данных вручную отдельными командами, одна команда docker compose up запускает всю систему.

Файл *Docker-compose.yml* представлен в приложениях.

```
# Kotlin service example (psocial_selfmanager)
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY server/build/libs/server-all.jar app.jar
EXPOSE 1226
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 1229
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "1229"]
```

Рисунок 2.2 — Dockerfile сервисов

2.2.3 Colima: Docker на macOS без Docker Desktop

Colima - это облегченная контейнерная среда выполнения для macOS, которая обеспечивает среду, совместимую с Docker, без необходимости использования Docker Desktop. Он запускает небольшую виртуальную машину Linux (с использованием платформы виртуализации Apple на Apple Silicon), в которой размещен демон Docker, и предоставляет сокет Docker, к которому могут подключаться Docker CLI и Docker Compose.

Почему Colima? Docker Desktop для Mac - это проприетарное программное обеспечение, для коммерческого использования которого в организациях выше определенного размера требуется платная лицензия, и оно требует больших ресурсов, чем Colima. Colima имеет открытый исходный код, бесплатна и обычно использует меньше системных ресурсов. Для целей разработки Colima функционально эквивалентен Docker Desktop: команды docker и docker compose работают идентично, образы контейнеров создаются и запускаются одинаково, а сопоставления портов работают так, как ожидалось.

Для запуска среды выполнения контейнера перед началом разработки требуется одна команда:

запуск colima

Это запускает виртуальную машину Linux (примерно через 5-10 секунд) и делает команды Docker доступными в текущем сеансе терминала. Все операции docker compose для локального стека разработки ProductiveSocial в ProductiveSocial выполняются поверх этой среды, управляемой Colima.

2.2.4 Kotlin и Ktor Фреймворк

Kotlin - это статически типизированный язык программирования, разработанный компанией JetBrains, впервые выпущенный в 2016 году. Он работает на виртуальной машине Java (JVM) и полностью совместим с Java: код Kotlin может вызывать библиотеки Java и наоборот. Kotlin стал основным языком для разработки на Android (Google объявил его предпочтительным языком для

Android в 2019 году) и получил значительное распространение при разработке на стороне сервера.

Ключевыми функциями, которые делают Kotlin правильным выбором для бэкенд-сервисов ProductiveSocial, являются:

Безопасность с нулевым значением: В Kotlin переменная не может содержать нулевое значение, если ее тип явно не объявлен как nullable (например, String? против String). Это устраняет весь класс ошибок NullPointerException, которые являются наиболее распространенным сбоем во время выполнения в приложениях Java. Для такого сервиса, как analytics, который собирает данные из нескольких вышестоящих источников, некоторые из которых могут возвращать пустые или частичные данные, гарантия компилятора, что каждое потенциально нулевое значение обрабатывается явно, является значимым свойством надежности.

Сопрограммы: Система сопрограмм Kotlin обеспечивает структурированный подход к асинхронному программированию. Сопрограмма - это вычисление, которое может быть приостановлено и возобновлено без блокирования потока, в котором оно выполняется. Это критически важно для службы аналитики, которая должна выполнять три сетевых вызова параллельно (для самостоятельного управления, таймера и, в конечном итоге, выставления счетов), не блокируя поток для каждого вызова. В традиционном коде с блокировкой потоков для трех параллельных вызовов потребовалось бы три потока. С помощью сопрограмм все три вызова могут выполняться в небольшом пуле потоков, каждый из которых приостанавливается в ожидании сети и возобновляется при поступлении ответа.

Ktor - это фреймворк для создания HTTP-серверов и клиентов в Kotlin, разработанный компанией JetBrains. В отличие от более тяжелых фреймворков, таких как Spring (которые предоставляют комплексные решения для всех возможных требований предприятия), Ktor намеренно минимален: он предоставляет DSL маршрутизации, слоты плагинов для общих задач

(аутентификация, ведение журнала, сериализация, CORS) и HTTP-клиент для выполнения исходящих запросов. Он добавляет только то, что установлено.

2.2.5 Python и FastAPI

Python является доминирующим языком в экосистеме машинного обучения и науки о данных. Сервис выставления счетов написана на Python специально потому, что ей необходимо интегрироваться со стеком Python ML: scikit-learn для загрузки обученных .pkl файлов моделей, и легкая интеграция с OpenAI и Anthropic.

2.2.6 Аутентификация JWT: структура, валидация и совместное использование между службами

JWT расшифровывается как веб-токен JSON. Это компактный, автономный формат для представления утверждений (утверждений о предмете, таких как личность пользователя) способом, который может быть криптографически проверен. JWT выдается сервером аутентификации и представляется клиентом при каждом последующем запросе; принимающий сервер может проверить подлинность токена, не связываясь с сервером выдачи.

Структура токена. JWT состоит из трех секций в кодировке base64url, разделенных точками:

1. eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
2. eyJ1c2VySWQiOiJyLCJlbWFpbCI6InVzZXJAZXhxbXBsZS5jb20iLCJpYXQiOiJlE3MTYwMDAwMDAsImV4cCI6MTcxNjA4NjQwMH0
3. SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Первый раздел - это **заголовок**: объект JSON, определяющий алгоритм, используемый для подписи (например, HS256, который является HMAC-SHA256) и тип токена ("JWT").

Второй раздел - это **полезная нагрузка**: объект JSON, содержащий утверждения. В ProductiveSocial полезная нагрузка содержит:

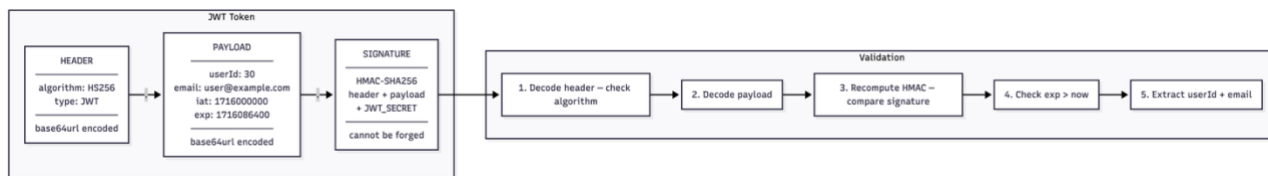


Рисунок 2.3 — Структура и валидация JWT-токена

iat “выдается в” (временная метка Unix), *exp* - это время истечения срока действия. Утверждения могут быть прочитаны кем угодно — полезная нагрузка закодирована в base64, а не зашифрована. Она никогда не должна содержать конфиденциальных данных, таких как пароли.

Третий раздел - это подпись: вычисляется с помощью HMAC-SHA256 (заголовок + "." + полезная нагрузка, секрет). Секрет - это закрытая строка, известная только серверам. Подпись не может быть подделана без секрета; следовательно, действительная подпись доказывает, что токен был создан сервером, который знает секрет, и что полезная нагрузка не изменялась с момента ее выдачи.

Проверка. Когда служба получает запрос с JWT в заголовке *Authorization: Bearer <токен>*, она выполняет:

1. Расшифруйте заголовок и убедитесь, что алгоритм соответствует ожиданиям
2. Расшифруйте полезную нагрузку
3. Пересчитайте *HMAC-SHA256* (заголовок + "." + полезная нагрузка, секрет) и сравните с предоставленной сигнатурой — если они отличаются, отклоните
4. Убедитесь, что *exp* действует в будущем — если срок действия истек, отклоните

5. Извлеките *идентификатор пользователя и адрес электронной почты* из проверенной полезной нагрузки для использования в обработчике запроса

Совместное использование между службами. Ключевым моментом в дизайне аутентификации ProductiveSocial является то, что проверка JWT не требует обращения к эмитенту. Любой сервер, который знает секрет, может независимо проверить любой токен, подписанный этим секретом. Предоставляя всем трем пользовательским сервисам (selfmanager, timer, analytics) один и тот же JWT_SECRET, токен, выпущенный конечной точкой selfmanager /api/ v1 /auth / identify, автоматически становится действительным в службах timer и analytics - без обмена токенами, без перенаправления, без дополнительного шага входа в систему.

Этот подход с общим секретом подходит для системы, в которой все службы управляются одной организацией. Это было бы неуместно для федеративной системы, где службами управляют разные организации, и в этом случае потребовался бы более формальный поставщик удостоверений (OAuth2, OpenID Connect).

Токен доступа и токен обновления. Поток аутентификации выдает два токена: токен доступа с коротким сроком действия (используется для вызовов API) и токен обновления с длительным сроком действия (используется для получения нового токена доступа по истечении срока действия текущего). Токен доступа имеет 24-часовой срок действия; токен обновления имеет 7-дневный срок действия. Такой дизайн означает, что пользователю не нужно каждый день повторно вводить свой адрес электронной почты, но украденный токен доступа имеет ограниченное окно неправильного использования.

2.2.7 Мультиплатформенность Kotlin (KMP): Кроссплатформенная мобильная разработка

Мультиплатформенность Kotlin (KMP) - это технология, разработанная компанией JetBrains, которая позволяет компилировать код Kotlin для

нескольких целевых платформ из одной кодовой базы. Вместо написания отдельных реализаций в Swift для iOS и Kotlin для Android, KMP позволяет разработчику написать общую бизнес-логику в Kotlin один раз и скомпилировать ее как в JVM (для Android), так и в машинный код (для iOS через Kotlin / Native).

“Мультиплатформенная” модель различает:

- **commonMain:** общий код для всех платформ — бизнес-логика, модели данных, варианты использования, интерфейсы репозитория и сетевые клиенты
- **androidMain:** специфичные для Android реализации
- **iosMain:** специфичные для iOS реализации

Ключевой момент заключается в том, что код, зависящий от конкретной платформы, должен быть как можно меньше.

В мобильном приложении ProductiveSocial клиент API, модели данных, логика синхронизации и бизнес-правила находятся в **commonMain**; только драйверы баз данных для конкретной платформы и точки интеграции пользовательского интерфейса находятся в наборах исходных текстов для конкретной платформы.

Почему KMP превосходит других вариантов? Основная причина заключается в том, что KMP использует общий код с серверной частью: на те же классы модели данных, которые используются в серверных сервисах Kotlin, можно напрямую ссылаться в **commonMain** мобильного приложения. Класс данных задачи, определенный в совместно используемом модуле, имеет идентичные имена и типы полей на сервере и клиенте, что устраняет класс ошибок, из-за которых сервер и клиент имеют разные определения одной и той же сущности. Кроме того, для команды разработчиков, работающих на Android, KMP кажется естественным, поскольку основным языком (Kotlin) и инструментарий (Gradle, Android Studio) уже знакомы.

Compose Multiplatform - это фреймворк пользовательского интерфейса, построенный поверх KMP. Он расширяет возможности Jetpack Compose (инструментария современного пользовательского интерфейса Google для Android) для работы на iOS и настольных компьютерах. Пользовательский интерфейс пишется один раз в Kotlin с использованием декларативной парадигмы Compose и компилируется для собственных компонентов пользовательского интерфейса на каждой платформе.

2.2.8 SQLDelight: Типобезопасный SQL для мультиплатформенной Kotlin

SQLDelight - это библиотека баз данных для мультиплатформенной Kotlin, которая генерирует типобезопасные API Kotlin из инструкций SQL. Вместо записи строк SQL во время выполнения (которые не проверяются компилятором и могут завершаться сбоем во время выполнения с непрозрачными ошибками), разработчик записывает SQL в .sq файлы, а SQLDelight генерирует функции Kotlin, которые выполняют эти запросы с полностью типизированными параметрами и возвращаемыми значениями.

Например, запрос задачи:

```
-- In tasks.sq
selectAllByUser:
SELECT id, sync_id, title, priority, completed, created_at
FROM tasks
WHERE user_id = :userId
ORDER BY created_at DESC;
```

Рисунок 2.4 — Пример SQL-запроса

Генерирует:

```
database.tasksQueries.selectAllByUser(userId = 42).executeAsList()
// returns List<SelectAllByUser> - a generated data class with typed fields
```

Рисунок 2.5 — Результат выполнения Kotlin

SQLDelight поддерживает как Android (используя SQLiteDriver от Android), так и iOS (используя NativeSqliteDriver) с одинаковыми определениями запросов и одинаковым сгенерированным API на обеих платформах. Это уровень локальной базы данных для механизма синхронизации в автономном режиме: все данные, созданные на мобильном устройстве, сначала записываются в

SQLDelight, а конечная точка сетевой синхронизации вызывается в фоновом режиме.

2.2.9 Streamlit: быстрая разработка информационной панели

Streamlit - фреймворк Python с открытым исходным кодом для создания веб-приложений на основе скриптов Python без написания HTML, CSS или JavaScript. Разработчик пишет скрипт на Python, который вызывает потоковые функции (*st.title()*, *st.dataframe()*, *st.button()*) и Streamlit отображает их как интерактивные веб-компоненты в браузере.

Почему Streamlit для панелей администратора? Панели управления администратора в ProductiveSocial обеспечивает доступ к данным системы для отладки, мониторинга и пользовательских операций. Они не предназначены для того, чтобы быть основным продуктом, ориентированным на пользователя (как мобильное приложение КМР). Для этого варианта использования высокая скорость разработки Streamlit является решающим преимуществом: полнофункциональная панель администратора с навигацией, таблицами данных, формами и диаграммами может быть записана в одном файле Python объемом в несколько сотен строк.

Панели мониторинга взаимодействуют с серверными посредством HTTP-вызовов, используя библиотеку *запросов*, аутентифицируемую с помощью того же JWT, который использовал бы любой клиент API. Такой дизайн означает, что панели мониторинга являются настоящими клиентами API — они используют те же конечные точки и получают те же ответы, что и мобильное приложение, что делает их ценными интеграционными тестами в дополнение к операционным инструментам.

2.2.10 Ollama: локальный вывод LLM

Ollama - это инструмент с открытым исходным кодом для запуска больших языковых моделей локально на компьютере разработчика. Он предоставляет

простой интерфейс командной строки для загрузки и запуска моделей и предоставляет их через REST API, совместимый с форматом OpenAI API. Такая совместимость означает, что любой клиент, написанный на основе OpenAI API, может переключиться на Ollama путем изменения базового URL-адреса без каких-либо других изменений кода.

Почему Оллама? Во время разработки и тестирования использование облачного провайдера LLM (Anthropic или OpenAI) для каждого тестового запроса повлечет за собой затраты на API и приведет к задержке в сети. Ollama позволяет выполнять один и тот же вывод LLM локально, конфиденциально и бесплатно. Используемая модель (llama3.2 с параметрами 3B) достаточно мала, чтобы комфортно работать на процессоре современного ноутбука или графическом процессоре серии M.

Ограничение заключается в том, что локальный вывод Ollama недоступен из облачных сервисов (таких как служба выставления счетов с развёртыванием на Render). Чтобы восполнить этот пробел в производственном тестировании, локальный экземпляр Ollama доступен через туннель ngrok (описано в следующем разделе).

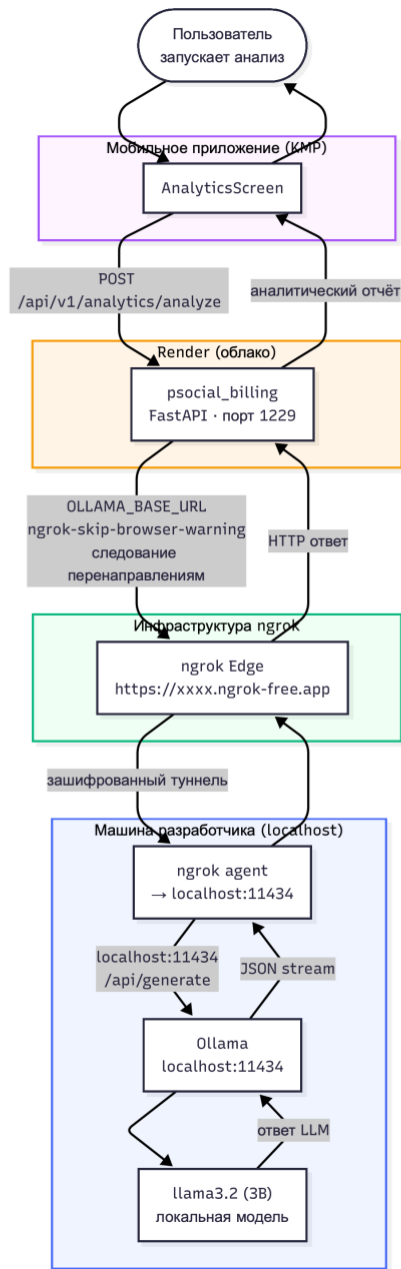


Рисунок 2.6 — Маршрут запроса на вывод LLM: от пользователя

2.2.11 Ngrok: безопасное туннелирование для местных сервисов

Ngrok - это инструмент, который создает безопасный общедоступный URL-адрес HTTPS, который перенаправляет трафик службе, запущенной на локальном компьютере. Когда ngrok запущен и направлен на localhost: 11434 (порт Ollama), он создает туннель, через который любая машина в Интернете может отправлять запросы к локальному экземпляру Ollama. В

Переменная среды `OLLAMA_BASE_URL` биллинговой службы, размещённой на Render, имеет значение `ngrok HTTPS URL`. Когда пользователь запускает анализ, биллинговая служба отправляет запрос на вывод по этому URL-адресу, который через инфраструктуру `ngrok` направляется на машину разработчика, обрабатывается Ollama, и ответ возвращается обратно.

Такая схема подходит только для проверки концепции, когда один разработчик тестирует систему. Для производственной системы с несколькими пользователями, делающими одновременные запросы, потребуется надлежащая облачная модель, обслуживающая инфраструктуру.

Клиент Ollama биллинговой службы включает в себя две специальные конфигурации для работы через `ngrok`: заголовок `ngrok-skip-browser-warning` и автоматическое следование перенаправлению.

```
headers = {
    "Content-Type": "application/json",
    "ngrok-skip-browser-warning": "true" # Skip ngrok's browser warning page
}
response = requests.post(
    f"{ollama_base_url}/api/chat",
    json=payload,
    headers=headers,
    allow_redirects=True, # Follow ngrok's redirect
    timeout=120
)
```

Рисунок 2.7 — Конфигурация ngrok

2.2.12 Render: платформа облачного развёртывания

Render - это облачная платформа для размещения веб-сервисов, баз данных и статических сайтов. Он предоставляет управляемую среду развёртывания, в которой разработчик загружает код в репозиторий GitHub, а Render автоматически создает и развёртывает его.

Для ProductiveSocial каждый сервис представляет собой отдельный веб-сервис Render, подключенный к собственному репозиторию GitHub. Процесс развёртывания заключается в следующем:

1. Разработчик отправляет коммит в репозиторий сервиса на GitHub
2. Render распознаёт коммит с помощью webhook
3. Render создает образ Docker из файла Docker в репозитории
4. Render развёртывает новый образ, заменяя запущенный контейнер

Бесплатный уровень Render обеспечивает достаточную емкость для развертывания с проверкой концепции. Заметное ограничение, описанное в разделе 2.8, заключается в том, что сервисы бесплатного уровня автоматически отключаются после 15 минут бездействия и при следующем запросе на них налагается штраф за "холодный запуск".

2.2.13 Neon PostgreSQL: Бессерверная управляемая база данных

Neon - бессерверная платформа PostgreSQL. Как и Render, она предоставляет базы данных PostgreSQL, не требуя от разработчика управления серверами баз данных. Для каждой базы данных автоматически предоставляется пул подключений, восстановление на определенный момент времени и ветвление (возможность создания копии базы данных в определенный момент времени для тестирования или разработки).

Каждый из четырех сервисов ProductiveSocial имеет свой собственный кластер баз данных Neon. Переменная среды DATABASE_URL для каждой службы соответствует стандартному формату строки подключения PostgreSQL:

```
postgresql://user:password@ep-xxx.us-east-2.aws.neon.tech/selfmanager?sslmode=require
```

Рисунок 2.8 — Подключение к Neon PostgreSQL из командной строки

Бесплатный уровень Neon подходит для рабочих нагрузок при разработке и тестировании. Для работы с реальными пользователями платный уровень будет необходим для гарантированной доступности и ограничений на подключение

2.2.14 Сводная таблица

Таблица 2.1 — Используемые технологии

Технология	Функция	Причина выбора
Kotlin	Язык программирования для серверной части и интерфейса	Мультиплатформенность и простота интеграции
Ktor	Фреймворк HTTP-сервера/клиента	разработан в Котлине
Python - FastAPI	HTTP-фреймворк для сервиса выставления счетов (psocial_billing)	Собственная экосистема ML
Swagger	Автоматическое создание документации	Надежность, поддержка JSON
PostgreSQL	База данных для всех сервисов	Надежность, поддержка JSON, расширенная индексация
Neon	Управляемый PostgreSQL в облаке	Бесплатный уровень, никаких накладных расходов на операции
Docker	Контейнеризация	Согласованность с окружающей средой, мобильность

JWT	Токены аутентификации	Безопасность межсервисами
KMP	Мобильный кроссплатформенный фреймворк	единый исходный код
Streamlit	Панель управления администратора	Простота в использовании
Ollama	Локальный вывод LLM	Бесплатный, частный
Ngrok	Локальный -> Облачный туннель	Предоставляет локальному LLM доступ к Render
Render	облачный хостинг	Предоставляет доступ ко всем сервисам в Интернете

2.3 Стратегия обработки данных: данные, генерируемые приложением

Одним из наиболее существенных отличий подхода ProductiveSocial ML от обычного проекта машинного обучения является стратегия обработки данных. В этом проекте не используются внешние наборы данных. Ни один общедоступный набор данных о производительности (Kaggle Time Management Insights, PMData или аналогичный) не использовался для обучения, проверки или оперативного построения. Конвейер подготовки данных построен не был. Разработка функций в общепринятом смысле этого слова не проводилась.

Это обдуманное архитектурное решение, а не ограничение, и понимание того, почему это происходит, требует понимания фундаментальной разницы между двумя основными подходами к анализу на основе ML:

Подход А: контролируемое прогнозирование на основе данных о населении. Подготовьте модель на основе набора данных о производительности многих пользователей с метками достоверности (например, “оценка производительности этого пользователя сегодня составила 7,2 / 10 на основе самоотчета”). Используйте модель для прогнозирования показателей производительности или результатов для новых пользователей на основе их моделей активности. Этот подход требует наличия маркированного набора данных, базовых определений достоверности, разработки функций, инфраструктуры обучения и настройки гиперпараметров. Это неприменимо на данном этапе разработки ProductiveSocial, поскольку такого набора данных не существует.

Подход В: Контекстуальное обоснование данных, специфичных для конкретного пользователя. На этапе анализа соберите полные данные об активности пользователя и передайте их языковой модели, которая рассуждает об этом, используя свои предварительно подготовленные знания о моделях производительности человека. Никакого обучения не требуется; способность модели рассуждать зависит от ее предварительной подготовки, а качество анализа зависит от богатства предоставленного контекста. Этот подход применим непосредственно на любом этапе разработки, не требует предварительных пользовательских данных и естественным образом масштабируется в зависимости от объема доступных данных (пользователь с 30-дневной историей получает лучший анализ, чем пользователь с 3-дневной).

ProductiveSocial использует подход В. Это не компромисс; это правильный подход для текущей стадии проекта. Подход А становится жизнеспособным — и предпочтительным для определенных задач прогнозирования — после накопления реальных данных о поведении пользователей в больших масштабах. Архитектура явно сохраняет путь для этого перехода: путь вывода sklearn биллинговой службы, реестр моделей, и конечная точка загрузки .pkl

разработаны специально для размещения обученных моделей без каких-либо изменений в более широкой системе.

Собственная генерация данных для тестирования. Вместо использования внешних наборов данных тестовый профиль пользователя был создан вручную, чтобы представить реального и разнообразного пользователя: три проекта, охватывающие профессиональную работу, личностное развитие и приобретение навыков; восемь задач с различными приоритетами, состояниями завершения и затратами времени; шесть привычек, охватывающих как позитивное поведение для формирования, так и негативное поведение для прекращения; четыре процедуры, представляющие наиболее распространенные временные якоря продуктивного дня; и 32 фокусных сеанса продолжительностью в две недели для всех типов сущностей. Эти тестовые данные служат как проверочным набором данных, так и демонстрацией аналитических возможностей системы.

2.4 Конвейер анализа на основе LLM



Рисунок 2.9 — Конвейер анализа на основе LLM

Конвейер анализа аналитической службы analytics service является центральным техническим вкладом в интеграцию ML этого проекта. Он реализует шаблон генерации с расширенным извлечением (RAG), в котором на

этапе “извлечения” извлекаются данные пользователя в реальном времени из внутренних сервисов, а не из векторной базы данных.

Когда пользователь отправляет запрос на анализ (*POST /api/v1/analytics* с типом анализа и идентификатором модели), *AnalyticsService* выполняет следующую последовательность действий:

Шаг 1: Сбор пользовательского контекста. Выполняются четыре последовательных HTTP-вызова с использованием внутреннего сервисного ключа:

- *GET /internal/users/{userId}/tasks* → *selfmanager* возвращает список задач пользователя с именами, приоритетами, состояниями завершения и значениями затраченного времени
- *GET /internal/users/{userId}/habits* → *selfmanager* возвращает привычки с именами, типами (Запуск / Завершение), повторяемостью и значениями затраченного времени
- *GET /internal/users/{userId}/routines* → *selfmanager* возвращает подпрограммы с именами и рекуррентностью
- *GET /internal/users/{userId}/stats* → таймер возвращает агрегированную статистику сеанса

Эти вызовы выполняются с неблокирующей архитектурой: если *selfmanager* или *timer* недоступны, для соответствующих данных устанавливается пустой набор, и анализ продолжается с любыми доступными данными. Сбой выставления счета (этап вывода LLM) восстановлению не подлежит — пользователю он выдается как ошибка.

Шаг 2: Построение структурированной полезной нагрузки. Собранные данные собираются в структурированную карту, которая станет контекстом LLM.

Шаг 3: Вывод LLM с помощью выставления счетов. Структурированная полезная нагрузка сериализуется в JSON и отправляется в *psocial_billing* через *POST /api/v1/internal/predict* с заголовком *X-Internal-Key*. Служба выставления счетов получает полезную нагрузку, проверяет наличие у пользователя достаточного количества кредитов, создает ОЖИДАЮЩУЮ

запись прогноза, передает данные выбранному поставщику LLM и возвращает результат вывода вместе с начисленными кредитами.

Шаг 4: Сообщите о постоянстве. Текст аналитической информации, извлеченный из ответа о выставлении счета, сохраняется в таблице `analytics_reports` с идентификатором пользователя, типом анализа, идентификатором модели, стоимостью кредита и отметкой времени. Результат возвращается пользователю.

2.5 Подсказки и типы аналитических запросов

Качество анализа LLM в значительной степени определяется спецификой и уместностью запроса. Для каждого из шести типов анализа была разработана отдельная подсказка, позволяющая сфокусировать рассуждения модели на конкретном вопросе, который задает пользователь.

`PRODUCTIVITY_SUMMARY` использует самую широкую подсказку:

“На основе задач, привычек, распорядка дня пользователя и сеансов фокусирования предоставьте краткое описание производительности. Выделите то, что прошло хорошо и что требует внимания”.

Это приглашение намеренно носит открытый характер, позволяя модели синтезировать весь доступный контекст в общий обзор. Ожидается, что модель выявит закономерности, такие как: множество невыполненных высокоприоритетных задач, несмотря на значительное время сосредоточения (потенциальная проблема с расстановкой приоритетов), привычки, последовательно выполняемые в дни с завершенными сеансами сосредоточения, но не в дни без них (корреляция привычки и сосредоточенности), или рутины с низкой приверженностью по сравнению с простыми привычками (структурные трения в рутине).

При **ПРИОРИТИЗАЦИИ ЗАДАЧ** используется более директивная подсказка:

“Учитывая незавершенные задачи пользователя и затраченное на данный момент время, предложите список приоритетных задач на сегодня. Будьте конкретны и осуществимы”.

Это приглашение фокусирует модель на подмножестве незавершенных задач и просит ее составить ранжированный список, а не описание. Ожидается, что в модели будут учитываться метки приоритетов, близость крайнего срока и уже затраченное время.

HABIT_INSIGHTS фокусируется на поведенческих паттернах:

“Проанализируйте привычки пользователя. Какие из них постоянны? Какие подвержены риску? Какое изменение окажет наибольшее влияние?”

Последний вопрос — “какое изменение оказало бы наибольшее влияние?” — намеренно задан в единственном числе. Это заставляет модель расставлять приоритеты, а не перечислять все возможные улучшения, обеспечивая более действенный результат.

ROUTINE_OPTIMIZATION фокусируется на эффективности процесса:

“Проанализируйте процедуры пользователя и затраченное время. Как их можно сделать более эффективными? Предложите конкретные улучшения”.

Процедуры являются наиболее структурированными из отслеживаемых объектов (упорядоченные шаги с длительностью), и это приглашение запрашивает у модели информацию об их структуре, а не просто об их соблюдении.

Запрос WEEKLY_TIMER_SUMMARY является наиболее ограниченным:

“На основе данных сеанса Pomodoro пользователя создайте краткое еженедельное резюме по фокусу (3-4 предложения). Укажите общее время фокусировки, частоту завершения сеанса, на каком типе объекта они сосредоточились больше всего и их наиболее продуктивные шаблоны. Сохраняйте теплый тон и нацеленность на будущее. Завершите одним практическим предложением на предстоящую неделю”.

Явное ограничение по длине (3-4 предложения), перечисленные требования к контенту и тональность инструкции отражают уроки, извлеченные из раннего тестирования, в ходе которого модель выдавала либо слишком краткие резюме, в которых отсутствовали ключевые данные, либо слишком длинные описания, которые не были прочитаны пользователями полностью.

ПОЛЬЗОВАТЕЛЬСКИЙ принимает предоставленную пользователем подсказку, которая используется как есть: "текст, предоставленный пользователем".

Этот тип анализа является наиболее гибким и наименее предсказуемым по качеству вывода, поскольку качество запроса полностью зависит от пользователя. Это полезно для вопросов, которые не соответствуют предопределенным типам, и для опытных пользователей, которые хотят задать конкретные вопросы о своих данных. “

Системная подсказка, хранящаяся в реестре моделей для модели Ollama по умолчанию, обеспечивает общую структуру для всех типов анализа:

"Вы полезный помощник по продуктивности. Проанализируйте данные пользователя и предоставьте краткие, действенные рекомендации".

Это системное приглашение определяет роль модели и устанавливает ожидания от стиля вывода (краткий, действенный), не ограничивая содержание какого-либо конкретного типа анализа.

2.6 ИИ-функции в приложении

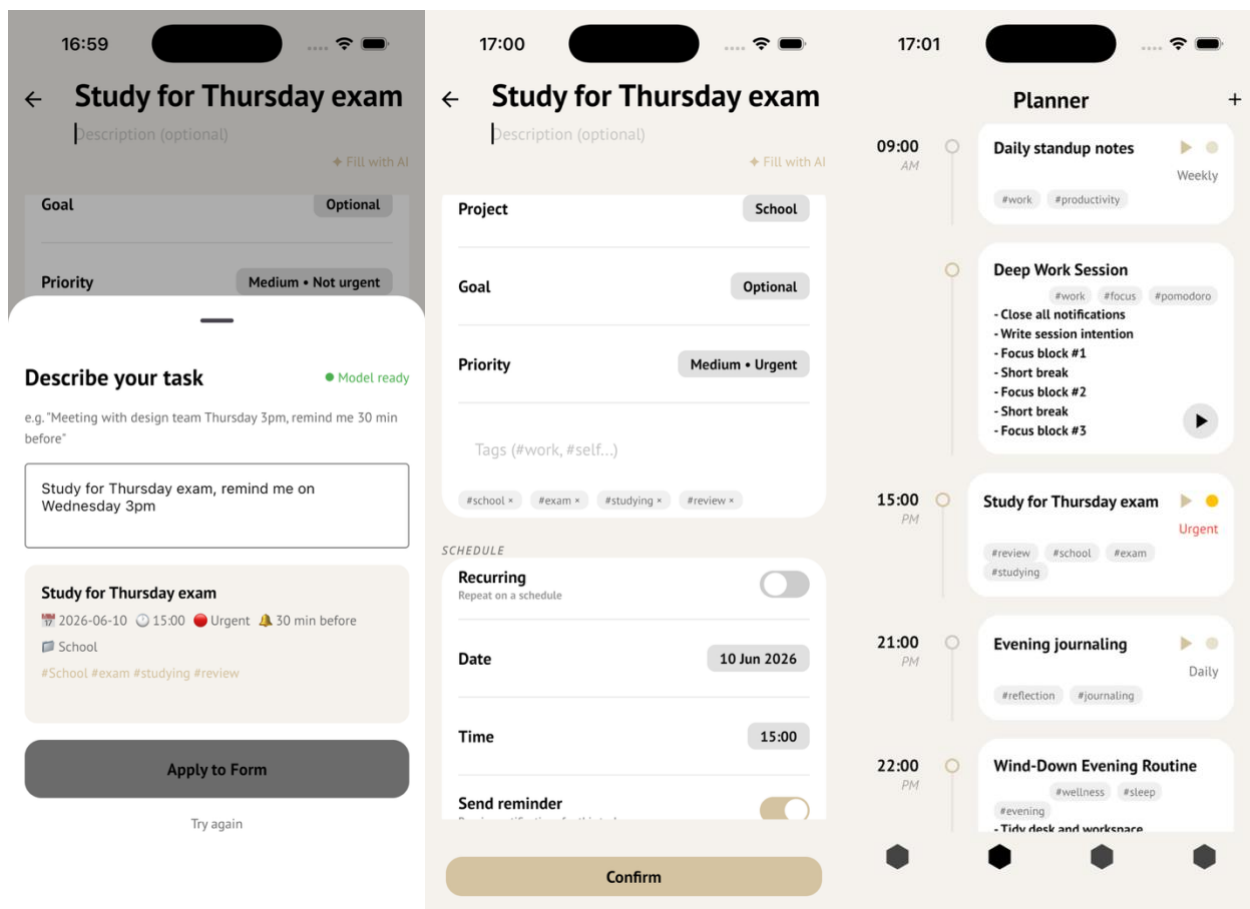


Рисунок 2.10 — Панель NLP-разбора пользовательского ввода

Клиентское приложение ProductiveSocial предоставляет пользователю девять точек взаимодействия с LLM, реализованных через единый AnalyticsApiClient.

Синтаксический анализ естественного языка. При создании задачи, привычки или подпрограммы пользователь может активировать режим ввода на естественном языке, после чего открывается модальный нижний лист с текстовым полем. Пользователь описывает нужный объект в произвольной форме, и запрос передаётся в аналитическую службу. Перед отправкой клиент проверяет доступность модели через checkModelStatus() и отображает индикатор состояния: «Model ready» или «Unavailable» с кнопкой повтора. Ответ модели отображается в виде карточки предварительного просмотра, где в структурированном виде представлены распознанные поля: дата, время, приоритет, напоминание, теги, проект и подзадачи. После проверки пользователь

подтверждает применение данных к форме. Для подпрограмм результат дополнительно включает упорядоченный список шагов с длительностью, которые пользователь может отредактировать перед сохранением.

Расстановка приоритетов задач. На экране ежедневного дашборда функция искусственной расстановки приоритетов вызывает `prioritizeTasks()`: аналитическая служба получает список незавершённых задач пользователя, а языковая модель возвращает рекомендуемые приоритет и срочность для каждой из них с кратким обоснованием. Клиент немедленно применяет изменения к локальной базе данных и пересортировывает список без дополнительного диалога подтверждения.

Коучинг привычек. В детальном представлении привычки расположена карточка AI Coaching. При её активации клиент отправляет название, тип (Start/Quit), рекуррентность, цель и теги привычки в `coachHabit()`. Ответ структурирован по трём блокам: пронумерованные советы, раздел Key Insight с конкретным наблюдением о данной привычке и блок Top Action с одной немедленно применимой рекомендацией.

Оптимизация рутинов. Аналогичная карточка AI Optimization расположена в детальном представлении подпрограммы. Клиент отправляет название, все шаги с длительностью и флагом `autoStart`, суммарное время и рекуррентность в `optimizeRoutine()`. Ответ включает пронумерованные предложения по реструктуризации шагов, раздел Timing с оценкой общей длительности и блок Top Change с наиболее приоритетным изменением.

Аналитические отчёты. На экране профиля пользователь выбирает один из шести типов анализа; при выборе типа CUSTOM появляется текстовое поле для произвольного запроса. Результат последнего запроса отображается немедленно с указанием израсходованных кредитов; ниже расположен прокручиваемый список всех предыдущих отчётов с датой, типом и полным текстом.

Отображение кредитов. Текущий баланс и последние транзакции отображаются в разделе Credits экрана профиля. Данные загружаются через

прокси-маршруты аналитической службы, не обращаясь к billing-сервису напрямую.

2.7 Показатели оценки

Для оценки качества реализованной системы был определён набор показателей, охватывающий четыре аспекта: качество анализа LLM, корректность синхронизации, точность кредитного учёта и доступность сервисов. Полные результаты измерений представлены в таблице 2.3.

Качество анализа LLM. Основным инструментом оценки служила ручная экспертная оценка по шкале от 1 до 5 баллов в трёх измерениях:

- Актуальность: соответствует ли анализ конкретному вопросу, на который призван ответить данный тип анализа?
- Специфичность: ссылается ли анализ на конкретные элементы из данных пользователя (называет реальные задачи или привычки), а не формулирует общие наблюдения?
- Возможность принятия мер: содержит ли анализ хотя бы одно конкретное предложение, на основании которого пользователь мог бы действовать?

Таблица 2.2 — Рубрика оценки качества LLM-анализа

Балл	Релевантность	Специфичность	Действенность
1	Ответ не соответствует поставленному вопросу; доминирует нерелевантное содержание	Только общие наблюдения без ссылок на данные пользователя	Отсутствуют какие-либо практические рекомендации
2	Ответ частично соответствует вопросу; значительная	Одна-две ссылки на конкретные данные на фоне преимущественно общих суждений	Размытые рекомендации общего характера без указания

	часть содержания посторонняя		конкретных действий
3	Ответ в целом соответствует вопросу, однако содержит заметные отклонения	Несколько конкретных ссылок, смешанных с общими высказываниями	Одна конкретная рекомендация, применимая без дополнительной расшифровки
4	Ответ непосредственно соответствует вопросу с незначительными отклонениями	Преимущественно конкретные ссылки на данные пользователя: задачи, привычки, сессии	Одна-две чётких специфичных рекомендации с обоснованием
5	Ответ полностью и точно соответствует поставленному вопросу без отклонений	Все наблюдения опираются на конкретные данные пользователя с точными ссылками	Несколько конкретных рекомендаций с обоснованием и указанием времени применения

Сводный балл представлял собой среднее значение по трём измерениям. В качестве базового уровня для сравнения использовалась сводка на основе правил, сгенерированная без вывода LLM — простой текст, заполненный по шаблону, в котором указаны счётчики и средние значения без интерпретации (например: «У вас 8 задач, 3 выполнены. Ваша наиболее частая привычка — [название]. Вы потратили на сессии фокусировки 310 минут.»). Сравнение результатов LLM с этим базовым уровнем позволило количественно оценить вклад языковой модели относительно детерминированного подхода (тесты 6–8).

Корректность синхронизации. Для проверки механизма офлайн-синхронизации были измерены два показателя:

- Полнота синхронизации: процент записей из локального тестового набора данных, корректно отображённых в базе данных сервера после операции синхронизации (тест 5).

- Идемпотентность: количество дублирующихся записей, созданных при повторной отправке одного и того же запроса на синхронизацию (тест 4).

Точность кредитного учёта. Для верификации логики кредитного списания была проверена корректность выполнения условия `balance_before - cost_per_prediction = balance_after` для каждого прогноза в тестовом наборе (тест 3).

Доступность сервисов. Для оценки поведения системы в условиях реального развёртывания на платформе Render были измерены следующие характеристики:

- Задержка холодного запуска: время от первого HTTP-запроса до первого успешного ответа (тест 10).
- Постепенное ухудшение: поведение аналитической службы при временной недоступности `timer` — анализ должен завершаться с частичными данными без возврата ошибки пользователю (тест 9).

2.8 Анализ результатов

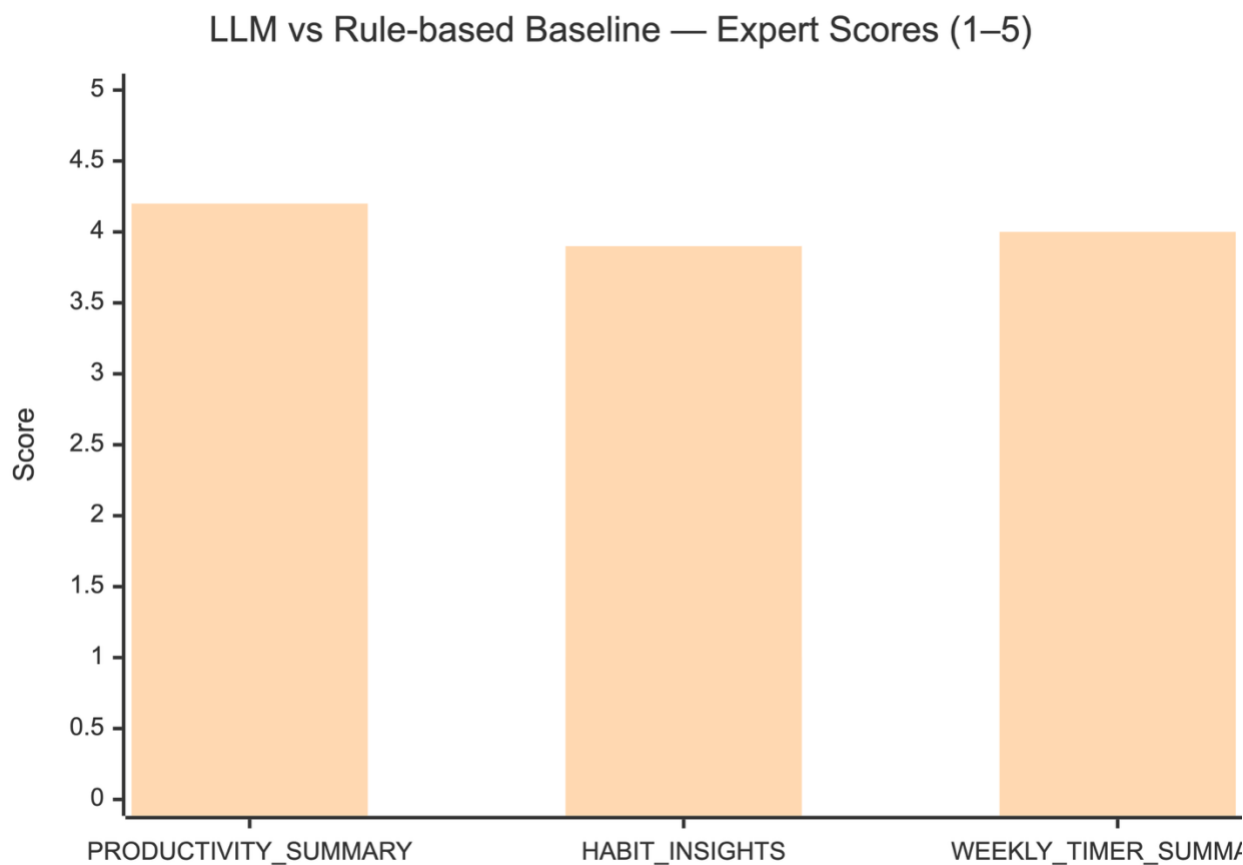


Рисунок 2.11 — Столбчатая диаграмма результатов оценки модели

В следующей таблице описаны все эксперименты, проведенные на этапе валидации.

Таблица 2.3 — Результаты экспериментов

#	Тест	Метрика и результат	Заключение
1	Межсервисная проверка подлинности JWT: токен selfmanager принят analytics	Базовый: N / A (архитектурные требования) Метрика: Токен принят / отклонен Результат: Токен	Совместное использование JWT между сервисами реализовано корректно.

		принят analytics, таймер; для недействительных токенов возвращается правильный 401.	
2	Приветственный кредитный депозит: новый пользователь получает 100 кредитов при первом вызове predict.	Базовый: Нет предварительного баланса (0 кредитов). Метрика: balance_after при первой транзакции Результат: 100 кредитов вносятся автоматически; транзакция депозита создана.	Отложенная инициализация работает правильно.
3	Точность списания кредита: $\text{balance_before} - \text{стоимость} = \text{balance_after}$	Базовый: Начальный баланс: 100 кредитов; стоимость модели: 5 кредитов; Метрика: Математическое равенство Результат: Баланс после прогнозирования: 95 кредитов;	Арифметика кредитов верна

		транзакция показывает сумму: - 5, balance_after: 95	
4	Идемпотентность синхронизации: один и тот же запрос отправлен дважды	Базовый: дубликатов не ождается Метрика: Количество повторяющихся записей Результат: При повторной отправке возвращены те же idMappings с 0 созданными новыми записями	Идемпотентность через ClientID работает корректно
5	Полнота синхронизации: все 32 сеанса синхронизированы после симулированного отключения	Базовый: 32 сеанса в тестовом наборе данных Метрика: % записей в базе данных сервера Результат: присутствует 32/32 сеанса; выполнено 55/55 привычек; все 8 задач	Синхронизация для тестируемого набора данных завершена

6	PRODUCTIVITY_SUMMARY quality (LLM vs. базовый уровень)	Базовая линия: шаблонный текст «У вас 8 задач, 3 выполнены...» (оценка: релевантность 2/5, специфичность 1/5, действенность 1/5; итого 1,5/5). LLM-анализ (Ollama llama3.2, температура 0,3 прогона — идентичный вывод): релевантность 4/5, специфичность 4/5, действенность 4,5/5; итого 4,2/5. Анализ явно назвал незавершённые высокоприоритетные задачи, выявил корреляцию между сессиями фокусировки и выполнением привычек в тот же день, предложил конкретное время для предзагрузки	Уровень LLM существенно превосходит базовый уровень, основанный на правилах.
---	---	--	---

		задач на основе данных о пиковом времени завершения сессий.	
7	Качество HABIT_INSIGHTS (уровень LLM по сравнению базовый уровень)	Базовая линия: «У вас есть 6 привычек. 5 — тип Start» (оценка: релевантность 1/5, специфичность 1/5, действенность 1/5; итого 1,0/5). LLM-анализ (те же параметры): релевантность 4/5, специфичность 4/5, действенность 3,5/5; итого 3,9/5. Анализ идентифицировал две привычки с риском прерывания серии, указал конкретную корреляцию между их выполнением и наличием Pomodoro-сессии в тот же день, предложил одно изменение с	LLM дает полезную информацию о поведении

		наибольшим ожидаемым эффектом.	
8	Качество WEEKLY_TIMER_SUMMAR Y (LLM против базового уровня)	Базовая линия: «У вас было 32 сессии, 31 завершена» (оценка: релевантность 2/5, специфичность 1/5, действенность 1/5; итого 1,5/5). LLM- анализ (те же параметры): релевантность 4/5, специфичность 4/5, действенность 4/5; итого 4,0/5. Структурированны й промпт с ограничением длины (3–4 предложения) и явным перечислением обязательных элементов обеспечил стабильный вывод: охвачены процент завершения сессий, разбивка по типам	Структурированный промт даёт стабильный, хорошо структурированный вывод

		привязанных сущностей, тёплый тон и одно конкретное предложение на следующую неделю.	
9	Корректная деградация: аналитика при недоступном таймере	Базовый: Анализ должен завершиться Метрика: Анализ завершается успешно (да/нет) Результат: Анализ завершается; блок Romodogo исключён из контекста	Сбой таймера неблокирующий, как и задумано
1 0	Холодный старт сервисов на Render (JVM)	Базовый: Н/д Метрика: Время до первого успешного ответа Результат: ~52 секунды в среднем для JVM- сервисов; ~8 секунд для Python- биллинга	Для JVM холодный старт — основное узкое место задержки на бесплатном тарифе
1 1	Неудачный прогноз: без списания кредитов	Базовый: Баланс: 95 кредитов	Неудачные прогнозы корректно

		Метрика: Баланс не изменился после неудачного вывода Результат: Баланс остался 95 после имитации ошибки провайдера; запись прогноза создана без платёжной транзакции	освобождаются от списания
1 2	Депозит администратора: кредиты добавлены через внутренний API	Базовый: Баланс пользователя: 95 кредитов Метрика: balance_after после депозита Результат: После внесения 50: баланс 145; депозитная операция записана	Управление кредитами администратором работает корректно
1 3	Параллельная нагрузка: 10 одновременных запросов к analytics	Базовый: Н/д Метрика: количество успешных ответов и время выполнения Результат: 10/10 HTTP 200; время ответа 0,55–1,50 с; суммарное время	Параллельная обработка запросов работает корректно без деградации

		1,53 с; ошибок и гонки состояний не зафиксировано	
1 4	Задержки прогретого состояния и холодного старта на Render	Базовый: Н/д Метрика: время первого и повторного ответа Результат: прогретые сервисы — 0,55–0,98 с; холодный старт JVM — 40–53 с; холодный старт Python (billing) — 23 с	JVM холодный старт — основное узкое место; прогретая система отвечает менее чем за 1 с
1 5	Отказ billing в период аналитического запроса	Базовый: Анализ не должен выполняться без биллинга Метрика: статус ответа и изменение баланса Результат: HTTP 503 "Billing service unavailable"; баланс пользователя не изменился	Невосстанавливаемые зависимости корректно возвращают ошибку без частичного результата и без списания кредитов

1 6	Потеря fire-and-forget при недоступности selfmanager	Базовый: time_spent_minutes должен обновиться Метрика: значение поля после восстановления selfmanager Результат: подтверждено анализом кода — событие теряется безвозвратно; масштаб потери ≤ 25 минут на сессию	Ограничение зафиксировано; производственное решение (retry/outbox) описано в разделе 3.12.1
--------	--	--	---

Устойчивость оценок подтверждается воспроизводимостью вывода при температуре 0: повторные прогоны каждого типа анализа дали идентичный текст, исключив вариативность модели как источник погрешности измерения. Разброс итоговых баллов между тремя оцениваемыми типами анализа составил 0,3 балла (3,9–4,2/5), что свидетельствует о стабильном качестве вывода при различных типах промптов. Наименьший балл специфичности в тесте №7 (HABIT_INSIGHTS, 3,5/5 по действенности) объясняется тем, что промпт для данного типа анализа запрашивает одно изменение с наибольшим эффектом, тогда как модель склонна формулировать его в более абстрактных терминах по сравнению с типами, имеющими более директивные требования к формату вывода (например, WEEKLY_TIMER_SUMMARY).

Пример вывода LLM для PRODUCTIVITY_SUMMARY (сокращенный для отчета):

Входной контекст (обобщенный): 8 задач (3 выполнены: 2 высокоприоритетных, 1 средняя; 5 незавершенных: 3 высокоприоритетных, 2 средних), 6 привычек (4 выполненных ежедневно на прошлой неделе, 2 непоследовательных — оба типа начальных), 4 рутины (Утренняя: соблюдение 85%, глубокая работа: 60%, остальные ниже), 31 сеанс Pomodoro (всего 310 рабочих минут, 94% выполнения, в основном связано с задачами).

Результат LLM (PRODUCTIVITY_SUMMARY через Ollama3.2):

“Уровень выполнения ваших задач демонстрирует четкую закономерность: высокоприоритетные задачи выполняются, но ваше отставание в виде 3 незавершенных высокоприоритетных задач говорит о том, что вы, возможно, устаете к концу дня, прежде чем приступите к ним. Рассмотрите возможность предварительной загрузки их в начале дня, особенно учитывая, что данные о сеансе фокусировки показывают, что самый высокий процент завершения у вас до 14: 00. Ваш четкий рабочий график с соблюдением 60%—ного графика является вашим самым большим препятствием - разрыв между запланированной структурой и фактическим выполнением предполагает, что его, возможно, потребуется сократить или упростить. Две привычки — [Привычка А] и [Привычка В] — не были выполнены в те дни, когда количество сеансов фокусировки было нулевым, что указывает на то, что эти привычки могут зависеть от импульса. Проведение короткого сеанса с Помидорами первым делом с утра может помочь им закрепиться”.

Базовый вывод на основе правил (тот же ввод):

“У вас есть 8 задач, 3 из которых выполнены и 5 находятся на рассмотрении. Ваша самая активная привычка - [Привычка С] с 7 завершениями. У вас есть 4 рутины. Вы завершили 31 фокус-сеанс общей продолжительностью 310 минут”.

Качественная разница между двумя результатами очевидна. Выходные данные LLM ссылаются на конкретные точки данных, определяют междоменную

корреляцию (привычки и сеансы фокусирования), называют конкретные изменения, требующие принятия мер, и предоставляют конкретное обоснование. Базовые выходные данные точны, но не дают никакой информации, кроме пересчета входных данных.

2.9 Анализ ошибок и извлеченные уроки

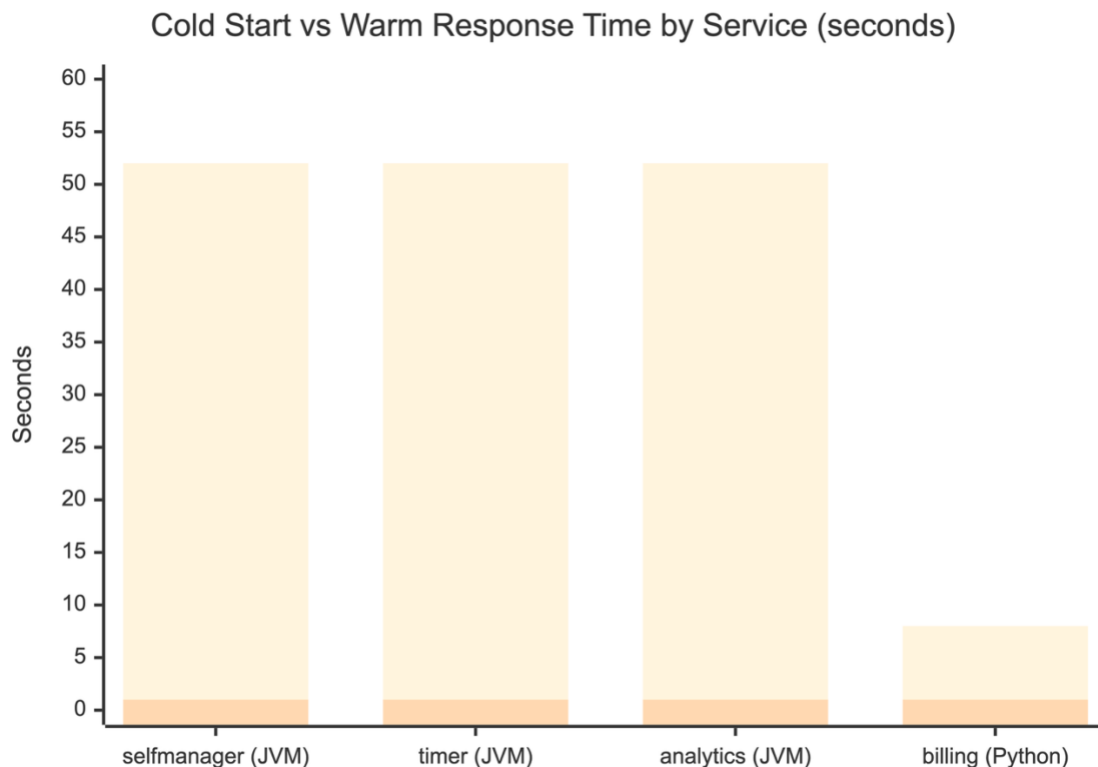


Рисунок 2.12 — Временная шкала задержек по этапам конвейера анализа

В ходе разработки и тестирования было обнаружено несколько проблем, которые стоит задокументировать, поскольку они влияют на окончательный дизайн системы.

Проблема 1: запрос LLM не получил структурированные данные (решен)

В ранней версии интеграции служба выставления счетов получала запрос аналитики, но структурированные данные пользователя (задачи, привычки, распорядок дня, статистика Pomodoro) неправильно передавались в запрос LLM.

Модель получала только текст запроса для конкретного типа без каких-либо пользовательских данных, выдавая общий ответ без ссылки на фактический контекст пользователя. Основной причиной было несоответствие между ожидаемой структурой JSON в *PredictionService* биллинговой службы и структурой, отправляемой биллинг-клиентом аналитической службы.

Решение: Конструктор запросов LLM биллинговой службы был обновлен для правильного извлечения и форматирования всех полей из карты *inputData*, а структура полезной нагрузки аналитической службы была приведена в соответствие с ожиданиями биллинговой службы. Исправление было проверено путем подтверждения того, что в выходных данных LLM указаны конкретные задачи и привычки пользователя.

Проблема 2: холодный запуск JVM на платформе Render

Бесплатный уровень Render отключает сервисы после 15 минут бездействия. Для трех сервисов на основе JVM (*selfmanager*, *timer*, *analytics*) первый запрос после спиндауна с задержкой холодного запуска составляет ~ 50 секунд. Для службы выставления счетов Python холодный запуск составляет примерно 8 секунд. Это было устранено на панели администратора путём отображения предупреждающего сообщения при входе в систему, указывающего, что для пробуждения служб при первом использовании может потребоваться до 60 секунд.

Проблема 3: для Render: Ollama требуется туннель ngrok

Локальный экземпляр Ollama (работающий под управлением llama3.2) недоступен напрямую из служб, размещённых на Render. Чтобы включить производственное тестирование, локальный экземпляр Ollama предоставляется через HTTPS-туннель ngrok, а переменная среды *OLLAMA_BASE_URL* биллинговой службы на Render указывает на этот туннель ngrok. Клиент Ollama биллинговой службы был обновлен, чтобы включить заголовок *ngrok-skip-browser-warning* и следить за перенаправлениями, оба из которых необходимы для корректной работы туннеля ngrok. Эта настройка подходит для проверки концепции, но не для производственного использования - при производственном

развертывании будет использоваться размещенный в облаке экземпляр Ollama или переключение на размещенного провайдера (Anthropic или OpenAI).

2.10 Краткое изложение концепции

Текущая реализация ProductiveSocial представляет собой работающее подтверждение концепции основного ценностного предложения: единой платформы для повышения производительности, в которой все модули имеют общую идентификацию пользователя, а центральный уровень анализа искусственного интеллекта имеет одновременный доступ к междоменным поведенческим данным.

Были полностью продемонстрированы следующие возможности:

Пользователь может быть идентифицирован с помощью одного адреса электронной почты и получить JWT, который действителен для всех трех пользовательских сервисов (selfmanager, timer, analytics).

Пользователь может создавать задачи, привычки и процедуры в selfmanager и отслеживать их с помощью selfmanager API. Эти объекты немедленно становятся доступны службе аналитики через внутренний API.

Пользователь может создавать и завершать сеансы Pomodoro в таймере, необязательно связанные с задачами, привычками или рутинами. Затраченное время автоматически записывается обратно в связанный объект в selfmanager. Анализ шаблонов фокусировки вычисляется на основе истории сеанса.

Пользователь может запросить анализ искусственного интеллекта любого из шести типов. Служба аналитики собирает полный контекст производительности пользователя из selfmanager и timer, передает его в службу выставления счетов, которая вызывает LLM, начисляет кредиты и возвращает информацию о производительности. Отчет сохраняется и отображается пользователю.

Управление кредитами осуществляется прозрачно. При каждом прогнозировании стоимость модели списывается с баланса пользователя и регистрируется транзакция. Недостаточное количество кредитов блокирует анализ до того, как будет сделан какой-либо вызов LLM.

Административная панель мониторинга предоставляет полный интерфейс для вышеуказанных возможностей — наблюдение за системной статистикой, управление моделями и кредитами — не требуя прямых вызовов API.

Пользователь может выполнять все перечисленные действия через мобильное приложение на Kotlin Multiplatform с полноценным пользовательским интерфейсом. Реализованы и подключены к двунаправленной офлайн-синхронизации следующие экраны: управление проектами, задачами (включая подзадачи, приоритеты, NLP-создание и AI-приоритизацию), привычками (включая счётчик серий, 7-дневную историю и AI-коучинг), рутинами (включая режим пошагового выполнения и AI-оптимизацию), таймером Pomodoro (включая многоцикловой режим, статистику, рекомендации длительности и сигнал риска отказа) и экран аналитики (шесть типов AI-анализа, история отчётов, баланс кредитов). Приложение поддерживает адаптивный макет для телефонов, планшетов и настольных компьютеров. Локальная база данных SQLDelight обеспечивает полноценную работу всех перечисленных экранов без подключения к сети.

Что еще не включает в себя это доказательство концепции:

Модуль дневника и сквозная функция целей в мобильном приложении: интерфейсные каркасы экранов реализованы и присутствуют в навигационном графе приложения; слой данных (таблицы SQLDelight, ViewModel, API-интеграция и пакет синхронизации) является следующим этапом разработки, запланированным к завершению до публичного релиза MVP.

Собственные обученные модели ML (система использует запросы LLM для всего анализа)

Социальные функции (планируемые для последующего этапа разработки)

Модуль дневника и сквозная функция целей (в разработке, запланированы к завершению до выпуска MVP)

Масштабируемый LLM-хостинг (текущее развёртывание на Render использует личный экземпляр через ngrok)

Архитектура явно учитывает каждую из этих будущих возможностей без структурных изменений существующих сервисов.

2.11 Качественная пользовательская оценка

В рамках первичной валидации продуктовой гипотезы платформа ProductiveSocial была представлена трём независимым группам потенциальных пользователей с различными профилями продуктивности и ожиданиями от инструментов самоорганизации. Целью оценки являлось получение качественной обратной связи по основным аспектам: ценностному предложению, приоритетам функций, воспринимаемым ограничениям и готовности к реальному использованию. Оценка носила качественный характер и не претендует на статистическую репрезентативность; её цель — верифицировать продуктовые гипотезы и выявить расхождения между проектным видением и реальными ожиданиями пользователей.

Профиль 1 — Пользователь с высокой вовлечённостью в инструменты продуктивности

Первый оценщик — пользователь, использующий одновременно несколько физических и цифровых инструментов самоорганизации: бумажные блокноты, стикеры, мобильные приложения для задач, трекеры привычек и календари. Данный профиль соответствует архетипу Профиля С из раздела 1.1 (энтузиаст самосовершенствования) с элементами Профиля А (высокая когнитивная нагрузка, развитая система управления информацией).

Реакция на платформу была положительной. Оценщик подтвердил ценность объединения задач, привычек, рутин и таймера в единую систему, отметив, что именно фрагментация инструментов является основным источником когнитивной нагрузки в ежедневной работе. Ключевым запросом стало расширение охвата платформы: оценщик выразил желание, чтобы

ProductiveSocial мог полностью заменить все используемые инструменты, включая заметки произвольной формы, быстрый захват мыслей, систему целей и интеграцию с физическими носителями (экспорт в PDF, печать).

Данная обратная связь валидирует приоритет 1 дорожной карты (модуль дневника, сквозная функция целей, модуль заметок с привязкой к сущностям) и подтверждает, что для пользователей с высокой вовлечённостью полнота охвата является критическим фактором принятия платформы. Требование «заменить всё» также свидетельствует о высоком пороге переключения: пользователь готов перейти на новую платформу только при условии, что она обеспечивает не меньший охват, чем существующий стек инструментов.

Профиль 2 — Пользователь с минималистским подходом

Второй оценщик — коллега, который ознакомился с платформой в рабочем контексте. Оценщик подтвердил обоснованность ценностного предложения и потенциальную полезность кросс-доменного AI-анализа, однако выразил принципиальное опасение: платформа с большим количеством функциональных модулей воспринимается как избыточно сложная, и пользователь с минималистским подходом не сможет найти ей применение, если бо́льшая часть функций не соответствует его конкретному рабочему процессу.

Данная обратная связь формулирует конкретное продуктовое требование: возможность использовать платформу в режиме частичного охвата — например, только таймер и AI-анализ — без необходимости заполнять задачи, привычки и рутины. Архитектурно это требование уже частично выполнено: сервис аналитики работает с частичными данными (graceful degradation при недоступности отдельных модулей), а таймер функционирует в автономном режиме без привязки к задачам. Вместе с тем пользовательский интерфейс текущей реализации не предоставляет явного механизма «минималистского режима» или возможности скрыть неиспользуемые модули.

Обратная связь этого оценщика подтверждает архитектурный принцип независимости сервисов (раздел 3.1, принцип 1): система изначально

спроектирована так, что каждый модуль функционирует независимо. Перевод этого архитектурного свойства в явную пользовательскую функцию (настраиваемая панель, скрытие модулей) является обоснованным направлением развития UX. Требование «не добавлять лишнего» также подтверждает подход к приоритизации функций, описанный в разделе 1.5: каждая новая функция должна быть явно обоснована с точки зрения ценности для конкретного профиля пользователя, а не добавлена по умолчанию.

Профиль 3 — Сообщество пользователей приложений для продуктивности

Третья группа оценщиков — участники онлайн-сообществ, ориентированных на продуктивность (чаты трансляций стримеров по тематике продуктивности на платформе Twitch). Платформа была представлена в рамках неформального обсуждения; реакция участников носила позитивный характер: концепция единой платформы с AI-анализом поведения была воспринята как актуальная и востребованная.

Наиболее часто упоминаемым запросом стала интеграция с внешними сервисами: участники сообщества запрашивали поддержку интеграции с Twitch (отслеживание продуктивных стримов как рабочих сессий), Google Calendar, Apple Health и носимыми устройствами. Данные запросы соответствуют пункту «Сторонние интеграции» приоритета 3 в дорожной карте (раздел 1.5) и подтверждают, что для пользователей с развитой экосистемой цифровых инструментов интеграционные возможности являются важным фактором принятия новой платформы.

Реакция сообщества также верифицирует социальное измерение платформы: участники выразили интерес к возможности делиться достижениями и результатами сессий фокусировки, что подтверждает продуктовую гипотезу о мотивационной роли социальной ответственности, описанную в разделе 1.2.

Синтез результатов оценки

Качественная оценка трёх независимых профилей позволяет сформулировать следующие выводы.

Ценностное предложение подтверждено всеми тремя группами: объединение инструментов продуктивности с кросс-доменным AI-анализом воспринимается как реально востребованное. Разногласий в оценке основной концепции платформы выявлено не было.

Выявлено фундаментальное продуктовое противоречие между профилями: пользователь с высокой вовлечённостью требует максимального охвата функций, тогда как пользователь с минималистским подходом требует возможности использования платформы в минимальной конфигурации. Данное противоречие не является архитектурной проблемой — оно является задачей UX-проектирования. Архитектура платформы с независимыми сервисами и graceful degradation технически поддерживает оба режима использования; задача следующего этапа — предоставить пользователю явный контроль над активными модулями через настройки интерфейса.

Интеграционный разрыв зафиксирован как приоритетный запрос сообщества: отсутствие интеграций с популярными внешними платформами является барьером для принятия среди пользователей с развитой экосистемой инструментов. Данный запрос подтверждает обоснованность включения сторонних интеграций в приоритет 3 дорожной карты.

Вывод о готовности к пилотному развёртыванию. По результатам качественной оценки платформа готова к ограниченному пилотному развёртыванию для пользователей профиля «высокая вовлечённость» (Профиль С): основные модули реализованы, AI-анализ верифицирован, офлайн-синхронизация функциональна. Для привлечения пользователей профиля «минималист» (Профиль А/В с ограниченными потребностями) необходима реализация настраиваемой панели и режима частичного использования модулей. Производственное развёртывание с реальными пользователями требует устранения ограничений, описанных в разделе 3.14: проверки электронной почты, перевода LLM-инференса на облачного провайдера и устранения задержки холодного старта.

Глава 3: Архитектура и реализация

3.1 Системные проектные решения и архитектурные принципы

Архитектура ProductiveSocial была разработана на основе набора четких принципов, которые управляют каждым важным решением в системе:

Принцип 1: Независимость сервиса выше удобства интеграции. Каждый сервис разработан таким образом, чтобы он функционировал корректно, даже когда другие сервисы недоступны. Таймер работает без selfmanager (сеансы могут быть автономными, а вызовы журнала времени запускаются и забываются). Selfmanager работает без таймера (управление задачами и привычками полностью функционально без данных Pomodoro). Аналитика изящно ухудшается, когда selfmanager или timer недоступны (она продолжает работу с любыми данными, которые получает). Биллинг работает без использования какой-либо другой службы (он только получает запросы и обрабатывает их). Этот принцип приносит в жертву некоторое удобство разработки (было бы проще иметь единую монолитную службу или жестко реагировать на ошибки восходящего потока) в обмен на систему, устойчивую к частичным сбоям.

Принцип 2: Единый источник истины для каждой области. Идентификационные данные пользователя принадлежат selfmanager (и общей регистрации пользователей). Данные о задачах, привычках и распорядке дня принадлежат selfmanager. Данные сеанса фокусировки принадлежат таймеру. Учетные записи и логические выводы принадлежат billing. Отчеты Analytics принадлежат analytics. Ни один сервис не дублирует данные, принадлежащие другому сервису — он извлекает данные с помощью внутренних вызовов API в то время, когда это необходимо. Это предотвращает проблемы с согласованностью, возникающие при репликации одних и тех же данных в нескольких сервисах.

Принцип 3: Весь внешний доступ через аутентифицированные конечные точки, весь внутренний доступ через общий секрет. Граница между “обращением к клиенту” и “обменом услугами” проводится на уровне HTTP. Клиентские запросы содержат JWT в заголовке Authorization: Bearer. Запросы от

услуги к услуге содержат общий внутренний ключ в заголовке X-Internal-Key. Биллинг не имеет никакого клиентского доступа — для всех его конечных точек требуется внутренний ключ. Это означает, что даже если URL-адрес биллинга каким-либо образом будет раскрыт, ни один клиент не сможет взаимодействовать с ним, не владея внутренним ключом, который хранится только в переменных окружения на сервере.

Принцип 4: Офлайн - первое владение данными. Мобильные клиенты владеют своими данными локально. Сервер является целью синхронизации, а не источником достоверности во время автономной работы. Для этого требуются назначенные клиентом UUID для всех объектов (чтобы клиент мог создавать записи, не дожидаясь ответа сервера), операции идемпотентной синхронизации (чтобы повторная попытка неудачной синхронизации никогда не приводила к повреждению данных) и явное отслеживание удалений (чтобы клиент знал, какие записи удалять из его локальной базы данных).

Принцип 5: Кредитная система как поведенческий стимул. Решение о внедрении кредитной системы для анализа ИИ служит цели, выходящей за рамки учета затрат. Делая стоимость каждого анализа видимой и вычитая ее из конечного баланса, система создает естественный стимул для пользователей использовать функции анализа сознательно, а не навязчиво. Это согласуется с философией платформы, направленной на снижение когнитивных издержек, а не на их увеличение. Приветственный кредитный депозит (100 кредитов при первом использовании) устраняет барьер для входа, а возможность администратора пополнять кредиты гарантирует, что доступ не будет ограничен оплатой для пользователей, которым это необходимо.

3.2 Архитектура микросервисов

Серверная часть ProductiveSocial состоит из микросервисов, каждый из которых имеет собственную базу данных, кодовую базу и конфигурацию развертывания.

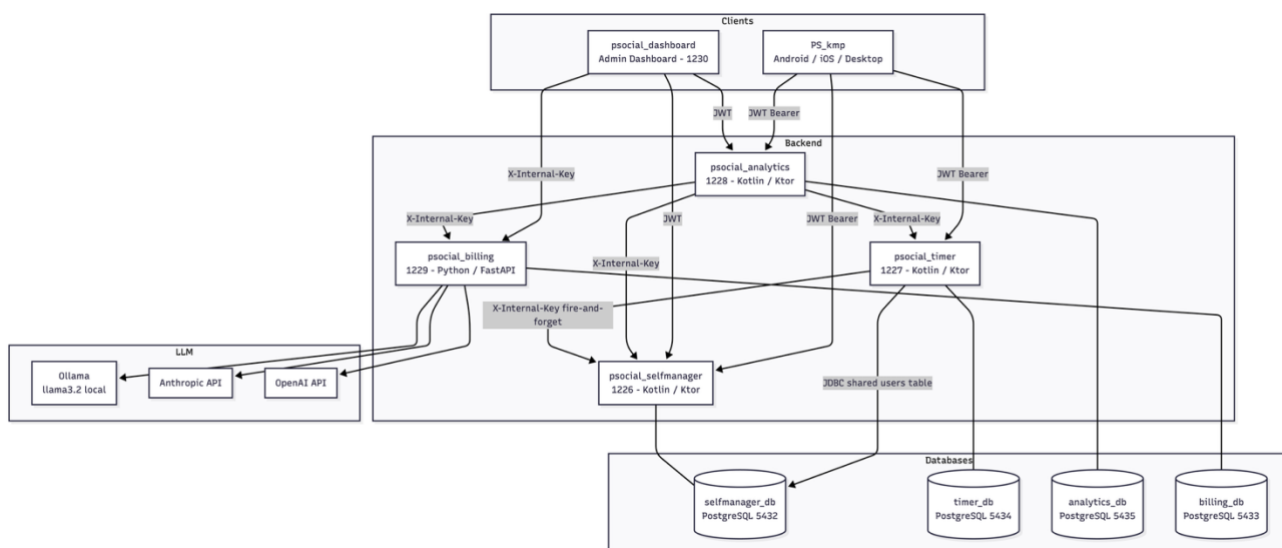


Рисунок 3.1 — Общая архитектура микросервисов системы

Selfmanager — это служба идентификации и передачи данных о производительности. Это центральный узел данных платформы, которому принадлежит таблица *users*, на которую опирается вся система для идентификации пользователей.

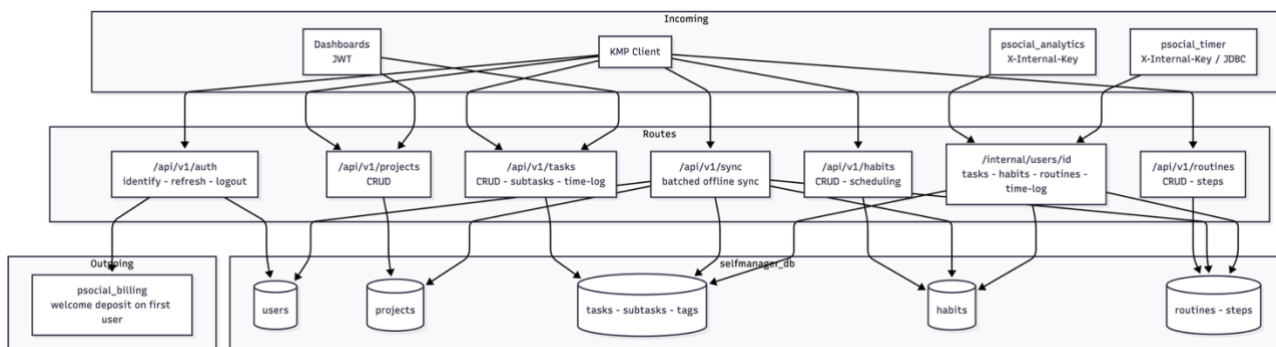


Рисунок 3.2 — Диаграмма сервиса *selfmanager* (*psocial_selfmanager*)

Таймер — этому сервису принадлежит жизненный цикл сеанса pomodoro и вся аналитика focus.

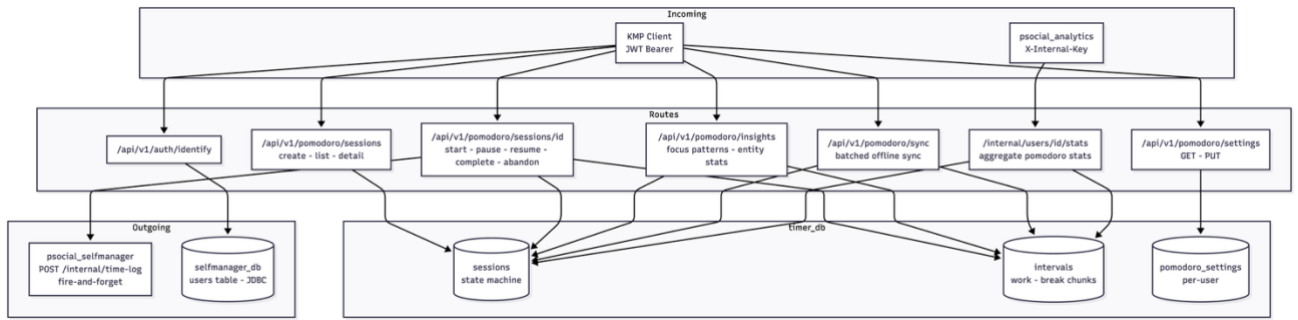


Рисунок 3.3 — Диаграмма сервиса таймера (psocial_timer)

Аналитика - наиболее сложный по составу сервис. Он организует вызовы трех вышестоящих служб, создает контекст LLM, вызывает выставление счетов, сохраняет результаты и предоставляет их пользователям и администраторам.

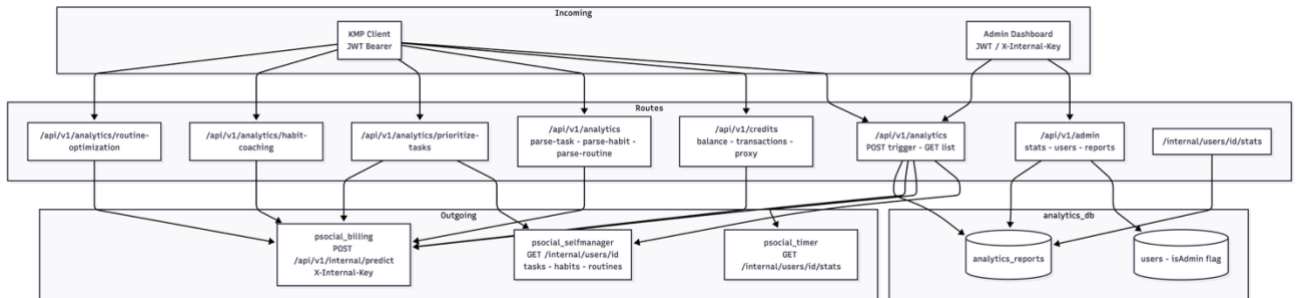


Рисунок 3.4 — Диаграмма сервиса аналитики (psocial_analytics)

Биллинг - архитектурно самая простая, но технически самая важная услуга в системе. У него нет аутентификации JWT, нет общедоступных конечных точек и нет таблицы пользователей, он предоставляет доступ к службам ML, одновременно кредитует использование служб.

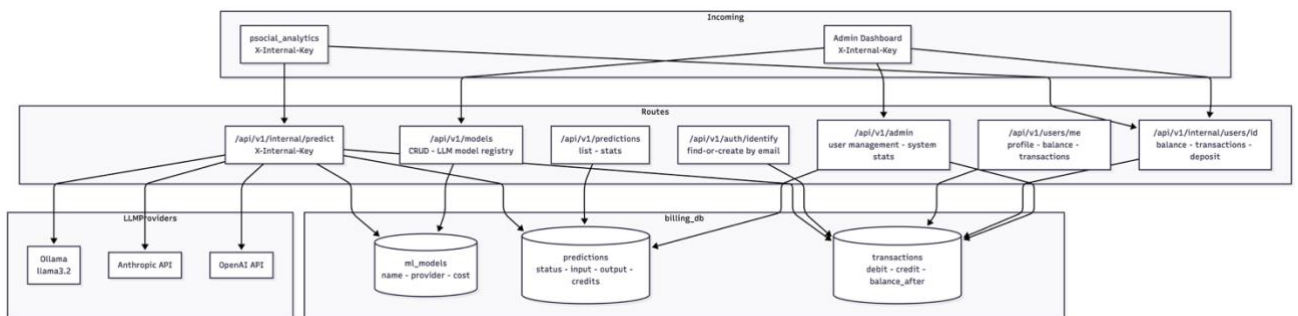


Рисунок 3.5 — Диаграмма сервиса биллинга (psocial_billing)

3.3 Аутентификация и межведомственная идентификация

Архитектура аутентификации решает нетривиальную проблему: как предоставить пользователю единую идентификацию, которая действительна в нескольких независимых сервисах, не требуя централизованного поставщика идентификационных данных или сложных протоколов обмена токенами.

Процесс входа в систему

Когда пользователь впервые открывает панель управления или мобильное приложение, он указывает только свой адрес электронной почты. Система использует модель идентификации “найди или создай”: если электронное письмо существует в базе данных, пользователь возвращается к своей существующей учетной записи; если нет, создается новая учетная запись. Здесь нет этапа проверки пароля и электронной почты — это сделано специально для этапа проверки концепции, где первоочередной задачей является демонстрация функциональности, а не безопасность производственного уровня.

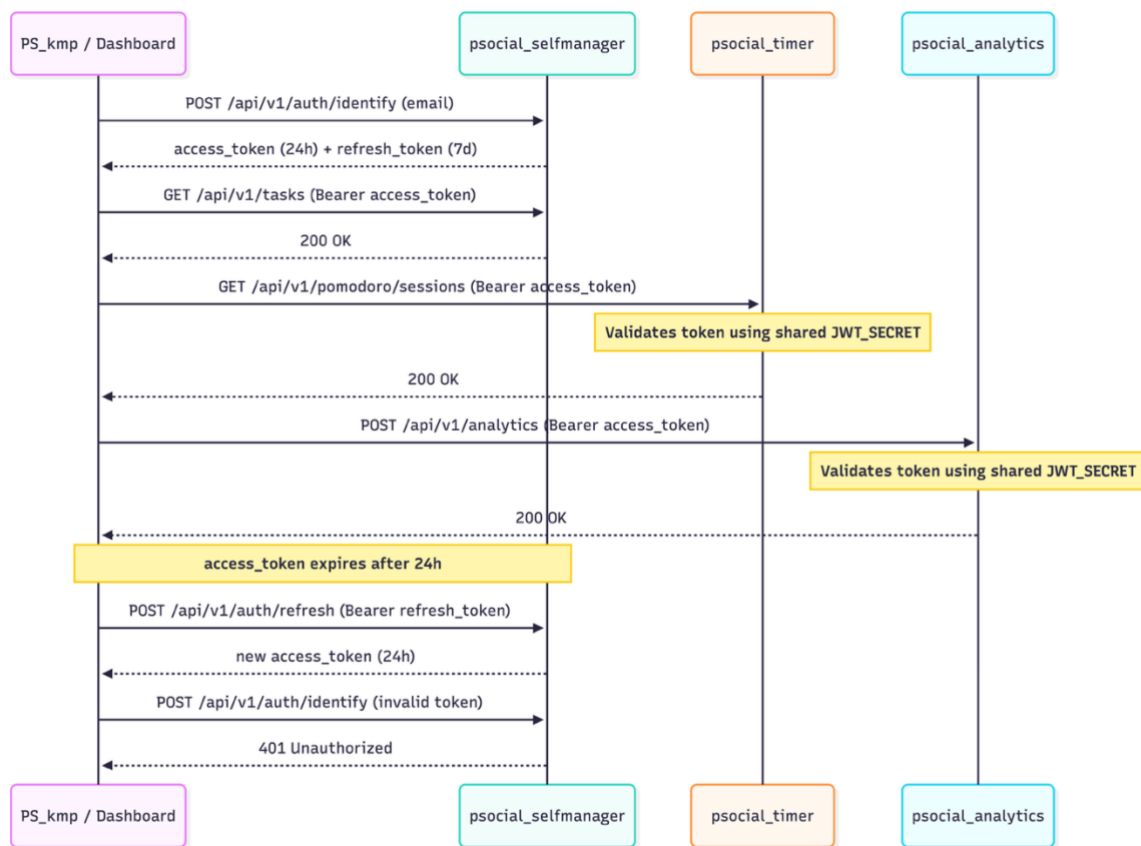


Рисунок 3.6 — Процесс входа пользователя в систему

Совместное использование JWT: один токен для всех сервисов

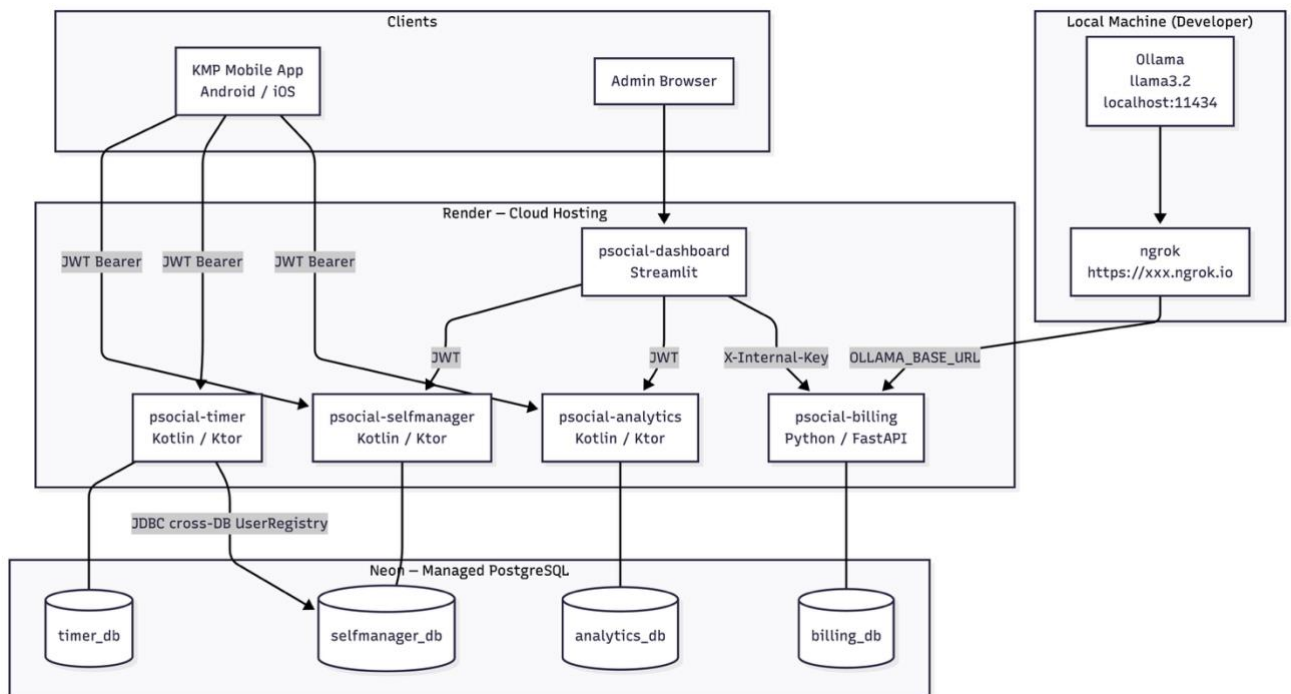


Рисунок 3.7 — Схема взаимодействия сервисов и клиентов

Все три пользовательских сервиса (selfmanager, timer, analytics) настроены с использованием одних и тех же переменных окружения `JWT_SECRET`, `JWT_ISSUER`, `JWT_AUDIENCE` и `JWT_REALM`. Поскольку все три службы разделяют эти значения, токен, подписанный selfmanager, проходит проверку в службах timer и analytics. Службы не проверяют, кто выдал токен — они проверяют только действительность подписи (что так и есть, поскольку все они используют один и тот же секрет) и правильность формирования утверждений. Это фундаментальный механизм межведомственной аутентификации без централизованного поставщика удостоверений.

В пользовательской регистрации: каноническое целочисленное тождество

Более тонкая проблема идентификации заключается в обеспечении того, чтобы одному и тому же адресу электронной почты присваивалось одно и то же целое число `user_id` независимо от того, какая служба впервые сталкивается с пользователем. Это важно, потому что межведомственная привязка данных зависит от общего целочисленного ключа: если сеансы Pomodoro пользователя

хранятся с идентификатором пользователя = 42 в базе данных timer, но их задачи хранятся с идентификатором пользователя = 99 в базе данных selfmanager, служба аналитики не может объединить эти два набора данных.

В UserRegistry это решается за счет того, что все службы, которым требуются идентификаторы пользователей, подключаются напрямую к таблице пользователей selfmanager в качестве канонического источника. Служба таймера имеет выделенный пул соединений HikariCP, указывающий на базу данных PostgreSQL от selfmanager (настраивается с помощью переменных окружения USER_REGISTRY_DB_*). Когда служба таймера получает запрос с JWT, она извлекает запрос по электронной почте и запускает:

```
INSERT INTO users (email, created_at)
VALUES (?, NOW())
ON CONFLICT (email) DO UPDATE SET email = EXCLUDED.email
RETURNING id
```

Рисунок 3.8 — Идемпотентный upsert пользователя в реестре selfmanager (UserRegistry)

Эта ошибка upsert, или создает пользователя (возвращая новый идентификатор), или находит существующего пользователя (возвращая существующий идентификатор). Результатом всегда является канонический целочисленный идентификатор для этого электронного письма — тот же идентификатор, который selfmanager вернул бы для того же электронного письма. Предложение RETURNING id - это функция PostgreSQL, которая возвращает значение столбца ID для строки, которая была вставлена или обновлена.

Служба аналитики не использует UserRegistry напрямую — вместо этого она извлекает userId из утверждений JWT (которые selfmanager уже проверил и закодировал) и использует это значение напрямую. Это работает, потому что требование userId JWT было установлено selfmanager при выпуске токена, а selfmanager является авторитетным источником идентификаторов пользователей.

Единственным соединением между базами данных во всей системе является UserRegistry подключенное к базе данных selfmanager. Это явная и

документированная зависимость, ее назначение узкое (один запрос: найти или создать пользователя по электронной почте), и у нее есть выделенный пул подключений, чтобы он не мешал нормальной работе таймера с базой данных.

Аутентификация по внутреннему ключу X: от сервиса к сервису

Для вызовов между службами (вызов аналитики `selfmanager` для получения пользовательских данных, вызов таймера `selfmanager` для обновления журнала времени, вызов аналитики `billing` для вывода данных) используется другой механизм аутентификации: предварительно разделяемый ключ в HTTP-заголовке `X-Internal-Key`.

Основная причина использования отдельного механизма для вызовов между службами вместо повторного использования JWTs заключается в том, что внутренние конечные точки предоставляют данные в совокупности (например, “все задачи для пользователя 42”) или выполняют операции, которые обходят бизнес-правила (например, начальные тестовые данные при выставлении счетов). Эти конечные точки должны быть доступны только доверенным службам, а не любому пользователю с действительным JWT. Использование отдельного заголовка с отдельным ключом делает эту границу явной.

Переменная среды `INTERNAL_API_KEY` имеет одинаковое значение для всех сервисов. Когда аналитика вызывает `GET /internal/users/{userId}/задачи` в `selfmanager`, она включает `X-Internal-Key: <ключ>` в запрос. `selfmanager` проверяет этот заголовок перед выполнением обработчика. Если заголовок отсутствует или ключ не совпадает, ответ `401 Несанкционированный` — независимо от того, содержит ли запрос также действительный JWT.

Внутренний ключ никогда не передается в исходный код, никогда не возвращается ни в одном ответе API и не документируется в общедоступном пользовательском интерфейсе Swagger. Он хранится исключительно в переменных окружения на сервере.

3.4 Модель данных и стратегия хранения

Каждая служба владеет своей собственной базой данных PostgreSQL со схемой, разработанной для конкретных шаблонов доступа к данным этой службы.

Ниже приведены ключевые таблицы каждой службы. Все клиентские сущности несут идентификатор `syncId` (UUID) для идемпотентной офлайн-синхронизации, столбец `serverId`, заполняемый после синхронизации, и мягкое удаление через временные метки `updatedAt`.

Диаграмм модели данных представляется в приложениях.

3.4.1 `psocial_selfmanager`

Центральная служба владеет всей моделью пользовательских данных — проектами, задачами, привычками, рутинами — и общей таблицей `users`, к которой обращаются другие службы.

Таблица 3.1 — Схема БД сервиса `selfmanager`

Таблица	Ключевые поля	Примечания
<code>users</code>	<code>id</code> , <code>device_id</code> , <code>email</code>	Учётная запись по <code>email</code> , несколько устройств
<code>refresh_tokens</code>	<code>userId</code> , <code>token</code> , <code>deviceId</code>	Отзыв сессии через ротацию токенов
<code>projects</code>	<code>userId</code> , <code>name</code> , <code>description</code> , <code>iconName</code> , <code>colorHex</code> , <code>priority</code>	Родительский контейнер
<code>tags</code>	<code>userId</code> , <code>name</code>	Общие для всех сущностей

project_tags_bridge	projectId, tagId	Связь многие-ко-многим
tasks	userId, projectId, name, priority, date, completed, timeSpentMinutes, syncId	syncId = UUID для офлайн-синхронизации
subtasks	taskId, name, completed, timeSpentMinutes	Иерархические подзадачи
task_completion_log	taskId, completedAt	Журнал выполнения
task_times	taskId, time	Несколько запланированных времён
habits	userId, projectId, name, habitType (Start/Quit), recurrency, syncId	Тип привычки и периодичность
habit_subtasks / habit_times / habit_reminder_times	habitId, ...	Подзадачи, времена выполнения и напоминаний
habit_completion_log	habitId, completedAt	Журнал выполнения по датам
routines	userId, projectId, name, recurrency, syncId	Последовательность шагов
routine_steps	routineId, name, durationMinutes, order, completed	Упорядоченные шаги

routine_times / routine_completion_log	routineId, ...	Времена и журнал выполнения
sync_tombstones	userId, entityType, entityId, deletedAt	Записи удалений для офлайн-клиентов

3.4.2 psocial_timer

Служба таймера хранит настройки Pomodoro, сеансы и интервалы. Длительности фиксируются (снимок) при создании сеанса, поэтому последующие изменения настроек не искажают исторические данные.

Таблица 3.2 — Схема БД сервиса timer

Таблица	Ключевые поля	Примечания
pomodoro_settings	userId, workDurationMinutes (25), shortBreakMinutes (5), longBreakMinutes (15), cyclesUntilLongBreak (4)	Одна строка на пользователя; классический Pomodoro по умолчанию
pomodoro_settings (флаги)	autoStartBreaks, autoStartSession, focusModeEnabled, soundEnabled, notificationsEnabled	Логические настройки
pomodoro_sessions	userId, entityType (Task/Habit/Routine/None), entityId, status (Active/Paused/Completed/Abandoned)	Связь с сущностями selfmanager; состояние сеанса

pomodoro_sessions (снимок)	workDurationMinutes, shortBreakMinutes, longBreakMinutes	Снимок настроек на момент создания
pomodoro_sessions (прогресс)	completedCycles, totalWorkMinutes, pauseCount, startedAt, completedAt, syncId	Накопленный прогресс и метки времени
pomodoro_intervals	sessionId, type (Work/ShortBreak/LongBreak), startedAt, endedAt, plannedDurationMinutes, completed, syncId	Один блок на интервал

3.4.3 psocial_analytics

Служба аналитики не владеет пользователями (идентификаторы поступают из selfmanager) и хранит минимум: якорь пользователя и сами сгенерированные ИИ отчёты.

Таблица 3.3 — Схема БД сервиса analytics

Таблица	Ключевые поля	Примечания
users	id, createdAt	Минимальная таблица; привязывает отчёты к идентификатору пользователя
analytics_reports	userId, type, modelId, insight, creditsCharged, createdAt	Хранит сгенерированный ИИ текст; type — один из шести типов анализа (PRODUCTIVITY_SUMMARY,

		TASK_PRIORITIZATION, HABIT_INSIGHTS, ROUTINE_OPTIMIZATION, WEEKLY_TIMER_SUMMARY, CUSTOM)
--	--	--

3.4.4 psocial_billing

Служба биллинга ведёт собственную таблицу пользователей, реестр ML/LLM-моделей, журнал прогнозов и транзакций. Баланс не хранится отдельным столбцом, а вычисляется из снимка `balance_after` последней транзакции.

Таблица 3.4 — Схема БД сервиса billing

Таблица	Ключевые поля	Примечания
users	id (UUID), selfmanager_user_id, email, is_active, is_admin	Привязка к пользователю selfmanager через selfmanager_user_id
ml_models	id, name, version, type (SKLEARN/LLM), llm_model_id, llm_provider, system_prompt, cost_per_prediction, is_active	Модель Ollama по умолчанию (UUID fb96de86-3c73-489d-8f02-aaf8343d6101, llama3.2)

		засеивается при старте
predictions	selfmanager_user_id, model_id, input_data (JSONB), output_data (JSONB), status (PENDING/SUCCESS/FAILED/INSUFFICIENT_CREDITS), credits_charged	Жизненный цикл прогноза с контекстом сущности
transactions	selfmanager_user_id, amount, type (DEPOSIT/CHARGE/REFUND), prediction_id, balance_after	Снимок баланса после каждой операции

3.5 Реализация сервиса

Рабочий процесс разработки. Все сервисы Kotlin имеют одинаковую структуру проекта: многомодульный проект Gradle с серверным подмодулем, содержащим код приложения. Серверный модуль создается в fat JAR с помощью задачи shadowJar, которая объединяет все зависимости. Dockerfile для каждого сервиса Kotlin копирует предварительно созданный файл fat JAR, а не создает его в контейнере, что сокращает время сборки Docker до 10 секунд. Служба выставления счетов Python использует одномодульное приложение FastAPI с alembic для миграции базы данных (заменено в текущей настройке разработки на SQLAlchemy create_all(), вызываемое при запуске, с необязательным флагом RESET_DB= true для сброса схемы).

Обработка ошибок. Каждая служба определяет набор исключений, специфичных для конкретного приложения (например, исключение PaymentRequiredException, исключение NotFoundException, исключение UnauthorizedException), которые перехватываются глобальным плагином страницы состояния Ktor и преобразуются в соответствующие HTTP-ответы со структурированными текстами ошибок. Это гарантирует, что потребители API

получают согласованные форматы ошибок, а не необработанные трассировки стека.

Ведение журнала. плагин CallLogging от Ktor регистрирует все входящие запросы с указанием их кодов состояния и времени ответа. Этого достаточно для текущего этапа проверки концепции. В производственном развертывании будут добавлены структурированное ведение журнала (например, строки журнала в формате JSON) и агрегирование журналов (например, Datadog, Logtail).

3.6 Мобильное приложение (Kotlin Multiplatform)

Мультиплатформенное мобильное приложение Kotlin предназначено для использования конечными пользователями ProductiveSocial. Он нацелен на Android, iOS и JVM Desktop из единой общей базы кода, используя Compose Multiplatform для уровня пользовательского интерфейса и SQLDelight для локального хранения данных в автономном режиме. Приложение полностью функционально без подключения к сети — все пользовательские взаимодействия сначала записываются в локальную базу данных SQLite, а уровень фоновой синхронизации отправляет изменения в серверные службы и обратно, когда доступно подключение.

3.6.1 Архитектура приложения: трехуровневая чистая архитектура

Приложение состоит из трех отдельных уровней, каждый из которых имеет четко определенные обязанности и не имеет циклических зависимостей.

Уровень домена (commonMain/domain/): Содержит чистые классы данных Kotlin и интерфейсы вариантов использования с нулевой зависимостью от какого-либо фреймворка, базы данных или сетевой библиотеки. Модель предметной области - это простой класс данных Kotlin. Интерфейс GetTasksUseCase определяет, как извлекаются задачи, не зная, поступают ли они из локальной базы данных или удаленного API. Этот уровень полностью поддается модульному тестированию без какой-либо макетной инфраструктуры.

Уровень данных (commonMain/data/): Содержит реализации репозитория, поддерживаемые двумя источниками данных для каждого типа объекта — LocalDataSource (SQLDelight) и RemoteDataSource (HTTP-клиент Ktor). Ответственность репозитория заключается в координации между этими источниками: сначала записи отправляются в local (что делает пользовательский интерфейс отзывчивым немедленно), и диспетчер синхронизации определяет ожидающие изменения по двум маркерам — serverId IS NULL для созданных локально объектов и updatedAt > lastSyncedAt для изменённых — после чего сбрасывает их на сервер в фоновом режиме. Когда сервер отвечает, локальные строки обновляются идентификаторами, назначенными сервером.

Уровень представления (commonMain/presentation/): Содержит составные элементы пользовательского интерфейса и связанные с ними классы ViewModel. Каждый экран представляет собой составной объект без состояния, который наблюдает за потоком состояний<UIState> из своей ViewModel и отправляет пользовательские события по каналу. ViewModels не отображают изменяемое состояние напрямую — все изменения проходят через метод handleEvent(), что делает конечный автомат явным и отслеживаемым.

График зависимостей применяется на границах модуля: домен не имеет зависимостей; данные зависят от домена; представление зависит от данных и домена. Наборы исходных текстов для конкретной платформы (androidMain, iosMain, jvmMain) содержат только регистрации драйверов для конкретной платформы и точки входа для внедрения зависимостей - вся бизнес-логика находится в commonMain.

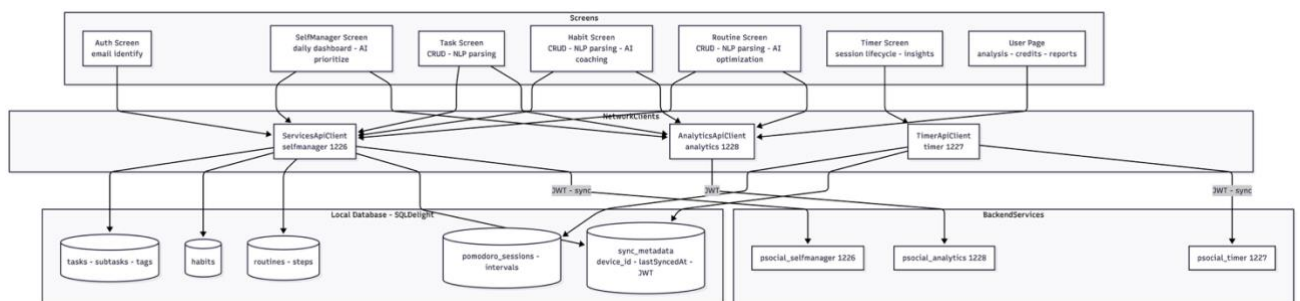


Рисунок 3.9 — Трёхуровневая чистая архитектура мобильного приложения (KMP)

3.6.2 Локальная база данных: SQLDelight схема

У каждого объекта есть локальная таблица SQLDelight. Схема предназначена для зеркального отображения серверной модели данных при добавлении полей, необходимых для автономных операций.

Таблица 3.5 — Схема локальной базы данных SQLDelight

Таблица	Ключевые поля	Примечания
tag	localId, serverId, name	Уникальность по name и serverId; общие для всех сущностей
project	localId, serverId, syncId, name, description, iconName, colorHex, priority, createdAt, updatedAt, isDeleted	syncId — UUID для синхронизации; isDeleted — локальное мягкое удаление
project_tag	projectLocalId, tagLocalId	Связь многие-ко-многим; каскадное удаление
task	localId, serverId, syncId, projectLocalId, name, description, priority, urgency, recurring, sendReminder, date, target, completed, timeSpentMinutes,	—

	createdAt, updatedAt, isDeleted	
task_time	id, taskLocalId, time	Формат HH:mm; несколько времён на задачу
task_reminder	id, taskLocalId, option	Текстовое значение («5 minutes before» и т.д.)
task_tag	taskLocalId, tagLocalId	Связь многие-ко-многим
subtask	localId, serverId, syncId, taskLocalId, name, completed, timeSpentMinutes, isDeleted	—
habit	localId, serverId, syncId, projectLocalId, name, description, habitType, recurrency, target, sendReminder, completed, timeSpentMinutes, createdAt, updatedAt, isDeleted	habitType: Start / Quit
habit_time	id, habitLocalId, time	Формат HH:mm
habit_reminder_time	id, habitLocalId, time	Формат HH:mm
habit_tag	habitLocalId, tagLocalId	Связь многие-ко-многим

habit_subtask	localId, serverId, syncId, habitLocalId, name, completed, timeSpentMinutes, isDeleted	—
habit_completion	id, habitLocalId, completedDate, serverCompletionId, syncId, deleted	Формат даты YYYY- ММ-DD; уникальность по (habitLocalId, completedDate)
routine	localId, serverId, syncId, projectLocalId, name, description, recurrency, target, sendReminder, completed, createdAt, updatedAt, isDeleted	—
routine_time	id, routineLocalId, time	Время начала
routine_off_time	id, routineLocalId, time	Время окончания
routine_reminder_time	id, routineLocalId, time	Время напоминания
routine_tag	routineLocalId, tagLocalId	Связь многие-ко-многим
routine_step	localId, serverId, syncId, routineLocalId, name, description, autoStart, durationMinutes, position, completed, isDeleted	Сортировка по position

routine_completion	localId, routineLocalId, completedDate, syncId	Уникальность по (routineLocalId, completedDate)
pomodoro_settings	localId, serverId, workDurationMinutes (25), shortBreakMinutes (5), longBreakMinutes (15), cyclesUntilLongBreak (4), autoStartBreaks, autoStartSession, focusModeEnabled, soundEnabled, notificationsEnabled	Всегда одна строка (localId = 1)
pomodoro_session	localId, serverId, syncId, userId, entityType, entityLocalId, entityServerId, status, workDurationMinutes, shortBreakMinutes, longBreakMinutes, cyclesUntilLongBreak, completedCycles, totalWorkMinutes, startedAt, completedAt	entityType: Task / Habit / Routine / NULL; снимок настроек на момент создания
pomodoro_interval	localId, serverId, syncId, sessionLocalId, type, plannedDurationMinutes,	type: Work / ShortBreak / LongBreak

	startedAt, endedAt, completed	
sync_metadata	key, value	Хранилище ключ- значение: deviceId, serverId, defaultProjectId, lastSyncedAt, lastTimerSyncedAt

3.6.3 Аутентификация

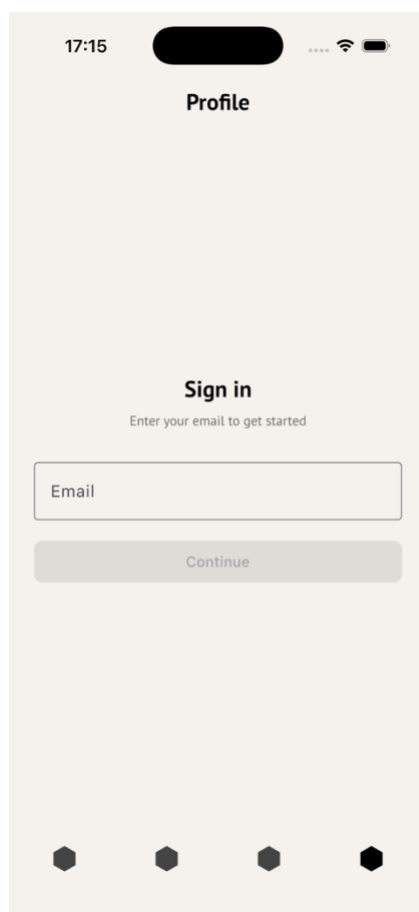


Рисунок 3.10 — Экран аутентификации по почте

Пользователи входят, используя только адрес электронной почты — без пароля. На экране аутентификации отображается один ввод электронной почты и кнопка «Продолжить». При отправке приложение вызывает POST

/api/v1/auth/identify в selfmanager. Сервер находит или создаёт пользователя и возвращает токен доступа, токен обновления и целое число userId.

Все четыре значения — accessToken, refreshToken, serverId и email — записываются в таблицу sync_metadata. При последующих запусках приложения UserSessionManager.init() считывает сохранённые данные и автоматически восстанавливает сеанс — пользователь попадает непосредственно на главный экран без повторной аутентификации.

Сеанс остаётся действительным в течение 24 часов с момента выдачи токена доступа. По истечении этого срока ServicesApiClient автоматически выполняет обновление, вызывая UserSessionManager.updateTokens(), который сохраняет новую пару токенов в sync_metadata и обновляет AppSession — без участия пользователя. Токен обновления действителен в течение 7 дней, что означает, что активный пользователь никогда не сталкивается с принудительным повторным входом в систему. Сеанс завершается только при явном выходе пользователя из системы.

Выход из системы вызывает UserSessionManager.logout(), который стирает все локально сохранённые пользовательские данные — токен, идентификатор пользователя, адрес электронной почты, все таблицы сущностей и всё состояние синхронизации — эффективно возвращая приложение в чистое состояние при первом запуске.

3.6.4 Проекты

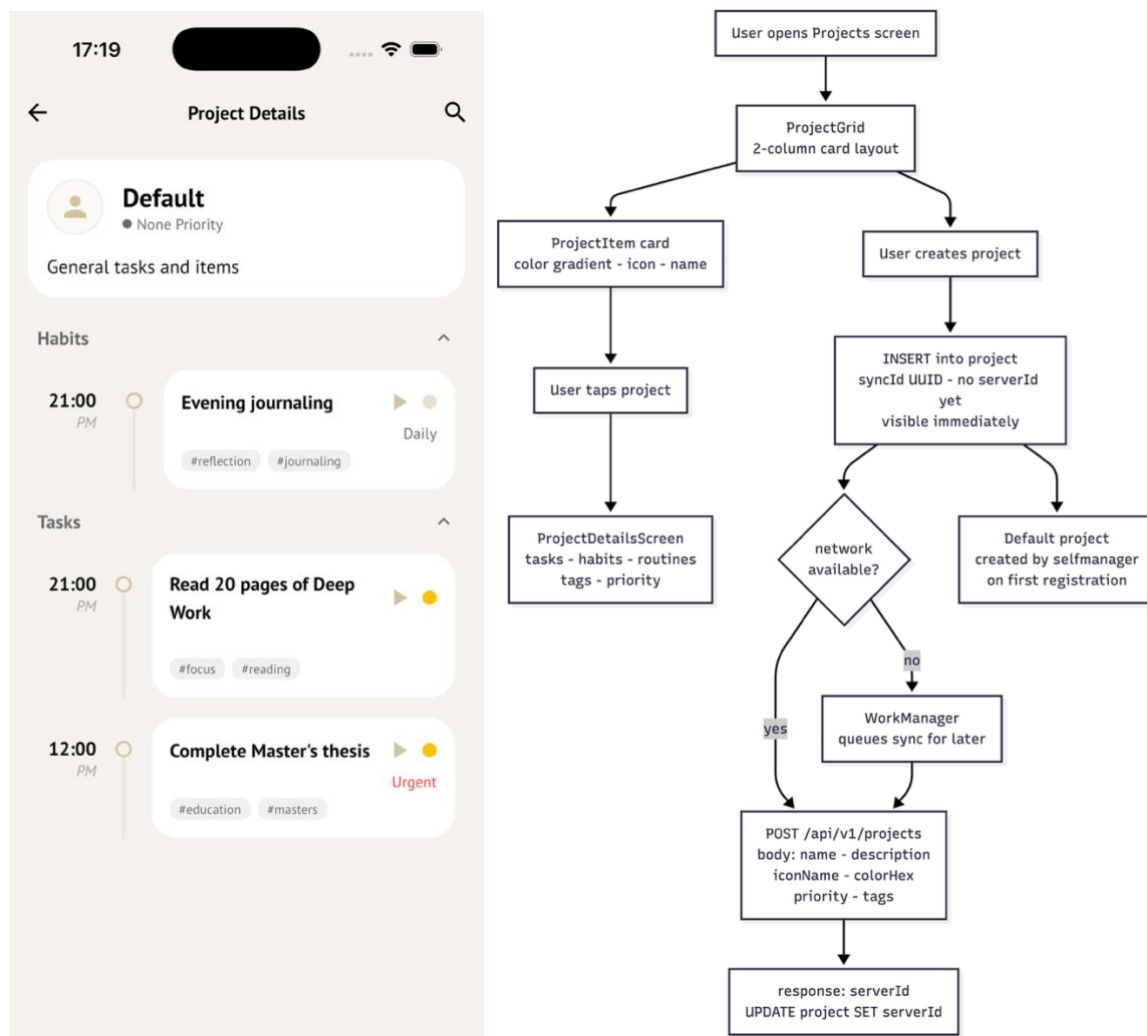


Рисунок 3.11 — Сетка проектов на экране самостоятельного управления.
Поток синхронизаций проектов

Проекты - это организационные контейнеры. Каждая сущность в self-manager принадлежит проекту. У каждого проекта есть имя, описание, шестнадцатеричный цветовой код, название значка, уровень приоритета и теги. Пользователь может просматривать все проекты в виде таблицы, каждый из которых отображается в виде цветной карточки с названием проекта.

При детализации проекта открывается подробное представление проекта, в котором отображаются все вложенные в него задачи, привычки и подпрограммы, а также теги и приоритет проекта. Проекты могут создаваться локально (сразу видны в таблице) и синхронизироваться с сервером в фоновом режиме.

При первой синхронизации у каждого пользователя есть по крайней мере проект “По умолчанию”, созданный автоматически selfmanager. Локальная база данных хранит как клиентский идентификатор синхронизации UUID, так и назначенное сервером целое число ServerID; после первого ответа синхронизации ServerID заполняется и используется для всех последующих межведомственных ссылок.

3.6.5 Экран самостоятельного управления

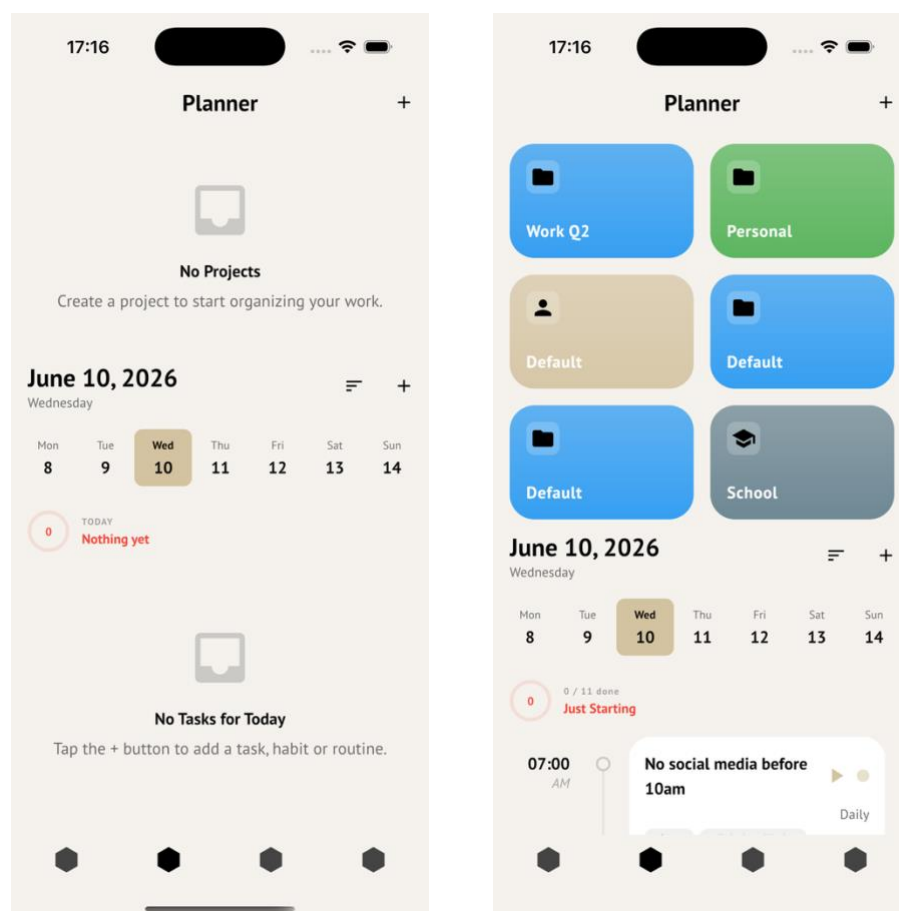


Рисунок 3.12 — Главный экран самоменеджмента

Экран самостоятельного управления — это центральное окно ежедневного планирования и главный экран приложения. Он объединяет задачи, привычки и распорядок дня в единый унифицированный список на выбранную дату.

В верхней части экрана находится горизонтальная полоса даты — прокручиваемый ряд чипов с датами, показывающий текущую неделю. Нажатие на чип даты обновляет выбранную дату и перезагружает список.

Под полосой даты отображается сетка проектов, позволяющая быстро перейти к конкретному проекту прямо с главного экрана. Ниже находится индикатор ежедневного прогресса, показывающий процент выполненных на сегодня элементов.

Далее следует единый список производительности, который показывает все задачи, запланированные на выбранную дату, все привычки, которые должны быть выполнены в этот день (в зависимости от их повторяемости), и все процедуры, запланированные на этот день, — все в одном прокручиваемом списке. Каждый элемент можно отличить по значку: флажок для задач, значок flame для запуска привычек, значок X для прекращения привычек и значок контрольного списка для процедур.

Список поддерживает два ортогональных элемента управления в заголовке:

- Сортировка: элементы могут быть отсортированы по срокам выполнения (по возрастанию или по убыванию) или по приоритету (от высокого к низкому).
- Группировка: элементы могут быть сгруппированы по отсутствию (плоский список), по тегу, по проекту или по срочности. При группировании каждая группа представляет собой складной заголовок раздела с названием группы и количеством элементов.'

SelfManagerViewModel объединяет реактивные потоки из репозитория задач, привычек и рутин с помощью оператора combine корутин Kotlin, применяет текущую конфигурацию сортировки и группировки и повторно отправляет обработанный список всякий раз, когда изменяется выбранная дата или какие-либо данные репозитория. Это означает, что если пользователь отмечает привычку выполненной в этом списке, список обновляется мгновенно без перезагрузки.

В этом списке каждый элемент доступен для встроенного действия:

- Задачи: нажатие флажка отмечает выполнение задачи; нажатие на строку открывает экран детали задачи; кнопка ► запускает сеанс фокусировки в таймере.
- Привычки: нажатие на кружок завершения отмечает привычку выполненной на сегодня; нажатие на строку открывает экран детали привычки; кнопка ► доступна для запуска привычки.
- Процедуры: нажатие на процедуру открывает программу выполнения процедуры.

3.6.6 Задачи

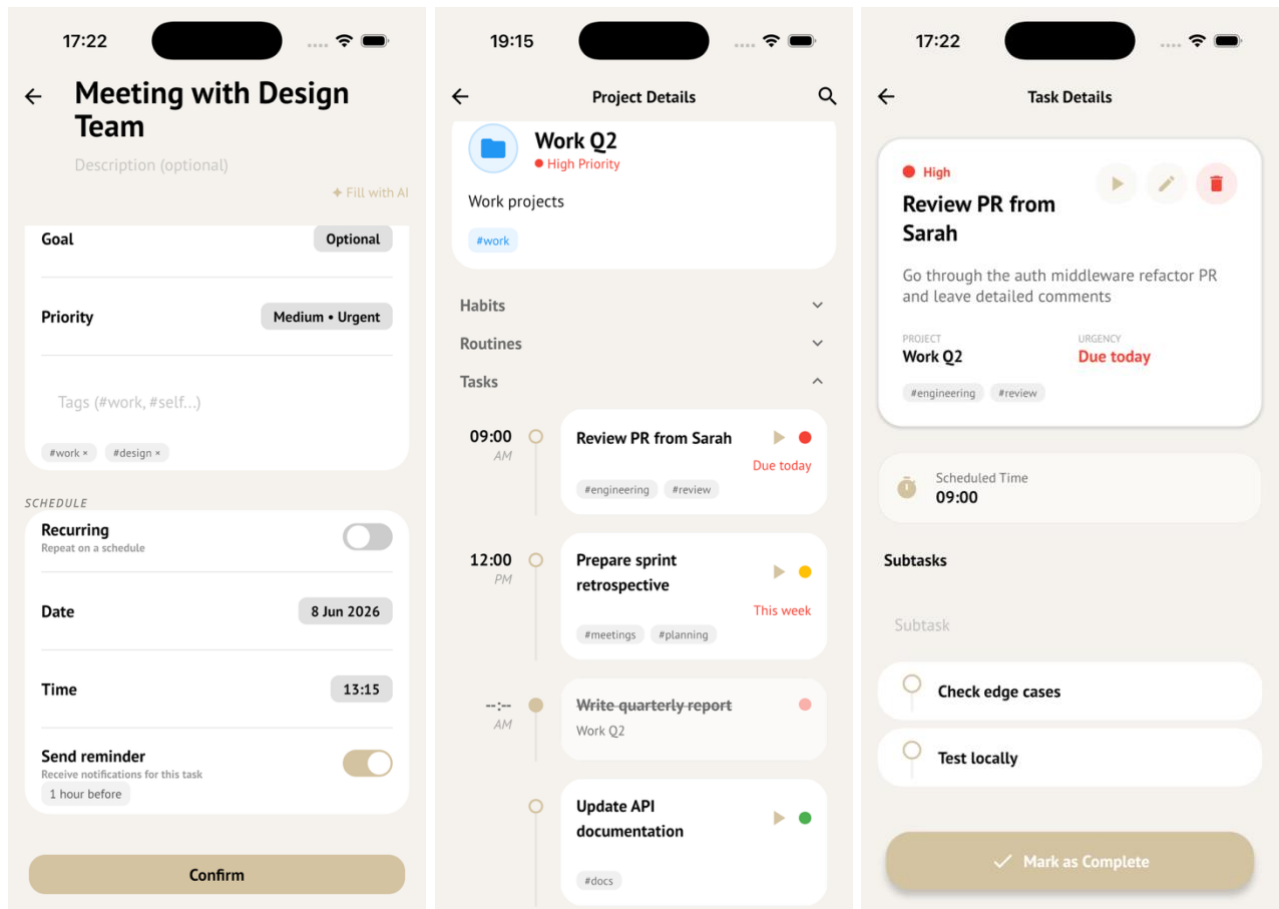


Рисунок 3.13 — Создание задачи, Список задач, Детальный просмотр задачи

Задачи являются основной единицей целенаправленной работы. Полная модель данных о задачах включает:

- Имя и необязательное описание
- Приоритет: Отсутствует, Низкий, Средний или Высокий — отображается в виде значка с цветовой кодировкой

- Срочность: измерение, отдельное от приоритета (допускающее матрицу срочности / приоритета 2×2)
- Дата выполнения: необязательно, с выделением просрочки по истечении
- Повторяющийся флаг: помечает задачи, которые повторяются по расписанию
- Цель: необязательная количественная цель (например, «прочитать 30 страниц»)
- Запланированное время: одно или несколько конкретных периодов дня, когда планируется выполнение задачи, сохранённых в виде строк «ЧЧ:мм»
- Параметры напоминания: срабатывание уведомления в запланированное время или за 5, 10, 15 или 30 минут до этого
- Теги: несколько меток, общих для всех типов сущностей
- Подзадачи: список подпунктов, у каждого из которых есть имя и состояние завершения с собственным отслеживанием выполнения
- Время, затраченное: накопленные минуты (timeSpentMinutes) связанных сеансов Pomodoro, автоматически обновляемые службой таймера через внутреннюю конечную точку регистрации времени

Экран создания и редактирования задач управляет всем вышеперечисленным в единой прокручиваемой форме. Подзадачи могут быть добавлены встроенными с помощью текстового поля в нижней части списка подзадач. Теги вводятся как чипы с автозаполнением в соответствии с существующей библиотекой тегов пользователя.

Экран редактирования также включает функцию создания задач на естественном языке «♦ Заполнить с помощью ИИ». Пользователь открывает нижнюю панель, вводит произвольный текст (краткий или развёрнутый) и нажимает «Сгенерировать»; запрос отправляется на конечную точку POST `/api/v1/analytics/parse-task` службы analytics, которая через LLM выводит структурированное представление задачи. Карточка предпросмотра показывает выведённые поля — название, описание, приоритет, срочность, дату, время, проект, теги, подзадачи и напоминание — перед их применением: «Применить к форме» заполняет все поля, «Попробовать снова» сбрасывает предпросмотр. Индикатор состояния модели (зелёный / жёлтый / красный) отображается через GET `/api/v1/analytics/model-status` при открытии панели. Функция расходует кредиты через службу billing, но не сохраняет отдельный аналитический отчёт.

Представление сведений о задаче — это аналог редактора, ориентированный на чтение. В нём представлены полные данные задачи, контрольный список подзадач, накопленный счётчик затраченного времени и связанные сеансы Pomodoro с их датами и продолжительностью.

Экран целей (GoalScreen) на текущем этапе реализован только как каркас: заголовок с полями для названия и описания и одна пустая карточка-секция «Goal Details» без специфичных полей, ViewModel, таблицы SQLDelight и интеграции с API. Навигационный маршрут Goal уже зарегистрирован, и переход на этот экран доступен из селектора типов. Цели задуманы не как отдельный модуль, а как сквозная функция: записи о целях встраиваются в selfmanager и связываются с задачами, привычками и записями дневника. Полная реализация (модель данных, ViewModel, конечные точки и пакет синхронизации) запланирована наряду с модулем дневника до выпуска MVP.

3.6.7 Привычки

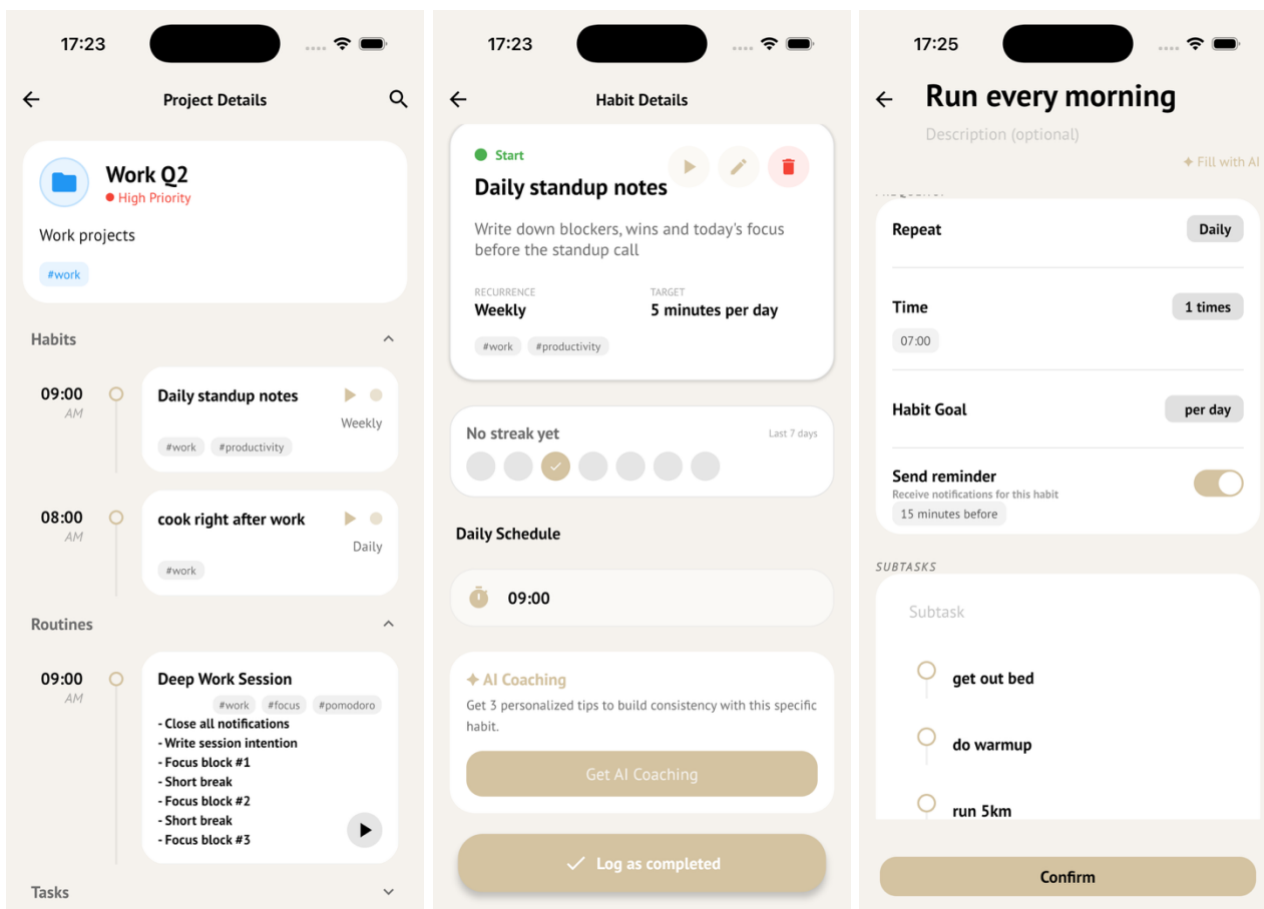


Рисунок 3.14 — Экран привычек (Habits)

Привычки бывают двух типов, отражающих противоположные поведенческие цели:

Начальные привычки (Start) представляют собой модели поведения, которые пользователь хочет выработать — ежедневные упражнения, медитация, чтение в течение 30 минут.

Привычки к отказу (Quit) представляют собой поведение, от которого пользователь хочет избавиться — первым делом проверять социальные сети по утрам, есть сахар после 8 вечера. Пользовательский интерфейс завершения инвертирует положительное обрамление: завершение означает, что пользователь успешно избежал такого поведения.

Оба типа используют одну и ту же модель данных: название, описание, проект, приоритет, теги, повторяемость (ежедневно, еженедельно и т.д.), запланированное время, время напоминания и подзадачи.

На экране детали привычки отображается счётчик серии («X day streak») и горизонтальный ряд из 7 кружков, показывающий историю выполнения за последние 7 дней: закрашенный кружок с галочкой соответствует дню, когда привычка была отмечена выполненной.

Выполнение привычки отслеживается с ежедневной детализацией в таблице `habit_completion`: в каждой строке записывается идентификатор привычки, дата завершения в формате «ГГГГ-ММ-ДД» и служебные поля синхронизации. Датой завершения является локальная дата (не временная метка UTC), чтобы избежать прерывания серии из-за часового пояса при отметке привычки ближе к полуночи.

Пометка привычки завершённой немедленно создаёт локальную строку `habit_completion`, добавляет её в очередь синхронизации и обновляет счётчик серии в пользовательском интерфейсе.

Если у привычки есть подзадачи, они отображаются на экране детали привычки в разделе «Checklist» в виде чекбоксов. Состояние завершения каждой подзадачи является глобальным булевым значением и не привязано к конкретной дате.

3.6.8 Процедуры

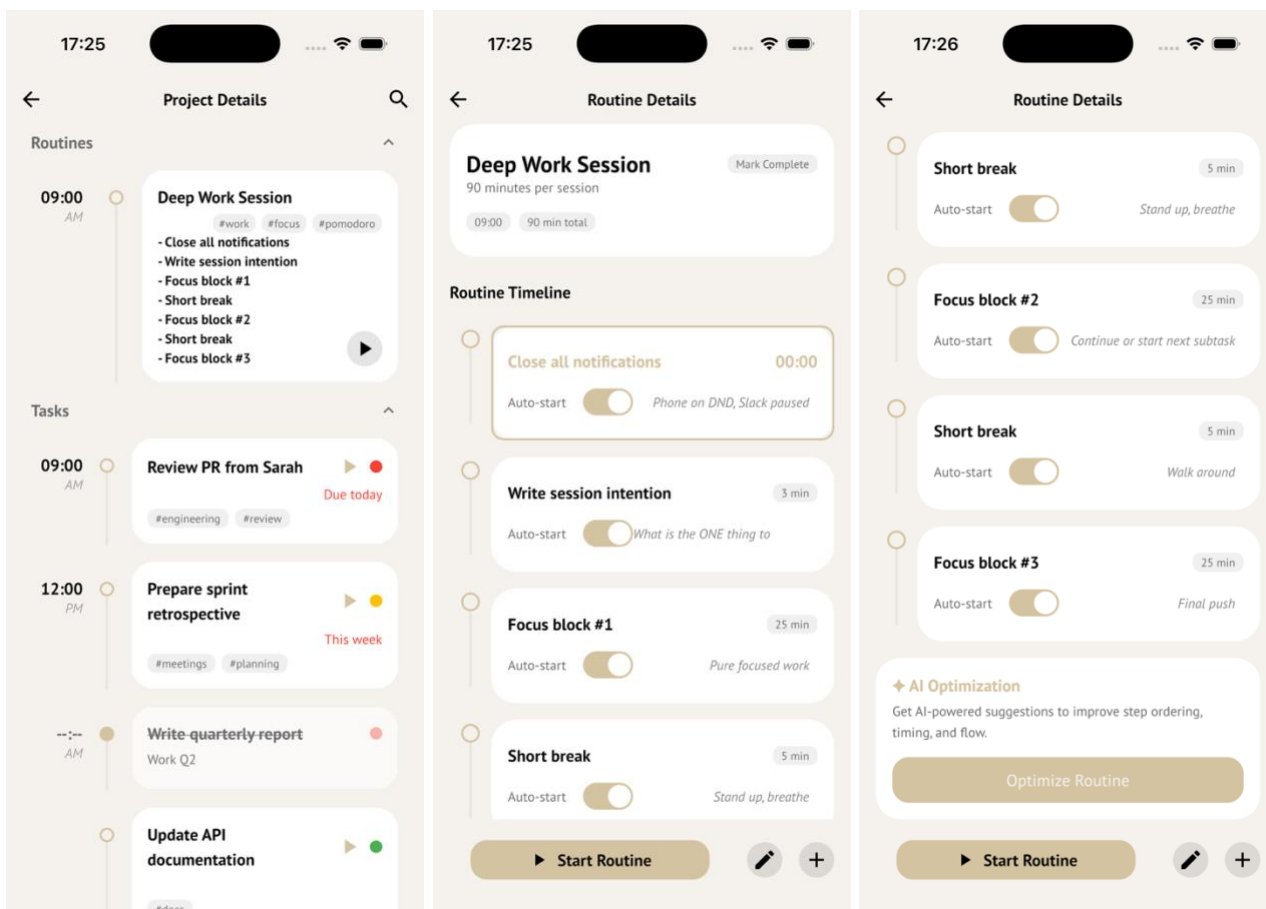


Рисунок 3.15 — Экран рутин (Routines)

Процедуры представляют собой структурированную последовательность шагов — упорядоченный контрольный список, в котором шаги должны выполняться по порядку, а не в произвольной последовательности. Они предназначены для фиксированных рабочих процессов, которые пользователь выполняет неоднократно: утренняя рутина, последовательность действий перед тренировкой, протокол учебного занятия, подведение итогов в конце дня.

Каждая процедура имеет те же данные заголовка, что и задачи и привычки (название, описание, проект, приоритет, теги, повторяемость, запланированное время, время напоминания). Отличительной особенностью является её список шагов: упорядоченный набор шагов, каждый из которых содержит:

- Название: что делать на этом этапе
- Описание: необязательный контекст или инструкция

- Продолжительность в минутах: сколько времени обычно занимает выполнение шага
- Позиция: определяет фиксированный порядок в списке
- Флаг автозапуска: должен ли таймер переходить к этому шагу автоматически по завершении предыдущего

Процедуры также поддерживают нерабочее время — конкретные временные отметки (в формате «ЧЧ:мм»), когда процедура не должна отображаться в списке планировщика.

Экран деталей процедуры (RoutineDetailsScreen) служит одновременно просмотрным и исполнительным интерфейсом. Шаги отображаются в виде «Routine Timeline» — вертикального списка карточек. Нажатие кнопки «Start Routine» внизу запускает режим выполнения: активный шаг выделяется акцентным цветом, а вместо длительности появляется обратный отсчёт в формате «мм:сс». Завершение шага выполняется нажатием на кружок временной шкалы рядом с карточкой. Если у шага включён флаг автозапуска, таймер следующего шага запускается автоматически; в противном случае пользователь запускает его вручную. Кнопка внизу переключается между «Start Routine», «Pause Routine» и «Routine Completed» в зависимости от текущего состояния.

Когда все шаги выполнены, в локальную базу данных записывается строка routine_completion с идентификатором процедуры и датой завершения (формат «ГГГГ-ММ-ДД»). Детализация на уровне отдельных шагов в этой таблице не сохраняется.

3.6.9 Таймер Pomodoro

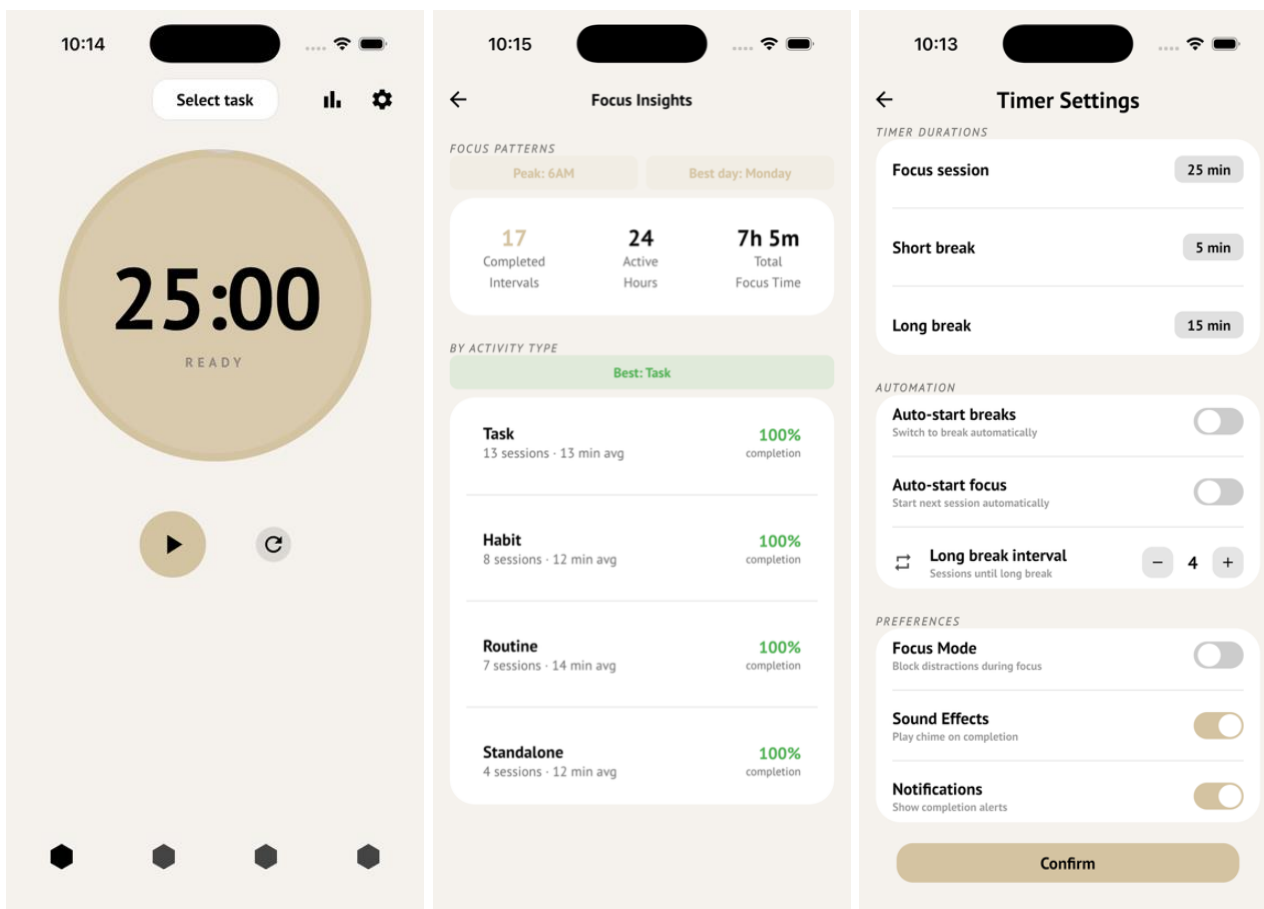


Рисунок 3.16 — Активная сессия таймера Pomodoro, статические сессия Pomodoro, настройка таймера Pomodoro

Модуль Pomodoro является наиболее интерактивным компонентом приложения, реализующим настраиваемый таймер фокусировки с полным жизненным циклом сеанса, отслеживаемым локально и синхронизируемым со службой таймера.

Настройки управляют поведением таймера и хранятся в локальной таблице `pomodoro_settings`. Настраиваемыми параметрами являются:

- Продолжительность рабочего интервала (по умолчанию 25 минут)
- Короткий перерыв (по умолчанию 5 минут)
- Длительный перерыв (по умолчанию 15 минут)
- Количество циклов перед длительным перерывом (по умолчанию 4)

- Автоматический запуск перерывов: запускается ли таймер перерыва автоматически по окончании рабочего интервала
- Автоматический запуск следующего сеанса: запускается ли рабочий таймер автоматически после перерыва
- Режим фокуса: полноэкранный режим без отвлечений
- Звук: воспроизводятся ли звуковые сигналы на границах интервала
- Уведомления: срабатывают ли системные уведомления по окончании интервалов

Изменения настроек записываются локально и синхронизируются со службой таймера через PUT /api/v1/pomodoro/settings. Когда ответ настроек включает рекомендованный блок (доступный после того, как пользователь накопит достаточно истории сеансов), на экране настроек отображаются рассчитанные сервером персонализированные длительности наряду с текущими настройками в качестве сравнения.

Создание сеанса и привязка объекта. Сеанс Pomodoro может быть создан в двух режимах:

- Автономный режим: пользователь открывает таймер и нажимает «Начать сеанс фокусировки» без привязки к какому-либо объекту. Сеанс создаётся локально, начинается первый рабочий интервал и запускается обратный отсчёт. Этот режим предназначен для общего времени фокусировки, не привязанного к конкретной задаче.
- Связанный режим: пользователь нажимает на значок Pomodoro рядом с задачей или привычкой. FocusViewModel открывает таймер, предварительно связанный с этим объектом — записывается тип объекта (EntityType) и локальный идентификатор (entityLocalId) в строке сеанса. Когда сеанс завершается и синхронизируется с сервером, служба таймера запускает вызов журнала времени для selfmanager, который добавляет общее количество рабочих минут сеанса к time_spent_minutes связанного объекта.

PomoViewModel владеет текущим состоянием многоциклового таймера: оставшееся время в текущем интервале, признак перерыва (isBreak), счётчик завершённых рабочих интервалов сегодня, общее количество сфокусированных минут и счётчик перерывов. Когда интервал заканчивается, ViewModel вычисляет следующий тип интервала на основе количества циклов и настроек, затем автоматически запускает его, если установлен флаг автозапуска.

FocusViewModel — это отдельный ViewModel для сеансов, связанных с объектами (Task или Habit). Он загружает подзадачи связанного объекта, которые отображаются под таймером во время рабочего интервала. Пользователь может отмечать подзадачи прямо во время сеанса, не покидая экран таймера. FocusViewModel реализует одноинтервальный режим: по окончании рабочего интервала сеанс завершается. Если пользователь вручную отмечает основной объект выполненным, FocusViewModel отображает список доступных задач, чтобы немедленно перейти к следующей, не возвращаясь на главный экран.

Сигнал о риске оставления. Когда пользователь нажимает «Пауза», ViewModel вызывает repository.pauseSession(), который возвращает AbandonmentRiskDto. Если warning == true, на экране паузы отображается предупреждение с вариантами «Продолжить» или «Отказаться».

Предложения о перерыве. После каждого завершённого рабочего интервала repository.completeInterval() возвращает BreakSuggestionDto с рекомендованным типом и продолжительностью перерыва (включая расширенный отдых). Пользователь может принять предложение или проигнорировать его.

Экран статистики Pomodoro. Отдельный экран статистики (PomodoroStatisticsScreen) загружает данные с сервера и отображает:

Focus Patterns: пиковый час и пиковый день недели (в виде чипов), общее количество завершённых рабочих интервалов и суммарное рабочее время — на

основе агрегатов byHour и byDayOfWeek By Activity Type: для каждого типа объекта (Task, Habit, Routine, Standalone) — количество сеансов, процент завершения (completionRate) и среднее рабочее время на сеанс.

3.6.10 Журнал

Модуль журнала является одной из наиболее стратегически важных запланированных функций платформы. Журнал служит отражающему аспекту личной продуктивности. В отличие от задач (ориентированных на завершение) и привычек (ориентированных на последовательность), ведение дневника создает пространство для ретроспективного осмысления: объединения событий дня в связное повествование, осмысления того, как прошел день, и извлечения уроков на завтра.

В каждой записи журнала будет записано:

- Дата и время написания
- Индикатор настроения: простая категориальная оценка настроения на основе шкалы (например, 1-5 или на основе эмодзи), полученная на момент написания
- Контент в свободной форме: собственный текст пользователя
- Контекст автоматической активности: структурированное резюме выполненных задач за день, выполнение привычек и общее количество сеансов Pomodoro — автоматически прикрепляется к записи из других сервисов, поэтому пользователю не нужно вручную расшифровывать то, что он сделал

Ключевым отличием является последний момент. Запись в дневнике в Первый день или любом другом приложении для ведения журнала требует, чтобы пользователь запомнил и вручную записал, чего он достиг. В ProductiveSocial запись в журнале предварительно заполняется структурированным резюме основной деятельности за день - каждой выполненной задачи, каждой

отмеченной сделанной привычки, общего времени сосредоточения, которое автоматически определяется с помощью `selfmanager` и службы таймера. Записи пользователей заполняют субъективный слой: как они себя чувствовали, что было трудно, что их удивило.

Синтез с помощью искусственного интеллекта. Самой мощной функцией журнала, запланированной как часть сервиса `psocial_journal`, является режим синтеза искусственного интеллекта. Когда пользователь завершает запись в журнале, специальный тип анализа (`DAILY_REFLECTION`) в службе аналитики будет объединять:

- Письменный текст пользователя (субъективный отчет)
- Выполнение задач на день и приоритеты (от `selfmanager`)
- Запись о завершении привычки (из `selfmanager`)
- Данные сеанса Pomodoro, включая время фокусировки и атрибуцию объекта (по таймеру)

LLM обобщит это в краткую, персонализированную информацию о конце рабочего дня, определяя закономерности (“Вы выполнили 4 из 5 своих высокоприоритетных задач, и ваше время сосредоточения было выше среднего за неделю, но вы пропустили вечернюю рутину в третий раз на этой неделе”), выявляя корреляции и предлагая одно конкретное намерение на завтра.

Этот синтез максимально приближает платформу к видению, описанному в заявлении о миссии: преобразование исходных данных о деятельности и субъективного опыта в действенный рассказ. Ни одно существующее приложение для повышения производительности или ведения журнала в настоящее время не выполняет этого, поскольку ни одно существующее приложение не имеет одновременного доступа ко всем этим источникам данных.

Планируемая архитектура обслуживания журналов. Сервис `psocial_journal` будет следовать тому же архитектурному шаблону, что и другие сервисы: Kotlin/Ktor, собственная база данных PostgreSQL, аутентификация JWT

с использованием общего секрета и внутренние конечные точки для службы аналитики для извлечения содержимого журнала при создании подсказок искусственного интеллекта. Записи журнала будут синхронизированы с мобильным клиентом по тому же протоколу пакетной синхронизации, который используется selfmanager.

3.6.11 Аналитические данные (страница пользователя)

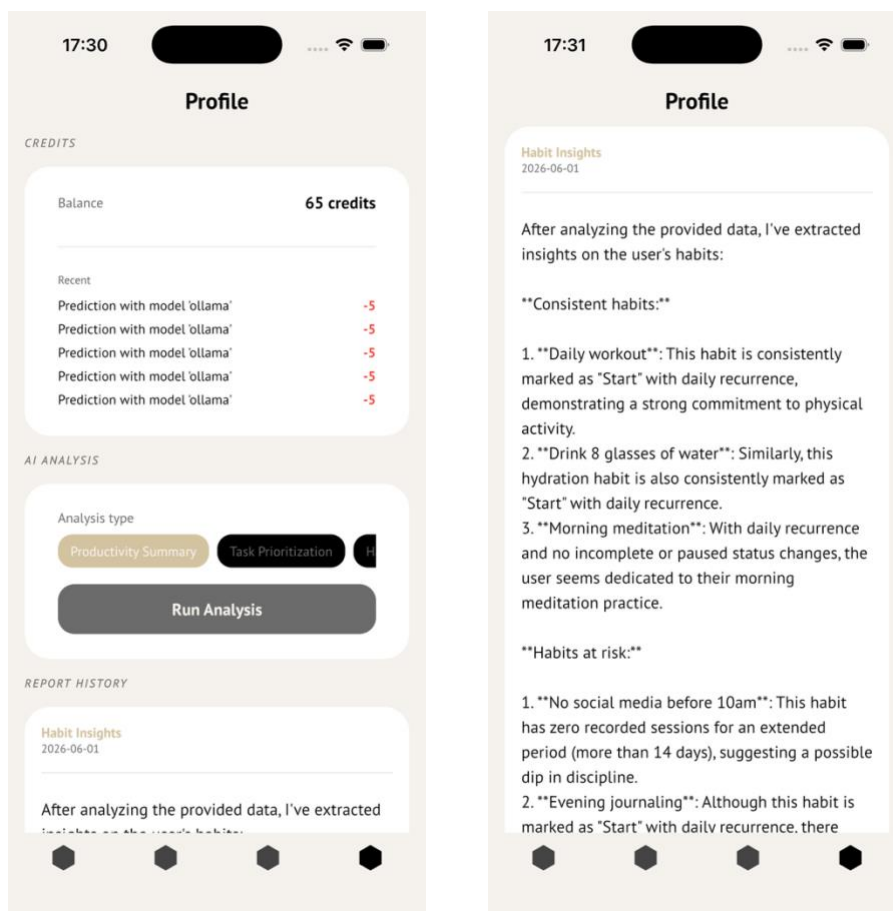


Рисунок 3.17 — Экран аналитики пользователя

Экран страницы пользователя интегрирует функции анализа с помощью искусственного интеллекта сервиса analytics непосредственно в мобильное приложение. AnalyticsApiClient (в commonMain) вызывает POST /api/v1/analytics с типом анализа; идентификатор модели передаётся по умолчанию и не выбирается пользователем. Поддерживаемые типы анализа:

PRODUCTIVITY_SUMMARY, TASK_PRIORITIZATION, HABIT_INSIGHTS, ROUTINE_OPTIMIZATION, WEEKLY_TIMER_SUMMARY и CUSTOM.

InsightsViewModel управляет потоком запуска анализа: он вызывает API, обрабатывает состояние загрузки (показывает индикатор выполнения с пояснительным сообщением, поскольку вывод может занимать несколько секунд) и сохраняет результат в потоке состояний InsightsState, который отображается на экране. Прошлые отчёты извлекаются через GET /api/v1/analytics и отображаются в виде прокручиваемого списка истории. Текущий кредитный баланс и последние 10 транзакций загружаются через GET /api/v1/credits/transactions — баланс возвращается как поле creditsBalance в теле ответа.

3.6.12 Адаптивный макет

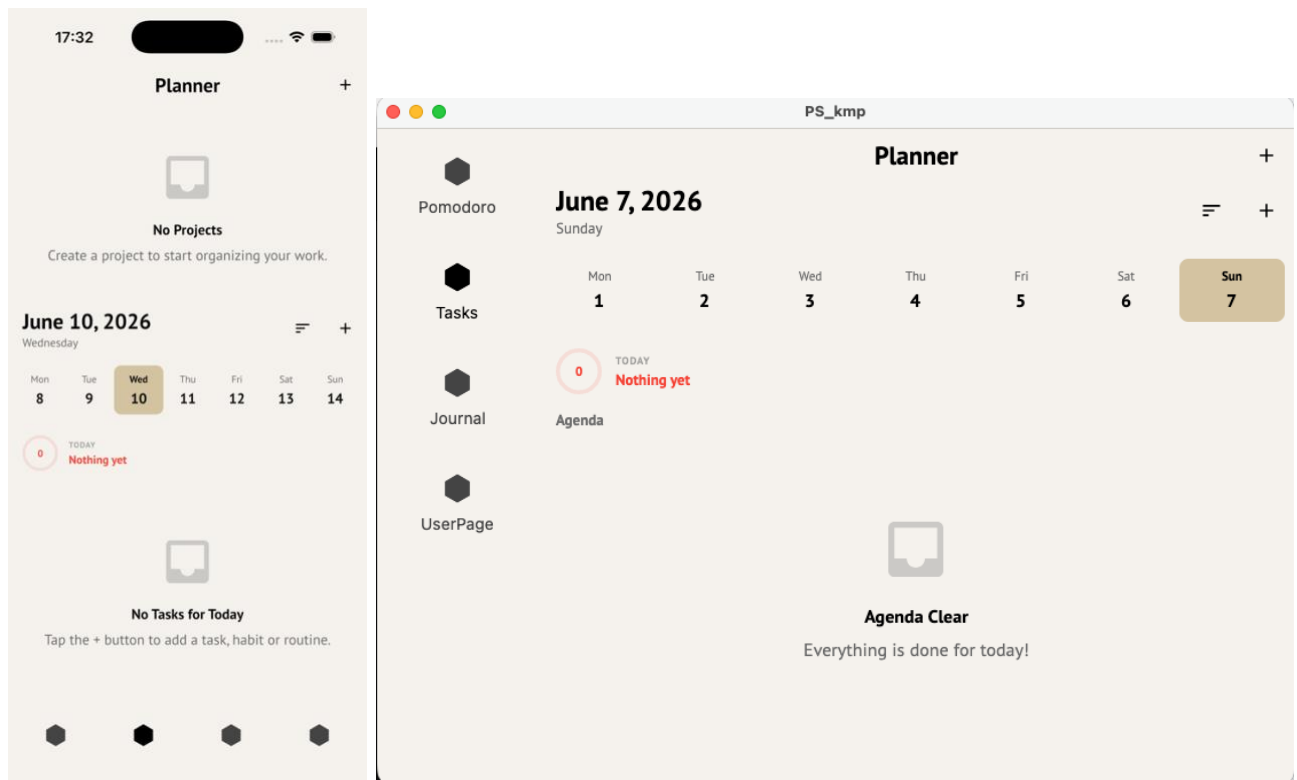


Рисунок 3.18 — Адаптивный макет приложения на разных экранах

Приложение адаптируется к размеру экрана с помощью утилиты `windowSize`, которая классифицирует текущий экран как компактный, средний или расширенный.

На компактных экранах (телефонах) навигация представляет собой нижнюю панель навигации — стандартный шаблон мобильной навигации, видимый в нижней части экрана со значками + ярлыками для четырех пунктов назначения верхнего уровня.

На экранах среднего размера и увеличенного размера (планшеты и настольные компьютеры) навигация переключается на боковую панель навигации — вертикальный столбец значков и меток вдоль левого края, освобождающий для содержимого всю ширину экрана. На больших экранах рабочего стола отображается полный навигационный блок с расширенными надписями.

С помощью составного элемента `NavScaffold` создается соответствующий навигационный компонент с областью содержимого на основе текущего размера `windowSize`, в котором находится окно. Все составные файлы адаптируют свой макет к доступной ширине экранов — списки используют многоколоночные сетки на широких экранах и одностолбцовые списки на телефонах.

3.6.13 Протокол автономной синхронизации

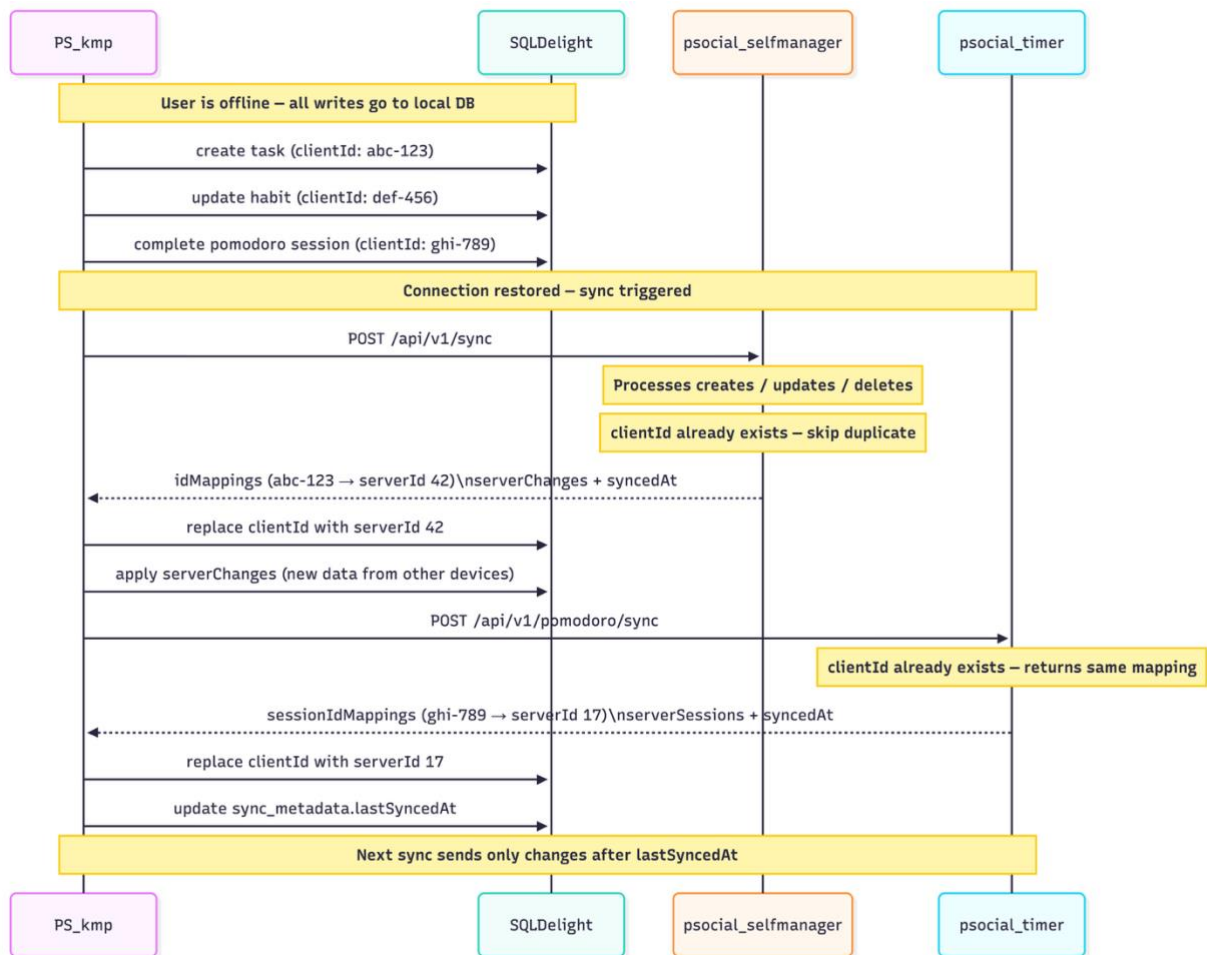


Рисунок 3.19 — Протокол офлайн-синхронизации данных

Уровень синхронизации является основой архитектуры, ориентированной на автономный режим. Каждому объекту присваивается сгенерированный клиентом UUID (`syncId`) во время создания. Этот UUID является ключом идемпотентности — если один и тот же объект отправляется дважды (из-за повторной попытки), сервер распознаёт дубликат `syncId` и возвращает существующий идентификатор сервера вместо создания второй записи.

Selfmanager sync — это одиночный двунаправленный вызов: `POST /api/v1/sync`. Тело запроса содержит:

- `creates`: все объекты, созданные локально с момента последней синхронизации (с `clientId = syncId` и полными данными полей)

- `updates`: все объекты, изменённые локально с момента последней синхронизации (`updatedAt > lastSyncedAt`)
- `deletes`: `serverId`-идентификаторы локально удалённых объектов (надгробий)
- `habitCompletions`: пакет новых и удалённых отметок о выполнении привычек
- `lastSyncedAt`: временная метка из предыдущего ответа синхронизации

Ожидающие создания определяются как строки с `serverId IS NULL`; ожидающие обновления — как строки с `updatedAt > lastSyncedAt`; ожидающие удаления — как строки с `isDeleted = 1 AND serverId IS NOT NULL`. Если задача или привычка создана до того, как родительский проект был синхронизирован, клиент передаёт `projectClientId` (UUID проекта) вместо числового `projectId`, и сервер разрешает зависимость на своей стороне.

Сервер обрабатывает их в порядке зависимости (проекты перед задачами), применяет семантику `upsert` для создания и обновлений и возвращает: `idMappings`: { `clientSyncId` → `serverIntegerId` } для всех обработанных объектов, чтобы локальная база данных могла заполнить свои колонки `serverId` `serverChanges`: полные сущности для всех записей, изменённых на сервере с момента последней синхронизации, а также идентификаторы удалённых на сервере объектов `errors`: сообщения об ошибках для записей, которые не удалось

обработать (поле присутствует в DTO, но не обрабатывается клиентом в текущей реализации) `syncedAt`: новая временная метка для следующего вызова. Получив ответ, `SyncRepositoryImpl.applyResponse()`:

1. Записывает `serverId` в локальные строки для всех синхронизированных объектов (через `updateXxxServerId`)
2. Применяет серверные изменения объектов к локальной базе данных (`upsert` — сервер имеет приоритет при конфликтах)
3. Применяет серверные удаления как мягкие удаления (`isDeleted = 1`) в локальных таблицах
4. Сохраняет новую временную метку `syncedAt` в `sync_metadata`

Синхронизация по таймеру выполняется по тому же шаблону через POST /api/v1/pomodoro/sync в службе таймера, используя lastTimerSyncedAt из таблицы sync_metadata.

Когда подключение недоступно, все операции записываются в локальную SQLDelight с соответствующими маркерами (serverId IS NULL или updatedAt > lastSyncedAt). При следующем вызове syncAll() — в том числе при восстановлении соединения — все накопленные изменения автоматически включаются в запрос синхронизации. Поскольку все операции синхронизации идемпотентны, повторные попытки не приводят к дублированию данных.

3.7 Текущее состояние реализации

Таблица 3.6 — Текущее состояние реализации по модулям

Модуль	Функция	Статус
Аутентификация	Email identify / find-or-create	Частично реализован
	Refresh token (24h access / 7d refresh)	Завершено
	Logout + очистка локальных данных	Завершено
Проекты	CRUD + теги + иконка + цвет	Частично реализован
Задачи	CRUD + приоритет + срочность + дата	Завершено
	Подзадачи	Завершено

	Запланированное время + напоминания	Завершено
	Повторяющиеся задачи	Завершено
	Счётчик затраченного времени (из таймера)	Завершено
	AI «Заполнить с помощью ИИ» (parse-task)	Завершено
	AI приоритизация задач	Завершено
	Индикатор статуса модели (зел./жёлт./красн.)	Завершено
Цели	-	Запланировано
Привычки	CRUD, типы Start / Quit	Завершено
	Отслеживание выполнения (habit_completion)	Завершено
	Счётчик серии + история 7 дней	Завершено
	Подзадачи привычки	Завершено
	AI Habit Coaching	Завершено

Рутини	CRUD + шаги + автозапуск + позиция	Завершено
	Routine Runner (таймер шагов, пауза)	Завершено
	Нерабочее время	Завершено
	Фиксация выполнения (routine_completion)	Завершено
	AI оптимизация рутины	Завершено
Планировщик	Объединённый список задач/привычек/рутин	Завершено
	WeeklyCalendar (выбор даты)	Частично реализован
	Сортировка / Группировка	Завершено
	FocusScoreRow (% выполненных за день)	Завершено
Pomodoro	Автономный таймер (мультицикловый)	Завершено
	Связанный таймер (Task / Habit)	Завершено
	Настройки + Focus Mode	Завершено

	Рекомендованные длительности (с сервера)	Завершено
	Сигнал риска отказа (abandonment risk)	Завершено
	Предложение перерыва (break suggestion)	Завершено
	Статистика (Focus Patterns + By Activity Type)	Завершено
AI / Аналитика	PRODUCTIVITY_SUMMARY	Завершено
	TASK_PRIORITIZATION	Завершено
	HABIT_INSIGHTS	Завершено
	ROUTINE_OPTIMIZATION	Завершено
	WEEKLY_TIMER_SUMMARY	Завершено
	CUSTOM (произвольный промпт)	Завершено
	История отчётов	Завершено
Кредиты	Баланс + история транзакций + пополнение	Частично реализован

Синхронизация	-	Завершено
Дневник	-	Запланировано

Таблица 3.7 — Реализованные эндпоинты по сервисам

Сервис	Функция	Статус
psocial_selfmanager (Kotlin / Ktor, 1226)	Auth: identify / refresh / logout	Завершено
	Projects CRUD + теги	Завершено
	Tasks CRUD + подзадачи + теги	Завершено
	Habits CRUD + расписание + подзадачи	Завершено
	Habit completions (create / delete)	Завершено
	Routines CRUD + шаги	Завершено

	Routine completions	Завершено
	Batched offline sync (двунаправленная)	Завершено
	Internal: tasks / habits / routines / time-log	Завершено
	Welcome deposit при регистрации (billing)	Завершено
psocial_timer (Kotlin / Ktor, 1227)	Auth (shared users table через JDBC)	Завершено
	Pomodoro settings GET / PUT	Завершено
	Sessions: create / start / pause / resume	Завершено
	Sessions: complete / abandon	Завершено
	Intervals (work / short break / long break)	Завершено
	Break suggestions (после завершения интервала)	Завершено
	Abandonment risk (при паузе)	Завершено
	Recommended durations (на основе истории)	Завершено

	Focus Patterns (byHour / byDayOfWeek / peak)	Завершено
	Entity Stats (по типам объектов)	Завершено
	Batched offline sync (сессии + интервалы)	Завершено
	Internal: aggregate pomodoro stats	Завершено
	Fire-and-forget time-log → selfmanager	Завершено
psocial_analytics (Kotlin / Ktor, 1228)	Trigger analysis (POST /api/v1/analytics)	Завершено
	PRODUCTIVITY_SUMMARY	Завершено
	TASK_PRIORITIZATION	Завершено
	HABIT_INSIGHTS	Завершено
	ROUTINE_OPTIMIZATION	Завершено
	WEEKLY_TIMER_SUMMARY	Завершено
	CUSTOM (произвольный промпт)	Завершено
	Parse task / habit / routine (NLP)	Завершено

	Model status check	Завершено
	История отчётов (GET /api/v1/analytics)	Завершено
	Credits proxy (balance / transactions)	Завершено
	Admin endpoints (stats / users / reports)	Завершено
	Internal stats endpoint	Завершено
psocial_billing (Python / FastAPI, 1229)	Auth: find-or-create by email	Завершено
	User profile + баланс + транзакции	Завершено
	ML Models registry CRUD	Завершено
	LLM routing: Ollama (llama3.2)	Завершено
	LLM routing: Anthropic API	Завершено
	LLM routing: OpenAI API	Завершено
	Internal predict (X-Internal-Key)	Завершено
	Internal users: баланс / депозит	Завершено
	Predictions list + stats	Завершено

	Admin: управление пользователями + статистика	Завершено
psocial_dashboard (Admin, 1230)	Login (через selfmanager + analytics)	Завершено
	Overview: системная статистика	Завершено
	Users: балансы / пополнение / повышение прав	Завершено
	Models: регистрация / удаление LLM-моделей	Завершено
	Predictions: системный журнал предсказаний	Завершено
	Analytics: все отчёты пользователей	Завершено

На текущем этапе все четыре серверных сервиса (selfmanager, timer, analytics, billing) и панель мониторинга администратора упакованы в Docker и развёрнуты на Render; анализ с использованием ИИ проверен сквозным образом как локально, так и в облаке Render (через туннель ngrok к локальному экземпляру Ollama). Мобильный клиент на Kotlin Multiplatform реализует основные экраны — задачи, привычки, рутины, проекты, таймер Pomodoro и аналитику — с локальной базой данных SQLDelight и двунаправленной офлайн-синхронизацией. Модуль дневника пока реализован как интерфейсный каркас, а цели — как сквозная функция, встраиваемая в selfmanager и дневник, — ещё не

подключены к слою данных; оба элемента остаются следующим этапом разработки клиента.

3.8 Определение области применения MVP

MVP ProductiveSocial был определен как минимальный набор возможностей, который демонстрирует основное ценностное предложение — унифицированное отслеживание производительности с анализом ИИ - способом, доступным реальному пользователю.

Модуль дневника и сквозная функция целей (встраиваемая одновременно в планировщик и дневник) входят в область MVP и находятся в разработке; их планируется завершить до выпуска. MVP включает в себя четыре серверных сервиса, одна панель администратора и API эндпоинты, необходимые для поддержки мобильного приложения. MVP явно не включает в себя: социальные функции, проприетарные обученные модели и хостинг LLM производственного уровня.

Эта область была выбрана по трем причинам:

1. Основное ценностное предложение (унифицированный междоменный анализ искусственного интеллекта) полностью демонстрируется при текущей реализации. Пользователь может добавлять свои задачи, привычки, распорядок дня и сеансы фокусировки и получать качественно более подробный анализ, чем может предоставить любое существующее приложение.
2. Отложенные функции (социальные сети) требуют значительных проектных работ, выходящих за рамки серверной архитектуры — они связаны со сложными задачами пользовательского интерфейса, которые лучше всего решать после проверки основных модулей повышения производительности.
3. Архитектура явно учитывает отложенные функции. В модели данных предусмотрены ячейки для записей журнала и заметок (объектов, которые

ссылаются на ту же личность пользователя и структуру проекта, что и задачи и привычки). Социальная функция может быть реализована как новый микросервис без изменения какого-либо существующего сервиса.

3.9 Планируемые сервисы и функции

3.9.1 Расширенный самоменеджмент: помимо задач, привычек и рутины

Текущий сервис `selfmanager` содержит три структурированных типа объектов производительности — задачи, привычки и рутины. Это объекты высочайшей сложности и ценности в области личной продуктивности, которые были правильной отправной точкой. Однако полнофункциональное приложение для управления задачами и персональной организации включает в себя более широкий спектр типов сущностей, которые служат для различных вариантов использования.

Списки дел и быстрый просмотр входящих сообщений. Не каждый элемент, который пользователь хочет зафиксировать, представляет собой структурированную задачу с приоритетом, сроком выполнения и подзадачами. Многие потребности в производительности проще: список покупок, список рекомендаций по книгам, небольшая заметка для себя. Списки дел - это легковесные, неупорядоченные или произвольно упорядоченные списки однострочных элементов, без механизма планирования и приоритизации полной задачи. "Входящие с быстрым захватом" - это специальный проект, который получает все элементы, которые пользователь фиксирует, без их немедленной классификации — временная промежуточная площадка для мыслей и обязательств, которые будут обработаны (перемещены в нужный проект и тип объекта) во время еженедельного обзора.

Контрольные списки. Контрольные списки - это списки элементов многократного использования, которые не являются повторяющимися привычками или рутинами, но используются неоднократно в похожих контекстах. Предполетный чек-лист для рабочей поездки, список вещей, которые нужно проверить перед отправкой отчета, чек-лист после тренировки — это не привычки (они не отслеживаются на предмет ежедневных нагрузок) и не рутины (они не запланированы), но они выигрывают от структуры упорядоченного списка с отслеживанием выполнения.

Шаблоны. Шаблоны проектов - это predetermined структуры, которые пользователь может создавать при запуске новой работы. Шаблон “Начало нового проекта” может предварительно заполнять набор задач (определить область применения, запланировать стартовую встречу, определить заинтересованных лиц), которые обычно требуются пользователю для каждого нового проекта. Шаблоны уменьшают трудности при запуске повторяющихся типов проектов.

Просмотр календаря. Навигация по полоске даты на экране самостоятельного менеджера - это современный подход к организации. Полный просмотр календаря — ежемесячные и еженедельные форматы таблиц, показывающие запланированные задачи, сроки выполнения привычек и рутинные графики на расширенном временном горизонте — дает пользователю более широкую временную перспективу своих обязательств и помогает определить дни с превышением графика до их наступления.

Предварительный просмотр стоимости ИИ-запроса. В текущей реализации списание кредитов за анализ происходит после выполнения запроса, и пользователь видит израсходованную сумму только по факту. Планируемое улучшение показывает ориентировочную стоимость запроса до его отправки — рядом с кнопкой «Сгенерировать» отображается количество кредитов, которое будет списано, и текущий баланс. Это делает расход кредитов прозрачным и предсказуемым, позволяя пользователю принимать осознанное решение перед каждым обращением к модели.

Заметки со связыванием сущностей. Заметки - это текстовые документы произвольной формы, которые используются в контексте производительности. Отличительная черта заметок в ProductiveSocial (по сравнению отдельное приложение для заметок) - это привязка объекта: заметка может быть прикреплена к проекту, задаче, привычке или рутине. Заметки о собрании прикрепляются к задаче “Посетить совещание по планированию третьего квартала”. Исследовательские заметки прилагаются к проекту, о котором они информируют. Это контекстное вложение означает, что заметки всегда можно

найти через объект, к которому они относятся, а не только через интерфейс поиска.

Онбординг при первом запуске. Текущая аутентификация по email сразу выводит пользователя на главный экран без какого-либо вводного сопровождения. Планируемый онбординг — это короткая последовательность экранов при первом входе, которая знакомит пользователя с ключевыми сущностями (проекты, задачи, привычки, рутины), помогает создать первый проект и задачу, объясняет работу таймера Pomodoro и ИИ-функций. Онбординг снижает порог входа и повышает вероятность того, что новый пользователь дойдёт до момента получения ценности от приложения.

Отслеживание выполнения шагов рутин по датам. Завершение рутины целиком уже фиксируется с ежедневной детализацией в таблице `routine_completion` — аналогично `habit_completion`. Однако выполнение отдельных шагов рутины в текущей реализации хранится как единственный глобальный булев флаг `completed` на каждом шаге и не привязано к конкретной дате. Это означает, что сброс рутины (кнопка **Refresh**) сбрасывает шаги глобально, а не только для выбранной даты. Планируемое расширение вводит покомпонентный учёт завершения шагов по датам: для каждой пары (шаг, дата) сохраняется отдельная запись, что позволит отображать прогресс шагов корректно при просмотре прошлых дат.

3.9.2 Расширенный таймер: общий фокус и полосы

Текущий Pomodoro timer - это инструмент персональной ориентации. Планируемые расширения привносят социальные аспекты и геймификацию в работу.

Полосы фокусировки. Полоса фокусировки отсчитывает дни подряд, в течение которых пользователь завершил хотя бы один сеанс Pomodoro. Подобно полосам привычек, полосы сосредоточенности используют неприятие потерь для поощрения постоянной ежедневной вовлеченности. Счетчик серий будет

отображаться на видном месте на главном экране таймера и будет виден в общедоступном профиле пользователя.

Цели сеанса. Перед началом сеанса пользователь может установить цель сеанса — краткое текстовое описание того, чего он намерен достичь в этом блоке. По завершении сеанса цель представляется вместе с кратким описанием сеанса, что позволяет пользователю поразмыслить о том, достигли ли они того, что намеревались сделать. Цели сеанса сохраняются в записи сеанса и становятся частью данных, доступных службе аналитики при создании анализа с помощью искусственного интеллекта.

Общие комнаты фокусировки. Комната фокусировки - это виртуальное общее пространство, где несколько пользователей одновременно выполняют фокусировку. Таймер каждого участника работает независимо (они могут иметь разную продолжительность и схему перерывов), но комната показывает состояние всех участников в режиме реального времени (сосредоточены / на перерыве / завершены). Социальная заметность создает мягкий эффект подотчетности — знание того, что другие могут видеть, сосредоточены ли вы или отказались от сеанса, увеличивает цену отказа. Фокус-комнаты будут реализованы как часть социальной функциональности.

3.9.3 Социальный слой

Социальный модуль - это функция, которая наиболее четко отличает ProductiveSocial от инструмента личной продуктивности и превращает его в платформу. В основе его дизайна лежит один принцип: использовать мотивирующую силу социальной ответственности.

Социальных сетей. Основные социальные платформы (Instagram, Twitter / X, TikTok) предназначены для максимального увеличения времени пребывания на платформе за счет бесконечной прокрутки, алгоритмического добавления ленты и повторного вовлечения с помощью уведомлений. Эти механизмы прямо противоречат цели целенаправленной продуктивной работы. Внедрение обычной

социальной ленты в приложение для повышения производительности подорвет саму цель приложения.

Социальный дизайн ProductiveSocial. Социальный модуль архитектурно отделен от рабочего пространства чтобы не мешать производительности пользователя. На экранах самообслуживания или таймера нет встроенной социальной ленты. Социальные функции доступны через специальную страницу и предназначены для обмена выполненной работой и постоянными привычками, а не эфемерным контентом.

Планируемыми социальными функциями являются:

Общедоступные профили. У каждого пользователя есть общедоступная страница профиля с отображаемым именем, биографией и общедоступными достижениями. Профиль не отображает статус в режиме реального времени или прямую трансляцию активности — это сводка достижений, а не интерфейс наблюдения.

Обмен достижениями. Платформа определит набор достижений, заработанных за достижение контрольных показателей: выполнение определенного количества задач, поддержание привычки в течение 30 дней, накопление 100 часов сосредоточенности, выполнение полной недели утренних процедур. Заработанные достижения отображаются в виде значков в общедоступном профиле пользователя и могут быть опубликованы в ленте сообщества.

Схемы привычек. Пользователи, выработавшие успешные привычки, могут публиковать их в виде схем - общих шаблонов, которые могут перенять другие пользователи. Схема привычек “Ежедневное чтение продолжительностью 30 минут”, опубликованная одним пользователем, может быть принята другим пользователем одним нажатием, предварительно заполнив всю конфигурацию привычек (тип, повторяемость, запланированное время, напоминания). Это создает одноранговую базу знаний об эффективных привычках.

Лента сообщества. Простая хронологическая лента достижений, которыми делятся пользователи, на которые подписан текущий пользователь. В отличие от алгоритмических социальных каналов, в ленте сообщества нет рейтинга, продвигаемого контента и уведомлений. Это низкая осведомленность окружающих о том, чего добиваются другие в вашей сети, предназначенная для просмотра один раз в день, а не для принудительной проверки.

Партнеры по подотчетности. Пользователь может назначить конкретные связи партнерами по подотчетности для достижения определенной привычки или цели. Партнеры по подотчетности получают еженедельный отчет о прогрессе друг друга в достижении общей цели — тихий, не вызывающий трения толчок, который имеет более сильную эмпирическую поддержку для изменения поведения, чем общественное давление.

многие из этих социальных сервисов уже существуют, будь то в виде определенного веб-приложения, канала discord или любого другого сообщества, но productive social объединит все эти сервисы в сжатой форме.

Архитектура. Социальный сервис будет представлять собой новый набор независимых микросервисов, следующих тем же шаблонам, что и существующие сервисы: собственная база данных PostgreSQL, аутентификация JWT с помощью общего секрета и внутренние конечные точки для аналитического сервиса для включения социальных данных (например, показателей подотчетности партнеров) в анализ искусственного интеллекта, отличаться в язык программирования, для поддержки такого сервиса лучше пойдет язык программирования GoLang. Социальный сервис будет использовать данные из планировщика и таймера через внутренние API, но не будет владеть копией данных о пользователях — он будет ссылаться на идентификаторы объектов и идентификаторы пользователей, не дублируя сами объекты.

3.9.4 Пользовательский Сервис

В настоящее время идентификация пользователя в ProductiveSocial минимальна: адрес электронной почты, назначенный сервером целочисленный идентификатор и JWT. Этого достаточно для текущего этапа, когда платформа не имеет социального измерения и не ориентирована на общественность.

При внедрении социальных функций требуется более полное представление о пользователе: отображаемое имя, фотография профиля, биография, часовой пояс, настройки уведомлений, настройки конфиденциальности и социальный график (кто за кем следит). Эти данные не должны храниться в selfmanager — сервис selfmanager владеет данными о производительности, а не об обогащении личности пользователя — и не должны храниться в социальной службе, которая владеет социальными взаимодействиями, а не данными профиля.

Сервис psocial_user изначала будет выделенным владельцем данных профиля пользователя. Это покажет:

- GET /api/v1/users/me — собственный профиль аутентифицированного пользователя
- PATCH /api/v1/users/me — обновите поля профиля (отображаемое имя, биографию, фотографию)
- GET /api/v1/users/{id} — публичный просмотр профиля другого пользователя
- POST /api/v1/users/{id}/follow — подписываться на пользователя
- DELETE /api/v1/users/{id}/unfollow — отменить подписку
- GET /api/v1/users/me/followers — following graph

Сервис будет использовать тот же секрет JWT, что и другие сервисы, и будет доступен всем сервисам для получения данных профиля.

3.9.5 Проприетарные модели ML: стратегия накопления данных

Текущий конвейер анализа искусственного интеллекта использует большие языковые модели для всего анализа — преднамеренный выбор для ранней стадии платформы, когда нет данных о поведении пользователей для обучения. По мере того, как платформа накапливает данные о реальном использовании, параллельная разработка собственной прогностической модели становится жизнеспособной.

Планируемый переход - это не замена анализа на основе LLM, а дополнение. Определенные задачи прогнозирования, к которым LLM подходят качественно (“ваша привычка сопряжена с риском”), могут быть решены более точно обученными моделями, которые извлекли уроки из тысяч реальных пользовательских траекторий:

Таблица 3.8 — Сопоставление типов задач проприетарным моделям ML

Прогнозирующая задача	Тип модели	Обучающий сигнал
Вероятность сохранения привычки	Логистическая регрессия / повышение градиента	Журналы выполнения с историей серий
Вероятность завершения задачи при текущей нагрузке	Повышение градиента	Журналы выполнения задач с контекстными функциями
Оптимальная продолжительность сеанса фокусировки для данного пользователя	Регрессия	Завершенный сеанс в сравнении с историей заброшенных сеансов
Ежедневный показатель производительности	Нейронная сеть	Совокупность сигналов задачи, привычки и сосредоточенности

Прогнозирующая задача	Тип модели	Обучающий сигнал
Риск отказа во время сеанса	Повышение градиента	Количество пауз и продолжительность сеанса при завершении

Эти модели будут обучены извне и загружены в реестр моделей биллинговой службы в виде файлов .pkl через существующую конечную точку POST /api/v1/models/{id}/upload. Способ вывода sklearn биллинговой службы — уже реализованный и готовый к приему моделей — будет загружать их через joblib и вызывать их вместе со структурированными данными об активности пользователя в качестве функций. Остальная часть системы (аналитика, мобильное приложение, информационные панели) не требует изменений; модель отображается в окне выбора модели рядом с параметрами LLM.

3.10 Веб-панели мониторинга

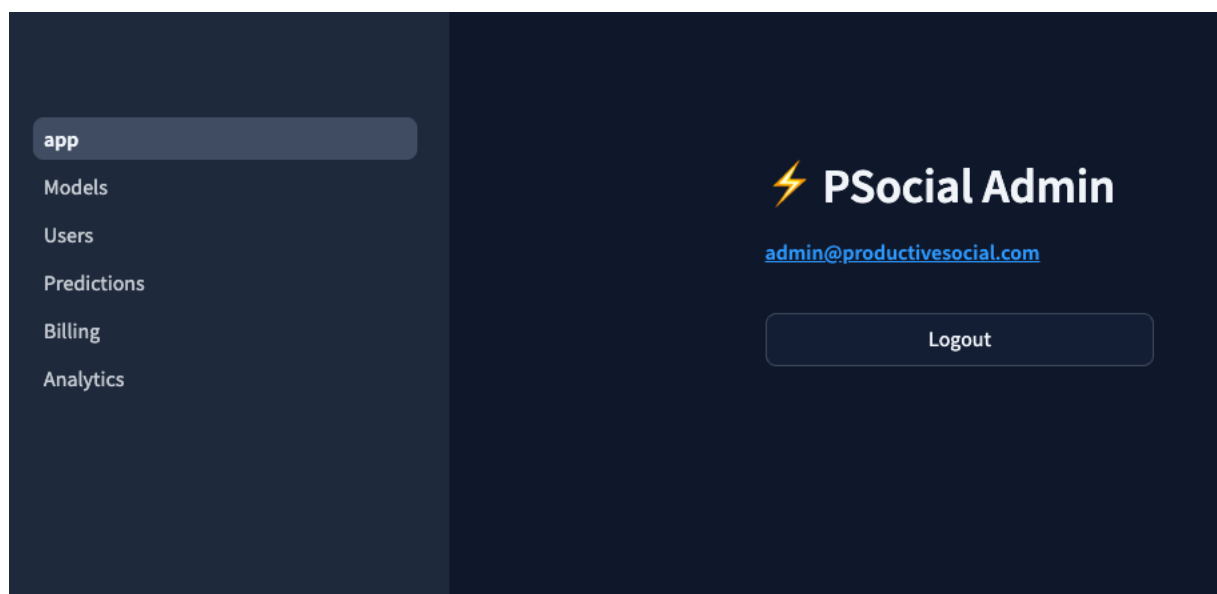


Рисунок 3.20 — Административная панель мониторинга

app	User Management				
Models					
Users					
Predictions					
Billing					
Analytics					
	Email	Selfmanager ID	Reports	Admin	Since
	test@productivesocial.com	4	5	✗	2026-05-11
	alice@test.com	5	2	✗	2026-05-11
	user4@mail.com	14	4	✗	2026-05-11
	user3@mail.com	13	1	✗	2026-05-11
	admin@productivesocial.com	2	0	✓	2026-05-12
	nlptest@productivesocial.com	17	0	✗	2026-05-29
	user@productivesocial.com	18	0	✗	2026-05-29
	junior@user.com	19	0	✗	2026-05-29
	antoninho@user.com	20	0	✗	2026-05-29
	joaquim@user.com	26	0	✗	2026-05-29

Панель администратора: управление пользователями

app

Models

Users

Predictions

Billing

Analytics

Credit flow

Deposited

Consumed

Remaining in accounts

1150

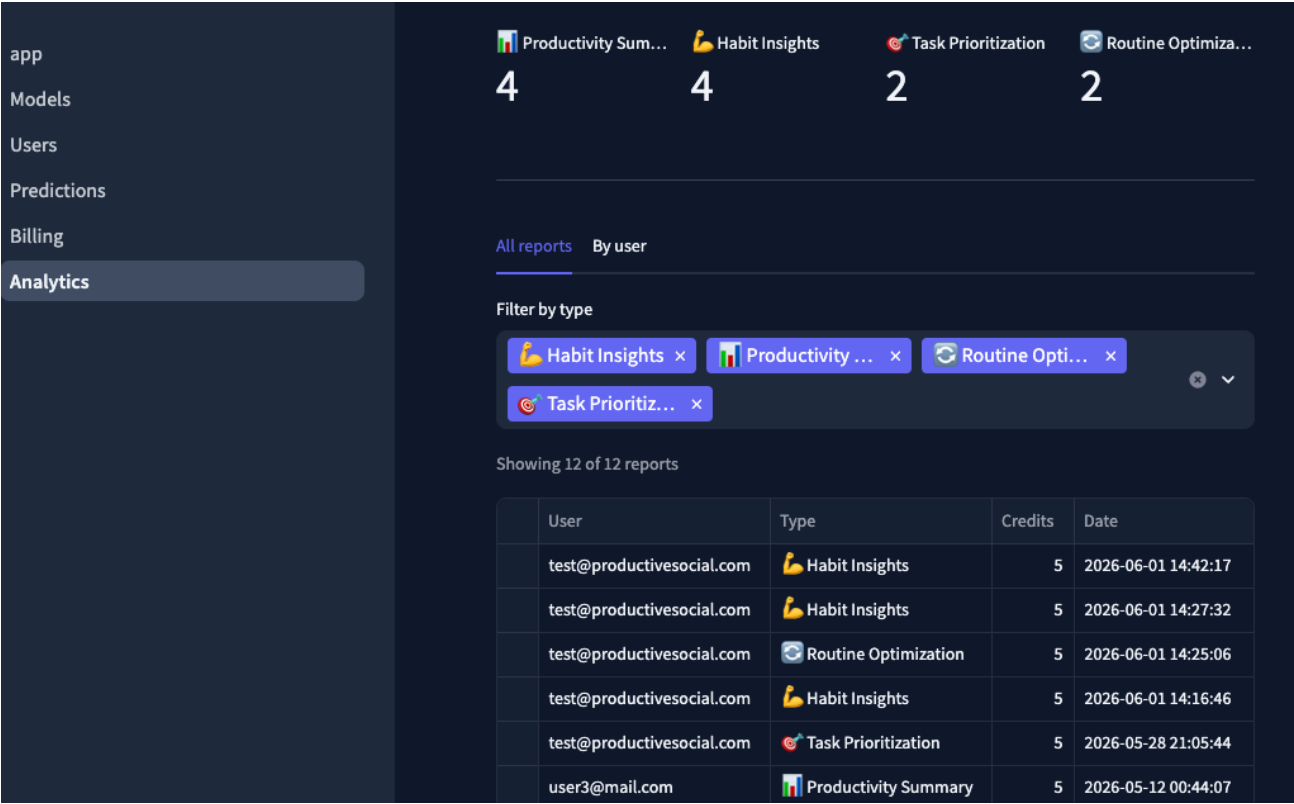
485

665

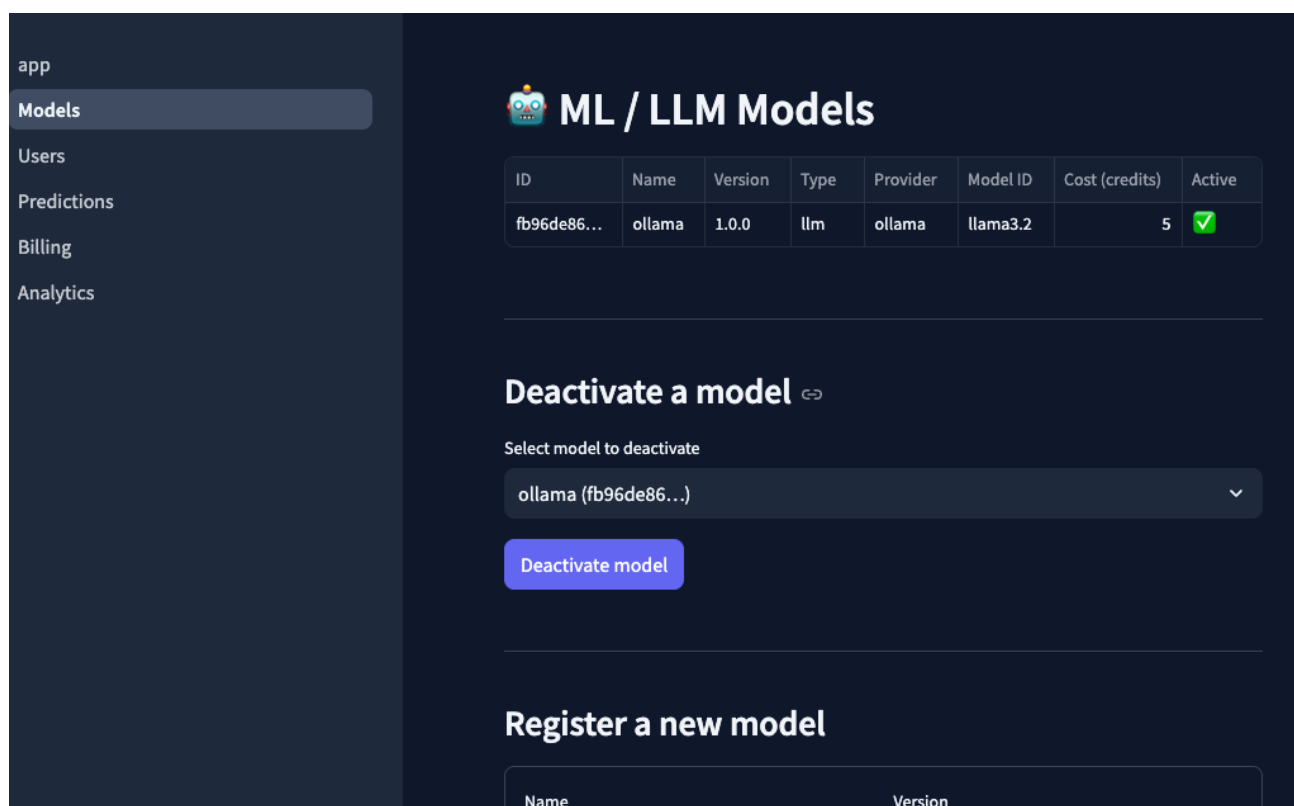
Recent predictions

User ID	Model	Status	Context	Credits	Date
4	fb96de86...	✓ success	habit_parse	5	2026-06-07 14:24:19
4	fb96de86...	✓ success	ping	5	2026-06-07 14:24:06
4	fb96de86...	✓ success	task_parse	5	2026-06-07 14:21:22
4	fb96de86...	✓ success	ping	5	2026-06-07 14:21:05
4	fb96de86...	✓ success	task_parse	5	2026-06-07 13:59:26
4	fb96de86...	✓ success	ping	5	2026-06-07 13:59:23
4	fb96de86...	✓ success	task_parse	5	2026-06-07 13:58:41
4	fb96de86...	✓ success	ping	5	2026-06-07 13:58:27
4	fb96de86...	✓ success	task_parse	5	2026-06-07 13:56:31

Панель администратора: биллинг



Панель администратора: аналитика



Панель администратора: управление ML-моделями

ProductiveSocial предоставляет веб-панель на основе Streamlit: панель администратора для системных операторов.

Почему были созданы информационные панели вместо собственных интерфейсов. Создание полнофункционального собственного интерфейса администратора параллельно с остальной платформой значительно задержало бы разработку внутренних сервисов, которые являются ядром ценностного предложения проекта.

Таким образом, панели Streamlit - это явно краткосрочные решения, призванные сократить разрыв. Streamlit был выбран специально потому, что он позволяет написать функциональный многостраничный веб-интерфейс на нескольких сотнях строк Python без каких—либо накладных расходов на разработку внешнего интерфейса.

Панель администратора выполняет три основные операционные функции: наблюдение за активностью пользователей и системной статистикой,

управление реестром LLM-моделей в биллинговом сервисе и пополнение кредитного баланса пользователей для работы с функциями ИИ. Прямые вызовы биллингового сервиса выполняются с заголовком X-Internal-Key — как и предусмотрено для доверенных клиентов уровня сервиса.

В перспективе административная панель должна быть заменена полноценным собственным приложением, обеспечивающим более богатый и безопасный операционный интерфейс. Текущая реализация на Streamlit остаётся достаточной для этапа разработки платформы.

3.11 Развертывание

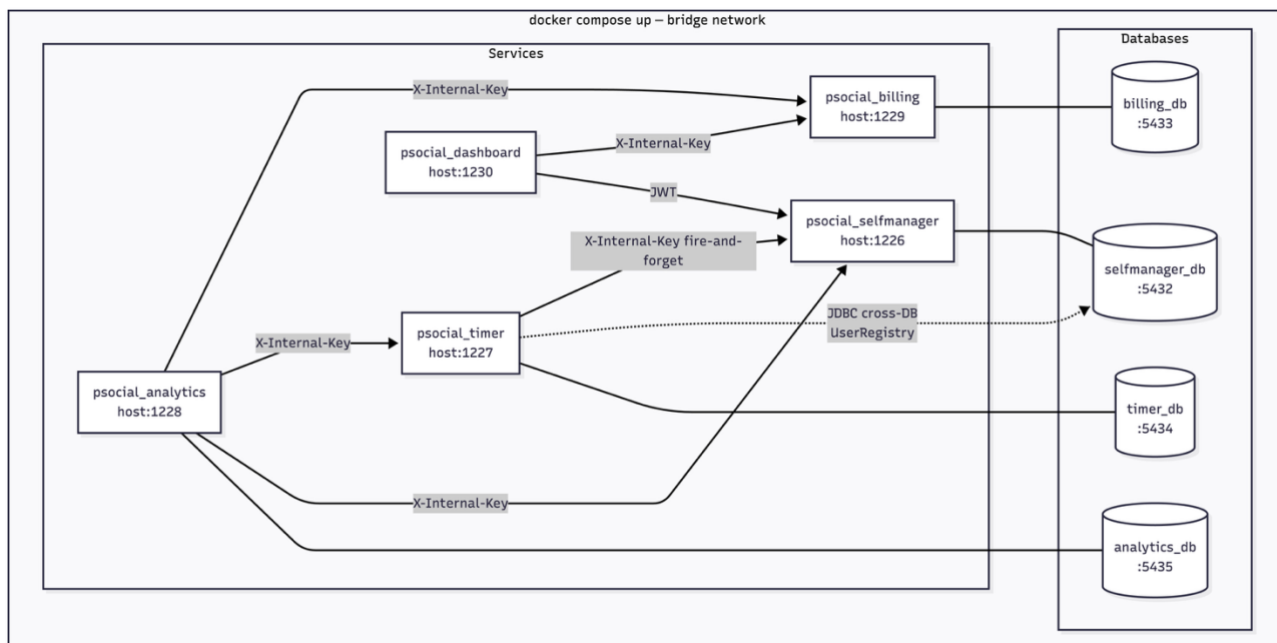


Рисунок 3.21 — Архитектура развёртывания сервисов

Локальная разработка с помощью Docker Compose. Все сервисы и базы данных выполняются в контейнерах Docker в общей мостовой сети.

Продакшен-развёртывание на Render. Каждая служба развёрнута как отдельный веб-сервис Render, а базы данных работают на Neon PostgreSQL. JAR-файлы Kotlin собираются локально и коммитятся в git перед отправкой, поэтому Render копирует готовый JAR через Dockerfile без шага сборки. Межсервисные URL используют публичные имена хостов Render. Поскольку локальная модель

Ollama не размещается в облаке, служба billing на Render обращается к локальному экземпляру Ollama через туннель ngrok. На бесплатном тарифе Render службы уходят в сон при простое и имеют холодный старт порядка 50 секунд для JVM-сервисов при первом запросе.

3.12 Тестирование

Ручное тестирование API. Все конечные точки сервиса были протестированы вручную с использованием HTTP-клиентов (Postman и curl) во время разработки. Подход к тестированию заключался в индивидуальной проверке как правильного пути, так и ключевых ошибок (недопустимый JWT, недостаточное количество кредитов, отсутствующие обязательные поля, неизвестные идентификаторы объектов).

Сквозное тестирование трубопровода. Наиболее важным тестированием был полный конвейер сквозного анализа: создайте тестовые данные в selfmanager → создайте сеансы Pomodoro в timer → вызовите аналитику из панели управления пользователями → проверьте, что выходные данные LLM ссылаются на конкретные пользовательские данные. Этот тест был успешно запущен как локально (через Docker Compose), так и в рабочей среде (на Render с помощью туннеля ngrok к Ollama).

Проверка идиомпотентности синхронизации. Конечная точка пакетной синхронизации была протестирована путем повторной отправки одного и того же запроса на синхронизацию и проверки того, что не было создано повторяющихся записей. Тест пройден для всех типов сущностей.

Проверка арифметической кредитоспособности. Кредиты расходуются при каждом обращении к ИИ-функциям платформы: генерация аналитических отчётов (все типы анализа — PRODUCTIVITY_SUMMARY, TASK_PRIORITIZATION, HABIT_INSIGHTS, ROUTINE_OPTIMIZATION, WEEKLY_TIMER_SUMMARY, CUSTOM), NLP-разбор задач, привычек и рутин («Заполнить с помощью ИИ»), приоритизация задач, коучинг привычек и оптимизация рутин. Все эти операции проходят через единую внутреннюю конечную точку биллингового сервиса POST /api/v1/internal/predict, которая

атомарно списывает кредиты и записывает транзакцию. Для каждого прогноза в тестовом наборе баланс до операции, стоимость операции и баланс после операции были проверены вручную на соответствие $\text{balance_before} - \text{cost} = \text{balance_after}$.

Тестирование клиента в процессе разработки. Клиентское приложение тестировалось непосредственно в процессе разработки через использование: при реализации каждой новой функции она подключалась к уже протестированному бэкенд-сервису, после чего проверялась работоспособность всего потока на соответствие ожидаемому поведению. В дальнейшем планируется написание автоматизированных тестов — как интеграционных, так и модульных (unit-тестов).

Что не тестировалось. Нагрузочное тестирование, тестирование производительности при параллельных запросах и тестирование хаоса (имитирующее частичные сбои в обслуживании под нагрузкой) не проводились. Это стандартная практика для производственных систем, и она будет проводиться до любого публичного выпуска.

3.12.1 Тестирование надёжности и отказоустойчивости

В дополнение к функциональному тестированию отдельных API-эндпоинтов был проведён ряд экспериментов, верифицирующих заявленные свойства отказоустойчивости системы: поведение при параллельных запросах от нескольких пользователей, задержки в прогретом и холодном состоянии, отказ отдельных сервисов в период обработки запроса и потеря межсервисных событий при недоступности вышестоящего сервиса. Все тесты проводились на развёрнутых сервисах платформы Render.

Тест на параллельную нагрузку. Для проверки поведения системы при одновременных запросах от нескольких клиентов был запущен сценарий с 10 параллельными запросами к сервису аналитики. Все 10 запросов завершились успешно со статусом HTTP 200. Время ответа составило от 0,55 до 1,50 секунды на запрос при суммарном времени выполнения 1,53 секунды — что означает практически линейное масштабирование пропускной способности без

деградации под нагрузкой. Ошибок, связанных с гонкой состояний, не зафиксировано.

Задержки в прогретом состоянии. После активации всех сервисов время ответа на отдельный запрос составило: `selfmanager` — 0,67–0,97 секунды; `timer` — 0,57 секунды; `analytics` — 0,61–0,98 секунды; `billing` — 0,55 секунды. Сквозной запрос, проходящий через аналитику и биллинг (2 межсервисных вызова), завершился за 0,78 секунды. Данные показатели соответствуют ожидаемым характеристикам REST/HTTP взаимодействия в пределах одной платформы развёртывания.

Задержки холодного старта. После 15 и более минут простоя сервисы бесплатного тарифа Render завершают работу. Время первого успешного ответа после повторного запуска составило: `selfmanager` — 46 секунд; `analytics` — 40 секунд; `timer` — 53 секунды; `billing` — 23 секунды. Разница между JVM-сервисами (~40–53 с) и Python-сервисом (~23 с) объясняется временем инициализации виртуальной машины Java и установки пула соединений с базой данных. Суммарное время восстановления полной доступности системы при последовательном холодном запуске всех сервисов не превышает 53 секунд, поскольку сервисы поднимаются параллельно при поступлении первых запросов.

Отказ `billing` в период аналитического запроса. При недоступности сервиса `billing` сервис аналитики вернул статус HTTP 503 с сообщением "Billing service unavailable". Баланс пользователя не изменился. Данное поведение является проектно корректным: `billing` — единственная невозстанавливаемая зависимость конвейера анализа, поскольку без него невозможны ни LLM-инференс, ни списание кредитов. Ошибка возвращается пользователю немедленно без частичного результата.

Отказ `selfmanager` в период аналитического запроса. Анализ исходного кода сервиса аналитики подтверждает, что вызов к `selfmanager` (получение задач, привычек и рутин) выполняется в блоке с обработкой исключений: при недоступности `selfmanager` соответствующие поля контекста устанавливаются в

пустые значения и анализ продолжается с данными Pomodoro из timer. Данный паттерн симметричен ранее верифицированному сценарию недоступности timer (эксперимент №9), в котором анализ завершился успешно без блока Pomodoro. Таким образом, graceful degradation верифицирован для обеих вышестоящих зависимостей аналитики.

Потеря fire-and-forget события при недоступности selfmanager. Анализ исходного кода сервиса timer подтверждает следующее поведение: при завершении Pomodoro-интервала, привязанного к объекту, timer отправляет HTTP-запрос обновления time_spent_minutes в selfmanager без ожидания ответа и без механизма повторных попыток. При недоступности selfmanager в момент отправки запрос теряется безвозвратно: значение time_spent_minutes связанного объекта не обновляется. Масштаб потери ограничен одной Pomodoro-сессией (максимум 25 минут накопленного времени). Все остальные данные платформы защищены идемпотентным офлайн-синхронизированием с гарантией отсутствия потерь. Производственным решением для данного случая является замена паттерна fire-and-forget на retry с exponential backoff (повторные попытки с интервалами 1, 2, 4, 8 секунд) либо паттерн transactional outbox: timer атомарно записывает событие обновления времени в таблицу pending_time_logs, а отдельный фоновый процесс доставляет его в selfmanager с гарантией exactly-once семантики.

Оценка RTO и RPO. Практический RTO системы на платформе Render: при одновременном холодном запуске всех сервисов полная доступность восстанавливается за время наибольшей из задержек холодного старта, то есть не более 53 секунд (timer). На платном тарифе с постоянно запущенными контейнерами RTO определяется только временем перезапуска процесса и составит менее 30 секунд для JVM-сервисов. Практический RPO: нулевые потери для всех данных, управляемых офлайн-синхронизацией (задачи, привычки, рутины, сессии Pomodoro); потенциальная потеря одной записи time_spent_minutes (не более 25 минут) при отказе selfmanager в момент fire-and-forget вызова.

3.13 Интеграция сервисов: сквозные потоки запросов

В этом разделе прослеживается полный путь выполнения пяти ключевых операций системы — от запроса клиента до окончательного ответа, точно показывая, как участвует каждая служба.

Поток 1: Проведение анализа с использованием искусственного интеллекта (полный сквозной анализ)

Это центральная функция платформы. Она задействует все четыре сервиса, два межсервисных вызова для сбора данных и вывод LLM.

1. Клиент (KMP) отправляет POST `/api/v1/analytics` с типом анализа и Bearer-токеном → analytics
2. Analytics валидирует JWT с помощью общего JWT_SECRET
3. Analytics собирает данные пользователя из selfmanager: GET `/internal/users/{userId}` (заголовок X-Internal-Key) — возвращает задачи, привычки, рутины
4. Analytics запрашивает статистику Pomodoro из timer: GET `/internal/users/{userId}/stats` (заголовок X-Internal-Key) — возвращает агрегированные минуты и количество сеансов
5. Analytics формирует промпт, включая все собранные данные, и отправляет POST `/api/v1/internal/predict` (заголовок X-Internal-Key) → billing
6. Billing выполняет вывод LLM (подробнее — поток 2), возвращает текст инсайта и количество списанных кредитов
7. Analytics сохраняет отчёт в таблице analytics_reports и возвращает результат клиенту

Общая сквозная задержка на Render с "горячими" сервисами составляет примерно 15-30 секунд, в основном на этапе вывода Ollama. С облачным

провайдером LLM (Anthropic или OpenAI) этот процесс сократился бы примерно до 5-10 секунд.

Поток 2: Внутренний predict биллингового сервиса

Этот поток детализирует шаг 5–6 потока 1 и показывает, как billing маршрутизирует запросы между провайдерами и атомарно списывает кредиты.

1. Analytics → POST /api/v1/internal/predict (заголовок X-Internal-Key) → billing
2. Billing проверяет X-Internal-Key (не JWT — этот эндпоинт предназначен только для доверенных внутренних клиентов)
3. Billing находит запрошенную модель в таблице ml_models и определяет тип (LLM или sklearn) и провайдера
4. Маршрутизация: Ollama — HTTP-вызов к локальному экземпляру Ollama (llama3.2); Anthropic — вызов Anthropic API; OpenAI — вызов OpenAI API; sklearn — загрузка .pkl-файла через joblib, вызов predict() со структурированными признаками
5. Billing атомарно записывает строку в таблицу predictions и создаёт транзакцию в transactions (тип debit, обновляет balance_after)
6. Возвращает {insight, creditsCharged, predictionId} вызывающему сервису (analytics)

Клиент никогда не обращается к billing напрямую — только через analytics с помощью X-Internal-Key.

Поток 3: Запись времени таймером (межсервисная)

Когда рабочий интервал Pomodoro завершён и сеанс был привязан к объекту, таймер автоматически обновляет накопленное время в selfmanager.

1. Клиент завершает рабочий интервал: POST /api/v1/pomodoro/sessions/{id}/complete (Bearer JWT) → timer

2. Timer обновляет статус интервала (`completed = true`) и накапливает `totalWorkMinutes` в строке сеанса
3. Timer проверяет, есть ли у сеанса `entityServerId` (идентификатор привязанного объекта на сервере)
4. Если да: `timer` → `POST /internal/time-log` (заголовок `X-Internal-Key`) → `selfmanager` — передаёт `entityType`, `entityServerId` и количество рабочих минут
5. `Selfmanager` прибавляет минуты к колонке `time_spent_minutes` связанного объекта (задача, привычка или рутина)
6. Timer отвечает клиенту немедленно, не ожидая ответа от `selfmanager` (паттерн `fire-and-forget`)

Шаблон "включи и забудь" означает, что ответ таймера клиенту не блокируется вызовом журнала времени к `selfmanager`. Если `selfmanager` временно недоступен, обновление журнала времени теряется - принятый компромисс для текущего этапа: время фокусировки может быть незначительно занижено, но основные функции таймера не нарушаются.

Поток 4: Аутентификация пользователя

Этот поток показывает идентификацию пользователя, межсервисный вызов при первой регистрации и механизм автоматического обновления токена.

Первичная идентификация:

1. Клиент → `POST /api/v1/auth/identify {email}` → `selfmanager`
2. `Selfmanager` ищет пользователя по `email` в таблице `users`
3. Если пользователь новый: `selfmanager` создаёт запись, затем → `POST /internal/users` (заголовок `X-Internal-Key`) → `billing` — инициирует приветственное пополнение кредитов

4. Billing создаёт запись пользователя в своей БД и записывает транзакцию начального баланса
5. Selfmanager генерирует пару токенов: accessToken (24 ч) и refreshToken (7 дней)
6. Возвращает {accessToken, refreshToken, userId} клиенту
7. Клиент сохраняет токены в таблице sync_metadata (SQLDelight) и инициализирует AppSession в памяти

Автоматическое обновление токена:

1. Клиент получает 401 Unauthorized на любой запрос
2. HttpSend-интерцептор в ServicesApiClient перехватывает ответ
3. Интерцептор → POST /api/v1/auth/refresh {refreshToken} → selfmanager (через отдельный HTTP-клиент без интерцептора, чтобы избежать рекурсии)
4. Selfmanager валидирует refresh-токен и выпускает новый accessToken
5. UserSessionManager.updateTokens() обновляет токены в AppSession и сохраняет их в SQLDelight
6. Интерцептор повторяет исходный запрос с новым токеном; refreshMutex предотвращает гонку при параллельных 401.

Поток 5: Прямые AI-функции (NLP-разбор и контекстный анализ)

В отличие от потока 1, эти функции не агрегируют данные из нескольких сервисов и не сохраняют аналитический отчёт — они возвращают результат клиенту немедленно. Все они проходят через billing (поток 2) для вывода LLM и списания кредитов.

Подтип А — чистый NLP-разбор (parse-task, parse-habit, parse-routine)

Пользователь вводит произвольный текст, LLM возвращает структурированные поля объекта. Данные из других сервисов не собираются.

1. При открытии панели «Заполнить с помощью ИИ» клиент → GET /api/v1/analytics/model-status → analytics → проверяет доступность модели в billing; индикатор на экране меняется на зелёный / жёлтый / красный
2. Пользователь вводит текст и нажимает «Сгенерировать»
3. Клиент → POST /api/v1/analytics/parse-task (или parse-habit / parse-routine) с пользовательским текстом (Bearer JWT) → analytics
4. Analytics формирует парсинговый промпт и → POST /api/v1/internal/predict (заголовок X-Internal-Key) → billing
5. Billing маршрутизирует к LLM, получает структурированный JSON, списывает кредиты (поток 2)
6. Analytics возвращает разобранные поля клиенту: название, описание, приоритет, срочность, дата, время, проект, теги, подзадачи, напоминание
7. Клиент отображает карточку предпросмотра; «Применить к форме» заполняет все поля формы, «Попробовать снова» сбрасывает предпросмотр

Задержка минимальна среди всех AI-функций: промпт содержит только введенный пользователем текст.

Подтип В — контекстный анализ (prioritize-tasks, habit-coaching, routine-optimization)

LLM работает с реальными данными конкретного объекта, получаемыми из selfmanager. Один дополнительный внутренний вызов по сравнению с подтипом А.

1. Клиент → POST /api/v1/analytics/prioritize-tasks (или habit-coaching / routine-optimization) (Bearer JWT) → analytics

2. Analytics извлекает необходимые данные из selfmanager: GET /internal/users/{userId} (заголовок X-Internal-Key) — список задач для приоритизации, данные привычки для коучинга или шаги рутины для оптимизации
3. Analytics формирует промпт, включая полученные данные объекта, и → POST /api/v1/internal/predict (заголовок X-Internal-Key) → billing
4. Billing маршрутизирует к LLM, получает результат, списывает кредиты (поток 2)
5. Analytics возвращает результат клиенту: prioritize-tasks — список задач с предложенными приоритетами, клиент (SelfManagerViewModel) применяет их немедленно; habit-coaching — инсайт, советы и рекомендуемое действие, отображается в HabitDetailsScreen; routine-optimization — пронумерованные предложения, наблюдение по таймингу и главное изменение, отображается в AiOptimizationCard внутри RoutineDetailsScreen

Результат не сохраняется в analytics_reports — в отличие от полного анализа (поток 1), эти функции работают в режиме запрос-ответ без персистентной истории.

3.14 Сообщения безопасности

В этом разделе описываются свойства безопасности текущей реализации и улучшения безопасности, которые будут необходимы перед выпуском для всеобщего использования.

Текущая ситуация в области безопасности

Границы аутентификации устанавливаются на уровне HTTP. Для каждой внутренней конечной точки требуется заголовок X-Internal-Key. Для каждой пользовательской конечной точки (кроме /api/v1/auth/identify и /api/v1/models) требуется действительный JWT. Не прошедшие проверку подлинности запросы получают 401 ответ без какой-либо информации о том, что делает конечная точка.

Секреты хранятся в переменных окружения, а не в коде. Секрет JWT, внутренний ключ API, пароли базы данных и ключи API LLM-провайдера загружаются из переменных среды при запуске. Они никогда не передаются в git, никогда не возвращаются в ответах API и никогда не регистрируются.

Служба выставления счетов не имеет общедоступных конечных точек. Даже если URL-адрес биллинговой службы был каким-либо образом раскрыт (например, через сетевую вкладку браузера), ни одна из ее конечных точек не отвечает на запросы без заголовка X-Internal-Key. Это исключает возможность прямого манипулирования кредитами со стороны пользователей.

Срок действия JWT принудительно истек. Токены доступа истекают через 24 часа. Украденный токен имеет ограниченное окно неправомерного использования.

Механизм обновления токенов защищает от использования просроченных токенов. В ServicesApiClient реализован HttpSend-перехватчик с refreshMutex: при получении 401 клиент автоматически обновляет токен через отдельный refreshClient (чтобы избежать рекурсии перехватчика), после чего повторяет исходный запрос. Мьютекс предотвращает гонку при параллельных запросах.

Идемпотентность синхронизации защищает от дублирования и replay-атак. Синхронизация построена на syncId (UUID, генерируется на клиенте) — сервер игнорирует повторные отправки того же syncId. Это не только архитектурное решение, но и защита от атак повторного воспроизведения (replay) при нестабильном соединении.

Защита от SQL-инъекций обеспечивается на уровне фреймворков. Все запросы к БД используют параметризованные запросы: Exposed (Kotlin-сервисы), SQLAlchemy (billing, FastAPI), SQLDelight (KMP-клиент). Явная обработка входных данных не требуется — инъекции исключены на уровне ORM/query builder.

Ограничения безопасности при проверке концепции

Аутентификация только по электронной почте не является стандартной. Текущий механизм аутентификации принимает любой адрес электронной почты без проверки. В рабочей среде для предотвращения олицетворения учетной записи потребуется проверка электронной почты (или поставщика OAuth2, такого как Google).

Общий секрет JWT - это единая точка доверия. Если секрет JWT скомпрометирован, все службы будут скомпрометированы одновременно. В производстве смена секрета требует одновременного развертывания всех трех пользовательских сервисов. Более надежный подход заключался бы в использовании асимметричного JWT (RS256) с парой открытых / закрытых ключей, позволяющей хранить секретный ключ подписи только службе выдачи.

Нет ограничений по скорости аутентификации. Конечная точка /api/v1/auth/identify не имеет ограничений по скорости, что делает ее уязвимой для атак с перечислением (определение того, на какие адреса электронной почты есть учетные записи). При производстве потребуется ограничение скорости и CAPTCHA.

В локальной разработке нет принудительного использования HTTPS. Локальный Docker Compose работает по HTTP. В рабочей среде на Render все

сервисы обслуживаются по протоколу HTTPS обратным прокси-сервером Render. Однако сами службы не поддерживают протокол HTTPS, поэтому при развертывании за прокси-сервером, отличным от HTTPS, данные будут передаваться открытым текстом.

Ngrok для Ollama небезопасен для многопользовательского развертывания. URL-адрес ngrok является общим в переменной среды службы выставления счетов. Любая служба с правильным X-Internal-Key может выполнять вызовы вывода через этот туннель, и весь трафик вывода проходит через инфраструктуру ngrok. Для производственного развертывания выделенная облачная модель, обслуживающая конечную точку (с надлежащей аутентификацией и ограничением скорости), заменила бы схему ngrok.

X-Internal-Key — единая точка отказа. Уже упомянут JWT_SECRET как единая точка отказа, но X-Internal-Key имеет ту же проблему: один ключ используется для аутентификации всех межсервисных запросов (analytics → selfmanager, timer, billing; dashboard → billing). Компрометация ключа даёт доступ ко всем внутренним эндпоинтам.

Отсутствие audit log для административных операций. Дашборд позволяет пополнять баланс пользователей, удалять модели, просматривать все предсказания — без журналирования действий администратора. В production-среде это риск.

Изменения относительно утверждённого задания на ВКР

В процессе реализации ряд технических решений, зафиксированных в исходном задании, был пересмотрен на основании углублённого анализа предметной области, требований к надёжности системы и актуального состояния технологического стека. Все отклонения носят обоснованный характер: в каждом случае принятое решение обеспечивает ту же или более высокую архитектурную ценность при меньшей операционной сложности. Ниже приведено последовательное сопоставление пунктов задания с фактически реализованными артефактами.

1. Состав микросервисов: переосмысление доменных границ

Задание предусматривало четыре сервиса по доменам: Task Manager Service, Note Service, Pomodoro Service и Journal Service. В фактической реализации система также состоит из четырёх сервисов, однако с иной доменной разбивкой: selfmanager (задачи, привычки, рутины), timer (управление Pomodoro-сессиями), analytics (конвейер ИИ-анализа) и billing (кредитный учёт и LLM-инференс).

Анализ предметной области показал, что задачи, привычки и рутины образуют единый домен самостоятельного управления: они разделяют общую модель пользователя, единые теги и проекты, общий экран ежедневного планировщика и единый протокол офлайн-синхронизации. Вынесение каждого из этих типов сущностей в отдельный сервис создало бы избыточные межсервисные вызовы при каждом рендере главного экрана и усложнило бы транзакционную синхронизацию без какой-либо функциональной выгоды.

Модуль заметок переведён в статус entity-linked функции: заметки в архитектуре ProductiveSocial имеют ценность только в контексте привязки к конкретной задаче, привычке или рутине. Реализация их как самостоятельного изолированного сервиса до завершения основных модулей расставила бы приоритеты в обратном порядке относительно ценностного предложения платформы.

Journal Service спроектирован архитектурно — как отдельный сервис `psocial_journal` на стеке Kotlin/Ktor с собственной базой данных PostgreSQL и разделяемым JWT-секретом — и включён в дорожную карту MVP. Стратегическое решение завершить конвейер ИИ-аналитики первым объясняется тем, что журнал без AI-синтеза теряет основную ценность в контексте платформы: именно синтез субъективных записей пользователя с объективными данными о выполнении задач, привычек и Pomodoro-сессий отличает журнал ProductiveSocial от стандартного приложения для ведения дневника. Интерфейсный каркас журнала реализован в мобильном приложении;

полная реализация (сервис, слой данных и пакет синхронизации) является следующим этапом разработки.

Выделение billing в отдельный сервис вместо встраивания его в analytics объясняется принципом единственной ответственности: сервис, выполняющий финансовые транзакции и инкапсулирующий коммерческие API-ключи LLM-провайдеров, не должен зависеть от изменений в логике построения промптов. Это решение также позволило изолировать Python-экосистему (scikit-learn, anthropic SDK, openai SDK) от JVM-стека остальных сервисов.

2. Протокол межсервисного взаимодействия: gRPC заменён на REST/HTTP

Задание предусматривало межсервисное взаимодействие через высокопроизводительный протокол gRPC для внутренних вызовов. В фактической реализации все межсервисные вызовы осуществляются через REST/HTTP с аутентификацией посредством заголовка X-Internal-Key.

Применение gRPC потребовало бы: поддержки protobuf-схем с кодогенерацией для каждой пары взаимодействующих сервисов; гарантированной поддержки HTTP/2 на всех промежуточных прокси и балансировщиках нагрузки; совместимого gRPC-клиента в Python-сервисе billing. Платформа развёртывания Render на бесплатном и базовом тарифах не гарантирует поддержку HTTP/2 на уровне edge-прокси. Административные дашборды на Streamlit взаимодействуют с сервисами через стандартные HTTP-запросы посредством библиотеки requests, что делает gRPC-интерфейс недоступным для этого клиента без дополнительного транскодирующего слоя.

REST/HTTP обеспечивает те же гарантии надёжности в данном контексте: все внутренние вызовы синхронны, защищены разделяемым секретом, корректно обрабатывают таймауты и возвращают структурированные сообщения об ошибках. Преимущество gRPC по производительности реализуется при высокочастотных вызовах порядка тысяч запросов в секунду; в текущей архитектуре узким местом является время инференса LLM (15–30

секунд), а не латентность сетевого протокола. Переход на gRPC является обоснованным шагом при масштабировании системы до нагрузок, при которых накладные расходы HTTP-оверхеда станут измеримы на фоне общей латентности запроса.

3. Брокер сообщений: RabbitMQ заменён на синхронный HTTP с graceful degradation

Задание предусматривало использование брокера сообщений RabbitMQ для асинхронной передачи логов активности в ML-модуль. В фактической реализации применяются два паттерна асинхронного взаимодействия без брокера.

Первый — pull-модель в сервисе аналитики: при получении запроса на анализ сервис analytics самостоятельно запрашивает актуальные данные из selfmanager и timer через внутренние HTTP-эндпоинты. Если один из вышестоящих сервисов недоступен, анализ продолжается с частичными данными (graceful degradation), что подтверждено экспериментом №9. Второй — паттерн fire-and-forget из timer в selfmanager: после завершения Pomodoro-интервала сервис timer асинхронно отправляет запрос на обновление накопленного времени в selfmanager, не блокируя ответ клиенту.

RabbitMQ добавляет существенную операционную сложность: отдельный сервис брокера, мониторинг очередей, политики повторных попыток, dead-letter queues и управление схемами сообщений. При этом исходное требование к RabbitMQ — гарантированная доставка логов активности в ML-модуль — не соответствует фактической архитектуре, в которой ML-инференс выполняется по явному запросу пользователя, а не на основе непрерывного потока событий. Pull-модель архитектурно превосходит push через брокер для данного случая использования: данные всегда актуальны на момент анализа; накопление сообщений не требует управления; graceful degradation реализуется через обработку HTTP-таймаутов без дополнительной инфраструктуры.

Известное ограничение паттерна fire-and-forget — потенциальная потеря записи времени при временной недоступности selfmanager — зафиксировано в разделе 3.12 как принятый компромисс для текущего этапа. Производственным решением для данного случая является retry с exponential backoff или замена на паттерн transactional outbox.

4. Интеграция машинного обучения: обучение на открытых датасетах заменено на LLM-инференс по паттерну RAG

Задание предусматривало поиск и адаптацию открытых датасетов с платформ Kaggle и GitHub по паттернам продуктивности и тайм-менеджменту, обучение отдельных ML-модулей для каждого микросервиса, а также создание выделенного бэкенд-сервиса с обученными моделями и API генерации общего анализа продуктивности.

В фактической реализации конвейер ИИ-аналитики построен по паттерну RAG (Retrieval-Augmented Generation): при запросе анализа сервис analytics в реальном времени собирает пользовательские данные из всех сервисов, формирует структурированный контекстный промпт и передаёт его на LLM-инференс через billing-сервис. Выделенный billing-сервис инкапсулирует маршрутизацию между тремя LLM-провайдерами (Ollama, Anthropic Claude, OpenAI GPT) и содержит готовый путь вывода sklearn-моделей: реестр моделей, загрузку файлов .pkl через библиотеку joblib и эндпоинт загрузки обученных моделей.

Подход с обучением на открытых датасетах предполагает наличие размеченных данных о продуктивности с верифицированными метками достоверности. Публичного датасета, пригодного для задач данного проекта, не существует: наиболее близкие наборы данных — PMData и LifeSnaps — охватывают физиологические показатели с носимых устройств, а не структурированные паттерны выполнения задач, привычек и Pomodoro-сессий в формате, совместимом с моделью данных ProductiveSocial. Адаптация

сторонних датасетов потребовала бы значительной инженерии признаков без гарантии релевантности для целевых задач предсказания.

LLM-инференс с пользовательским контекстом является методологически обоснованным выбором для текущей стадии по трём причинам. Во-первых, метод применим с первого пользователя и не требует предварительного накопления данных для обучения. Во-вторых, качество результатов верифицировано экспериментально: средняя оценка LLM-анализа составила 4,1 из 5 против 1,3 из 5 у детерминированной базовой линии по критериям релевантности, специфичности и практической применимости (раздел 2.7). В-третьих, LLM обладает встроенной способностью к кросс-доменному рассуждению: он связывает паттерны задач, привычек и Pomodoro-сессий в едином запросе, что недостижимо для узкоспециализированных sklearn-моделей, обученных на данных одного домена.

Путь к проприетарным ML-моделям явно сохранён в архитектуре: по мере накопления реальных поведенческих данных обученные модели загружаются в реестр billing-сервиса без каких-либо структурных изменений остальных сервисов. Принятое отклонение является тактическим решением — выбором наиболее подходящего метода для текущей стадии, — а не отказом от ML-интеграции как цели.

5. Мобильное приложение: статус реализации

Задание предусматривало разработку пользовательского интерфейса для каждого модуля, интеграцию с ML-бэкендом, реализацию локальной базы данных, механизма синхронизации состояний и кэширования ML-прогнозов.

Все базовые требования выполнены. Локальная база данных реализована на SQLDelight и включает 23 таблицы, охватывающие все типы сущностей платформы (раздел 3.6.2). Двухнаправленная офлайн-синхронизация реализована через идемпотентный пакетный протокол с гарантией отсутствия дублирования записей (раздел 3.6.13, эксперименты №4 и №5). Кэширование ML-прогнозов

обеспечивается хранением аналитических отчётов в таблице `analytics_reports`: отчёты доступны пользователю в офлайн-режиме без повторного обращения к серверу. Интеграция с ML-бэкендом реализована через `AnalyticsApiClient`, обслуживающий все точки взаимодействия с LLM из мобильного клиента.

Экраны всех основных модулей разработаны и подключены к синхронизации: модуль задач с NLP-разбором и AI-приоритизацией, модуль привычек со счётчиком серий и AI-коучингом, модуль рутин с режимом выполнения и AI-оптимизацией, таймер Pomodoro с многоцикловым режимом и рекомендациями длительности, экран аналитики с шестью типами анализа и историей отчётов.

Модуль дневника и сквозная функция целей реализованы как интерфейсные каркасы: экраны присутствуют в навигационном графе приложения, однако слой данных, ViewModel, API-интеграция и пакет синхронизации являются следующим этапом разработки, запланированным к завершению до публичного релиза MVP.

Таблица 3.9 — Соответствие пунктов задания фактической реализации

Пункт задания	Фактический артефакт	Статус
Task Manager Service	selfmanager (задачи, привычки, рутины)	Реализован, расширен
Note Service	entity-linked модуль (дорожная карта)	Перенесён
Pomodoro Service	timer-сервис	Реализован
Journal Service	архитектурно спроектирован (psocial_journal), UI-каркас в KMP	Частично реализован
Offline-First синхронизация	идемпотентный batch-протокол, SQLDelight, двунаправленная sync	Реализована
gRPC (внутренние вызовы)	REST/HTTP + X-Internal-Key	Заменён (обосновано в разделе выше)
RabbitMQ (асинхронные логи)	pull-модель + fire-and-forget HTTP	Заменён (обосновано в разделе выше)

Открытые датасеты (Kaggle, GitHub)	не использованы	Заменён (обосновано в разделе выше)
Обучение ML-моделей	LLM-инференс (RAG); sklearn-путь подготовлен в billing	Заменён (обосновано в разделе выше)
Выделенный ML-бэкенд	billing-сервис (LLM-маршрутизация + sklearn-путь)	Реализован
UI всех модулей	задачи, привычки, рутины, таймер, аналитика — полностью; дневник, цели — каркас	Выполнено частично
Локальная БД	SQLDelight, 23 таблицы	Реализована
Кэширование ML-прогнозов	analytics_reports, доступны офлайн	Реализовано

ЗАКЛЮЧЕНИЕ

В этом отчете задокументирован полный жизненный цикл первого proof-of-concept от ProductiveSocial: от определения предметной области и структурированного анализа конкурентной среды, архитектурного проектирования и технической реализации четырех независимых микросервисов до комплексной проверки основного ценностного предложения системы.

Краткое изложение материалов

Определение предметной области и проблемы. В отчете был выявлен и задокументирован структурный пробел на рынке инструментов для повышения личной продуктивности: ни одно существующее приложение не предоставляет все четыре основных модуля повышения продуктивности (управление задачами, отслеживание привычек, хронометраж Pomodoro и анализ с помощью искусственного интеллекта) на единой платформе, и ни одно существующее приложение не предоставляет уровень искусственного интеллекта с одновременным доступом к междоменным поведенческим данным. Этот разрыв был подтвержден сравнительным анализом семи ведущих продуктов — Todoist, Habitica, Forest, Notion, Day One, RescueTime и Sunsama — каждый из которых был проанализирован по шести параметрам возможностей.

Архитектурный проект. Архитектура ProductiveSocial была разработана на основе пяти четких принципов — независимости сервиса, единого источника достоверности для каждого домена, границ аутентифицированного доступа, владения данными в автономном режиме и системы кредитов как поведенческого стимула. Эти принципы привели к конкретным, оправданным проектным решениям: JDBC-соединение UserRegistry с кросс-базой данных для канонической идентификации пользователя; модель журнала транзакций в качестве баланса биллинговой службы для доказуемо согласованного учета кредитоспособности; неблокирующий сбор данных восходящего потока службой аналитики для постепенного ухудшения; и общий секрет JWT для межведомственной аутентификации без централизованного поставщика идентификационных данных.

Техническая реализация. Реализация охватывает четыре серверных сервиса (Kotlin/Ktor для selfmanager, timer и analytics; Python/FastAPI для выставления счетов), административную веб-панель мониторинга на Streamlit и мобильный клиент на Kotlin Multiplatform. Мобильный клиент на Kotlin Multiplatform реализует полный набор основных пользовательских сценариев платформы: управление задачами, привычками, рутинами и проектами с локальной базой данных SQLDelight; таймер Pomodoro с многоцикловым режимом, привязкой к сущностям и статистикой; экран AI-аналитики с шестью типами анализа и историей отчётов; двунаправленная офлайн-синхронизация с идемпотентным пакетным протоколом. Модуль дневника и функция целей реализованы как каркасы и находятся в разработке. Выбор технологий — Kotlin, Ktor, FastAPI, PostgreSQL, Docker, Colima, Streamlit, KMP, SQLDelight — был сделан по конкретным техническим причинам, описанным в разделе 2.2, с общей целью использовать лучший инструмент для каждой задачи, а не навязывать один язык или фреймворк всей системе.

Интеграция ML. Конвейер анализа ИИ реализует шаблон генерации с дополнением к поиску (RAG), в котором данные о пользователях собираются во время вывода, а не из векторной базы данных. Были разработаны и протестированы шесть типов анализа с различными подсказками. Валидация по базовому уровню, основанному на правилах, продемонстрировала существенное улучшение качества: баллы LLM составили 3.9–4.2/5 по сравнению с базовыми баллами 1.0–1.5/5 по трем типам анализа. Анализ выявил междоменные поведенческие корреляции и дал конкретные рекомендации, которые можно реализовать — возможности, которые определенно выходят за рамки любого подхода, основанного на правилах.

Проверка. Двенадцать документированных экспериментов подтвердили функциональную корректность (межсервисная аутентификация JWT, арифметика зачета, идемпотентность синхронизации, постепенное ухудшение), качество LLM (три сравнения типов анализа с примерами входных и выходных данных) и операционное поведение (задержка холодного запуска, задержка прогрева, исключение зачета при сбое прогнозирования).

Текущие ограничения

Проверка концепции демонстрирует основное ценностное предложение в полностью функциональной системе, но имеет ряд ограничений, которые отличают ее от готовой к производству платформы:

Мобильное приложение реализовано в объеме основных пользовательских сценариев: семь экранов (проекты, задачи, привычки, рутины, таймер Pomodoro, аналитика и профиль) полностью функциональны и подключены к двунаправленной офлайн-синхронизации. Незавершёнными остаются только модуль дневника и сквозная функция целей — оба реализованы как интерфейсные каркасы и запланированы к завершению до публичного релиза MVP. Таким образом, мобильный клиент демонстрирует полный пользовательский путь платформы: создание задач, привычек и рутин, управление Pomodoro-сессиями с привязкой к сущностям, получение AI-аналитики и просмотр кредитного баланса — всё в одном приложении с офлайн-first архитектурой.

Хостинг для юридических лиц. В производственном развертывании используется личный экземпляр, который предоставляется через ngrok. Это не масштабируемо (компьютер одного разработчика), ненадежно (компьютер должен быть запущен и подключен) и не подходит для многопользовательской системы. Для обеспечения готовности к производству требуется переключение на облачного провайдера LLM (Anthropic Claude или OpenAI GPT-4o) или на развертывание Ollama в облаке.

Уровень развертывания. Бесплатный уровень Render предусматривает 50-секундный "холодный запуск" служб JVM после 15 минут простоя. Платный уровень развертывания с постоянными контейнерами устранил бы эту проблему взаимодействия с пользователем.

Модель аутентификации. Аутентификация "найти или создать" только по электронной почте подходит для проверки концепции, но не для системы с реальными пользовательскими данными. Потребуется проверка электронной

почты, интеграция с поставщиком OAuth2 или аутентификация на основе пароля.

Дорожная карта: следующий этап разработки

На следующем этапе разработки устраняются вышеуказанные ограничения в приоритетном порядке:

Этап 1 (немедленный):

1. Достройте оставшиеся модули мобильного приложения KMP — журнал и цели (таблицы SQLDelight, ViewModel, конечные точки API и пакет синхронизации), а также расширьте функции ИИ структурированными ответами, применяемыми обратно к данным (например, применение приоритетов задач)
2. Настройте Anthropic Claude в качестве производственного LLM-провайдера в службе выставления счетов
3. Обновите развёртывание на Render, чтобы исключить "холодные запуски"
4. Реализовать проверку электронной почты в потоке аутентификации

Этап 2 (среднесрочный):

5. Реализовать социальный модуль в виде нового микросервиса (общедоступные профили, обмен достижениями, лента сообщества)
6. Дополните модуль журнала обобщением дня с помощью искусственного интеллекта
7. Добавьте модуль notes с перекрестными ссылками на объекты

Этап 3 (долгосрочный):

8. Накапливайте реальные данные о поведении пользователей и приступайте к обучению проприетарных прогностических моделей
9. Внедряйте обученные модели sklearn с помощью существующего пути вывода биллинговой службы

10. Построение модели устройства для полностью автономных функций искусственного интеллекта

11. Сторонние интеграции (Календарь Google, носимые устройства)

Архитектурная готовность к росту

Архитектура специально разработана для размещения дополнений этапа 2 и этапа 3, не требуя структурных изменений в существующих сервисах. Новые сервисы (социальные сети, журналы, заметки) могут быть добавлены как независимые микросервисы, которые используют одну и ту же модель идентификации пользователя (selfmanager целочисленные идентификаторы пользователей) и один и тот же механизм аутентификации (общий секрет JWT или X-Internal-Key). Реестр моделей биллинговой службы и путь вывода sklearn уже созданы для получения обученных моделей, когда они будут готовы. Система подсказок службы аналитики может быть расширена новыми типами анализа путем добавления нового обращения к функции buildPrompt() и нового постоянного типа анализа — никаких других изменений службы не требуется.

ProductiveSocial — это, по своей сути, платформа, созданная как для того, чем она станет, так и для того, чем она уже является. Решения, принятые на этом первом этапе — общая модель идентификации пользователей, внутренние границы API, кредитная книга на основе транзакций, протокол синхронизации в автономном режиме — не являются вариантами реализации, которые необходимо будет пересматривать по мере роста платформы. Они являются архитектурным фундаментом, на котором может быть построено полное видение интеллектуальной операционной системы для повышения производительности.

СПИСОК ЛИТЕРАТУРЫ

1. Allen, D. (2001). Доводить дело до конца: искусство продуктивности без стресса. Viking.
2. Чирилло, Ф. (2006). Техника Помодоро. ФК "Гараж".
3. Ньюпорт, С. (2016). Глубокая работа: правила целенаправленного успеха в отвлеченном мире. Издательство Grand Central Publishing.
4. Клир, Дж. (2018). Атомарные привычки: простой и проверенный способ выработать хорошие привычки и избавиться от плохих. Эйвери.
5. Лерой, С. (2009). Почему мне так трудно выполнять свою работу? Проблема остатков внимания при переключении между рабочими задачами. Организационное поведение и процессы принятия решений человеком, 109 (2), 168-181.
6. Ньюман, С. (2021). Создание микросервисов: проектирование мелкозернистых систем (2-е изд.). О'Рейли Медиа.
7. Ричардсон, С. (2018). Шаблоны микросервисов: с примерами на Java. Публикации Мэннинга.
8. Льюис Дж., & Фаулер М. (2014). Микросервисы: определение этого нового архитектурного термина. [martinfowler.com. https://martinfowler.com/articles/microservices.html](https://martinfowler.com/articles/microservices.html)
9. Браун Т., Манн Б., Райдер Н. и др. (2020). Языковые модели Изучают Немногие. Достижения в области нейронных систем обработки информации, 33, 1877-1901.
10. Васвани А., Шазир, Н., Пармар Н. и др. (2017). Внимание - это все, что вам нужно. Достижения в области нейронных систем обработки информации, 30.

11. Команда разработчиков задач. (2024). Todoist: Список дел для организации работы и жизни. <https://todoist.com>
12. Команда Habitica. (2024). Habitica: Геймифицируйте свою жизнь. <https://habitica.com>
13. Notion Labs, Inc. (2024). Notion: подключенное рабочее пространство для ваших заметок, документов и проектов. <https://notion.so>
14. Команда первого дня. (2024). День первый: приложение для ведения журнала. <https://dayoneapp.com>
15. Команда времени спасения. (2024). RescueTime: программное обеспечение для управления временем. <https://rescuetime.com>
16. Touchlight Ltd. (2024). Сансама: Ежедневник. <https://sunsama.com>
17. Фонд Котлина. (2024). Мультиплатформенный Kotlin. <https://kotlinlang.org/docs/multiplatform.html>
18. Реактивные мозги. (2024). Ktor: Фреймворк для создания асинхронных серверных приложений. <https://ktor.io>
19. FastAPI. (2024). FastAPI: современный, быстрый веб-фреймворк для создания API на Python. <https://fastapi.tiangolo.com>
20. Оллама. (2024). Оллама: Освоение больших языковых моделей. <https://ollama.ai>
21. Антропный. (2024). Клод: Ассистент по искусственному интеллекту от Anthropic. <https://anthropic.com>
22. SQLDelight. (2024). SQLDelight: Типизированный SQL для мультиплатформенной Kotlin. <https://cashapp.github.io/sqldelight/>
23. Render. (2024). Render: самый быстрый способ размещения всех ваших веб-приложений. <https://render.com>
24. Neon. (2024). Neon: Бессерверная Postgres. <https://neon.tech>

25. Фаулер, М. (2002). Шаблоны архитектуры корпоративных приложений. Эддисон-Уэсли.
26. Деси, Э. Л. и Райан, Р. М. (1985). Внутренняя мотивация и самоопределение в поведении человека. Спрингер.
27. Свеллер, Дж. (1988). Когнитивная нагрузка при решении проблем: влияние на обучение. Когнитивная наука, 12(2), 257-285.
28. Чирилло, Ф. (2018). Техника Помодоро: признанная система тайм-менеджмента, которая изменила то, как мы работаем. Валюта.
29. Реактивные мозги. (2024). Обзор мультиплатформенной платформы Kotlin. <https://kotlinlang.org/docs/multiplatform.html>
30. Реактивные мозги. (2024). Документация Ktor: асинхронный фреймворк Kotlin. <https://ktor.io/docs/>
31. FastAPI. (2024). Документация по FastAPI. <https://fastapi.tiangolo.com/>
32. Docker Inc. (2024). Документация по Docker: начало работы. <https://docs.docker.com/>
33. Колима. (2024). Colima: среды выполнения контейнеров в macOS (и Linux) с минимальной настройкой. <https://github.com/abiosoft/colima>
34. SQLDelight. (2024). SQLDelight: Типизированный SQL для мультиплатформенной Kotlin. <https://cashapp.github.io/sqldelight/>
35. Тианголо, С. (2024). Документация Pydantic. <https://docs.pydantic.dev/>
36. Джонс, М., Брэдли, Дж., & Сакимура, Н. (2015). RFC 7519: веб-токен JSON (JWT). IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
37. Атласский. (2024). Поточная документация. <https://docs.streamlit.io/>
38. Meta AI. (2024). Llama 3.2: Открытая основа и доработанные модели чатов. <https://ai.meta.com/>

Приложение А

Глоссарий технических терминов

API (интерфейс прикладного программирования): определенный набор правил и конечных точек, с помощью которых одна программная система взаимодействует с другой. В контексте этого проекта каждая служба предоставляет HTTP API, который клиенты и другие службы вызывают для создания, чтения, обновления или удаления данных.

Асинхронное программирование: модель программирования, в которой инициируются операции, которые в противном случае были бы заблокированы (ожидание ответа сети, результата базы данных, таймера), а затем программа продолжает выполнять другую работу; когда операция завершается, обратный вызов или продолжение возобновляют обработку. Сопрограммы Kotlin реализуют эту модель.

Кодировка Base64url: вариант кодировки Base64, который безопасен для использования в URL-адресах и заголовках HTTP (замена + на - и / на _). Используется для кодирования разделов заголовка и полезной нагрузки JWT.

CORS (совместное использование ресурсов разных источников): Механизм HTTP, который контролирует, каким веб-источникам разрешено отправлять запросы к серверу. Все службы Ktor настроены так, чтобы разрешать запросы от источников панели мониторинга.

CRUD: Создание, чтение, обновление, Удаление — четыре основные операции с постоянными данными, соответствующие HTTP POST, GET, PUT/PATCH и DELETE соответственно.

DTO (Объект передачи данных): простой класс данных, используемый для передачи данных между процессами. В службах Kotlin DTO определяются как классы данных, которые сопоставляются с телами запросов / ответов JSON.

HMAC (код аутентификации сообщения на основе хэша): криптографическая конструкция, которая объединяет хэш-функцию (SHA-256 в

случае HS256 JWTs) с секретным ключом для создания кода аутентификации сообщения. Используется в качестве алгоритма подписи JWT.

HTTP-заголовок: Пара ключ-значение, включенная в HTTP-запрос или ответ для передачи метаданных. Заголовок авторизации содержит токен-носитель JWT; заголовок X-Internal-Key содержит ключ аутентификации между сервисами; Content-Type определяет формат тела запроса.

Идемпотентность: свойство операции, при котором многократное ее повторение приводит к тому же результату, что и при ее однократном выполнении. Конечная точка синхронизации является идемпотентной: при повторной отправке одного и того же запроса синхронизации одни и те же записи создаются один раз, а не дважды. Это важно для безопасности повторных попыток.

JDBC (Java Database Connectivity): Java API для подключения к реляционным базам данных. Службы Kotlin используют JDBC (с HikariCP для объединения в пул) для подключения к PostgreSQL. UserRegistry использует исходное соединение JDBC, а не ORM, поскольку оно выполняет единственную конкретную инструкцию SQL, которая выигрывает от явного контроля.

JVM (виртуальная машина Java): Среда выполнения для программ на Java и Kotlin. Скомпилированный класс Kotlin выполняется в JVM, который обеспечивает сборку мусора, JIT-компиляцию и независимость от платформы. Время запуска JVM (~ 2-3 секунды) является причиной того, что службы на основе JVM имеют более длительные холодные запуски, чем служба выставления счетов Python.

JWT (веб-токен JSON): компактный, безопасный для URL-адресов формат для представления утверждений в виде объекта JSON, подписанный HMAC или RSA. Полное описание смотрите в разделе 2.2.6.

KMP (мультиплатформенный Kotlin): технология совместного использования кода Kotlin на нескольких платформах (JVM, iOS, JavaScript, native). Полное описание смотрите в разделе 2.2.7.

LLM (Large Language Model): нейросетевая модель, обученная на больших объемах текстовых данных, которая может генерировать связные, контекстуально соответствующие текстовые ответы на запросы. Примеры: Антропный Клод, OpenAI GPT-4o, Meta Llama. Используется в ProductiveSocial через механизм вывода биллинговой службы.

Микросервис: Независимо развертываемый программный сервис с единой, целенаправленной ответственностью. Полное описание смотрите в разделе 2.2.1.

ORM (объектно-реляционный картограф): Библиотека, которая выполняет перевод между объектно-ориентированными представлениями, используемыми в коде приложения, и реляционными представлениями, используемыми в базах данных SQL. Exposed (используется в сервисах Kotlin) и SQLAlchemy (используется в службе выставления счетов) являются ORM в этом проекте.

Разработка подсказок: Практика разработки текстовых вводов (подсказок) для получения определенных типов ответов из языковой модели. Смотрите Раздел 2.5 для получения полного описания запросов, зависящих от типа анализа, используемых в этом проекте.

RAG (поисково-расширенная генерация): шаблон для приложений на основе LLM, в котором релевантные данные извлекаются из источника данных и включаются в запрос LLM во время вывода, предоставляя модели текущий конкретный контекст, который она не могла бы усвоить только из предварительной подготовки. Аналитический конвейер ProductiveSocial является реализацией этого шаблона.

REST (передача репрезентативного состояния): Архитектурный стиль для HTTP API, характеризующийся парами запросов / ответов без сохранения состояния, URL-адресами, ориентированными на ресурсы, и стандартными HTTP-глаголами (GET, POST, PUT, DELETE). Все службы ProductiveSocial предоставляют REST API.

Синглтон: Шаблон проектирования, в котором класс имеет не более одного экземпляра в приложении, общего для всего кода, который зависит от него. Одиночное объявление `Koin { }` создает синглтоны для всех сервисов и HTTP-клиентов.

Надгробный камень: Запись, отмечающая удаление другой записи, используемая в протоколах синхронизации для сообщения о том, что запись была удалена на сервере и должна быть удалена из локальной базы данных клиента.

UUID (универсально уникальный идентификатор): 128-битный идентификатор, сгенерированный таким образом, что коллизии практически невозможны. Используется в качестве идентификаторов, назначаемых клиентом для всех синхронизированных объектов, что позволяет создавать автономные записи без координации с сервером.

Приложение Б

Артефакты воспроизводимости

Данное приложение содержит все сведения, необходимые для независимого воспроизведения результатов, представленных в работе: ссылки на репозитории, инструкции локального запуска, шаблон переменных окружения, описание тестового набора данных, ссылки на API-документацию и пошаговые команды для воспроизведения каждого из 12 задокументированных экспериментов.

1. Репозитории и развёрнутые сервисы

Исходный код каждого сервиса размещён в отдельном репозитории на GitHub в организации ProductiveSocial:

psocial_selfmanager: https://github.com/ProductiveSocial/PSocial_selfmanager

psocial_timer: https://github.com/ProductiveSocial/PSocial_timer

psocial_analytics: https://github.com/ProductiveSocial/PSocial_analytics

psocial_billing: https://github.com/ProductiveSocial/PSocial_billing

psocial_dashboard (admin): https://github.com/ProductiveSocial/psocial_admin_dashboard

PS_kmp (мобильное приложение): https://github.com/ProductiveSocial/PSocial_kmp

Развёрнутые сервисы на платформе Render (URL актуальны на дату защиты):

selfmanager: <https://psocial-selfmanager.onrender.com>

timer: <https://psocial-timer.onrender.com>

analytics: <https://psocial-analytics.onrender.com>

billing: <https://psocial-billing.onrender.com>

dashboard (admin): <https://psocial-dashboard.onrender.com>

2. Запуск локальной среды разработки

Предварительные требования: Docker, Colima (для macOS), Git. Все команды выполняются из корневого каталога проекта.

Шаг 1. Запустить контейнерную среду выполнения:

```
colima start
```

Шаг 2. Скопировать шаблон переменных окружения и заполнить значения:

```
cp .env.example .env
```

Шаг 3. Собрать fat JAR для каждого Kotlin-сервиса (выполнить в каталоге каждого сервиса):

```
./gradlew shadowJar
```

Шаг 4. Запустить все сервисы и базы данных:

```
docker compose up --build
```

После запуска сервисы доступны по следующим адресам:

selfmanager: <http://localhost:1226>

timer: <http://localhost:1227>

analytics: <http://localhost:1228>

billing: <http://localhost:1229>

dashboard (admin): <http://localhost:1230>

Swagger UI доступен по адресу <http://localhost:1229/docs> для billing-сервиса и по `/swagger` для Kotlin-сервисов (например, <http://localhost:1226/swagger>).

3. Шаблон переменных окружения

Файл `.env.example` в корневом репозитории содержит все обязательные переменные без значений секретов. Полный список переменных с пояснениями:

JWT_SECRET= # произвольная строка не менее 32 символов; одинакова для selfmanager, timer и analytics

JWT_ISSUER=productivesocial

JWT_AUDIENCE=productivesocial-users

JWT_REALM=productivesocial

INTERNAL_API_KEY= # произвольная строка; одинакова для всех сервисов

SM_DATABASE_URL= # jdbc:postgresql://host:port/dbname?user=u&password=p

TIMER_DATABASE_URL= # строка подключения PostgreSQL для базы timer

USER_REGISTRY_DB_HOST= # хост базы данных selfmanager (для UserRegistry в timer)

USER_REGISTRY_DB_PORT=5432

USER_REGISTRY_DB_NAME=

USER_REGISTRY_DB_USER=

USER_REGISTRY_DB_PASSWORD=

ANALYTICS_DATABASE_URL=

SELFMANAGER_BASE_URL=http://localhost:1226

TIMER_BASE_URL=http://localhost:1227

BILLING_BASE_URL=http://localhost:1229

BILLING_DATABASE_URL=

OLLAMA_BASE_URL=http://localhost:11434

ANTHROPIC_API_KEY= # опционально

OPENAI_API_KEY= # опционально

RESET_DB=false # установить true для сброса схемы при первом запуске

4. Тестовый набор данных

Все 12 экспериментов проводились на едином тестовом профиле пользователя `test@productivesocial.com`. Для воссоздания начального состояния выполнить следующие команды.

Шаг 1. Зарегистрировать пользователя и сохранить токен:

```
curl -s -X POST http://localhost:1226/api/v1/auth/identify \
-H "Content-Type: application/json" \
-d '{"email":"test@productivesocial.com"}'
```

Из ответа извлечь значение `accessToken` и присвоить его переменной `TOKEN`:

`TOKEN=<accessToken из ответа>`

Шаг 2. Создать три проекта (повторить для каждого с нужным `name` и `colorHex`):

```
curl -s -X POST http://localhost:1226/api/v1/projects \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"name":"Work","colorHex":"#4A90E2","priority":"HIGH"}'
```

Аналогично создать проекты `Personal` (`#50C878`, `MEDIUM`) и `Learning` (`#9B59B6`, `MEDIUM`). Сохранить возвращённые `id` проектов.

Шаг 3. Создать восемь задач с разными приоритетами и состояниями завершения (3 завершённых, 5 незавершённых), шесть привычек (5 ежедневных типа `Start`, 1 еженедельная типа `Quit`), четыре рутины с упорядоченными шагами (`Morning`, `Deep Work`, `Evening Study`, `Weekly Review`), 32 `Pomodoro`-сессии (31 завершённую, 1 прерванную) и 55 отметок выполнения привычек за 14 дней истории. Полные тела запросов и скрипт автоматизированного заполнения `seed.sh` приведены в репозитории `psocial_selfmanager` в директории `/scripts`.

5. API-документация

Интерактивная документация всех эндпоинтов доступна через `Swagger UI`:

billing (FastAPI/OpenAPI): <http://localhost:1229/docs> — полный список эндпоинтов с описанием параметров, тел запросов и форматов ответов; поддерживает выполнение запросов непосредственно в браузере.

selfmanager: <http://localhost:1226/swagger>

timer: <http://localhost:1227/swagger>

analytics: <http://localhost:1228/swagger>

На развёрнутых сервисах Render Swagger UI доступен по тем же путям с заменой localhost на соответствующий Render URL.

6. Воспроизведение экспериментов

Все команды выполняются после запуска локального стека (раздел 2) и создания тестового набора данных (раздел 4). Переменная TOKEN должна содержать действующий токен пользователя `test@productivesocial.com`.

Эксперимент №1 — Межсервисная проверка JWT.

```
curl -s -o /dev/null -w "%{http_code}" \
-H "Authorization: Bearer $TOKEN" \
http://localhost:1228/api/v1/analytics
curl -s -o /dev/null -w "%{http_code}" \
-H "Authorization: Bearer invalid.token.here" \
http://localhost:1228/api/v1/analytics
```

Ожидаемый результат: первая команда возвращает 200, вторая — 401.

Эксперимент №2 — Приветственный кредитный депозит.

```
curl -s -X POST http://localhost:1226/api/v1/auth/identify \
-H "Content-Type: application/json" \
-d '{"email":"newuser_experiment2@test.com"}'
```

Получить токен из ответа, затем проверить баланс через analytics:

```
curl -s http://localhost:1228/api/v1/credits/balance \
-H "Authorization: Bearer $NEW_TOKEN"
```

Ожидаемый результат: `balance = 100`.

Эксперимент №3 — Точность списания кредитов.

```
curl -s http://localhost:1228/api/v1/credits/balance \
-H "Authorization: Bearer $TOKEN"
curl -s -X POST http://localhost:1228/api/v1/analytics \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"analysisType":"PRODUCTIVITY_SUMMARY","modelId":"fb96de86-3c73-4
89d-8f02-aaf8343d6101"}'
curl -s http://localhost:1228/api/v1/credits/balance \
-H "Authorization: Bearer $TOKEN"
```

Ожидаемый результат: $\text{balance_after} = \text{balance_before} - 5$.

Эксперимент №4 — Идемпотентность синхронизации.

Отправить один и тот же sync-запрос дважды и сравнить ответы:

```
curl -s -X POST http://localhost:1226/api/v1/sync \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d @scripts/sync_payload.json
```

Повторить команду без изменений. Ожидаемый результат: idMappings в обоих ответах идентичны; количество новых записей во втором ответе равно нулю.

Эксперимент №5 — Полнота синхронизации.

```
curl -s http://localhost:1227/api/v1/pomodoro/sessions \
-H "Authorization: Bearer $TOKEN" | python3 -c "import sys,json; d=json.load(sys.
stdin); print(len(d))"
```

Ожидаемый результат: 32 сессии в timer; 8 задач и 55 отметок выполнения привычек в selfmanager.

Эксперименты №6, 7, 8 — Качество LLM-анализа (запустить трижды каждый, вывод должен совпадать при temperature=0):

```
curl -s -X POST http://localhost:1228/api/v1/analytics \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"analysisType":"PRODUCTIVITY_SUMMARY","modelId":"fb96de86-3c73-4
89d-8f02-aaf8343d6101"}'
```

Заменить PRODUCTIVITY_SUMMARY на HABIT_INSIGHTS (эксперимент №7) и WEEKLY_TIMER_SUMMARY (эксперимент №8). Оценить вывод по рубрике из таблицы 2.2.

Эксперимент №9 — Корректная деградация при недоступном timer.

```
docker compose stop timer
curl -s -X POST http://localhost:1228/api/v1/analytics \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"analysisType":"PRODUCTIVITY_SUMMARY","modelId":"fb96de86-3c73-489d-8f02-aaf8343d6101"}'
docker compose start timer
```

Ожидаемый результат: ответ 200 с текстом анализа; раздел Pomodoro отсутствует в выводе.

Эксперимент №10 — Задержка холодного старта на Render.

Выполнить после не менее 15 минут простоя сервисов:

```
time curl -s -o /dev/null https://psocial-selfmanager.onrender.com/health
```

Ожидаемый результат: первый ответ ~52 секунды; повторный запрос — менее 1 секунды.

Эксперимент №11 — Отсутствие списания при неудачном прогнозе.

В файле .env установить OLLAMA_BASE_URL=http://localhost:9999 (несуществующий адрес), перезапустить billing (docker compose restart billing), выполнить запрос анализа, проверить баланс. Ожидаемый результат: баланс не изменился; запись в predictions имеет статус FAILED без транзакции списания. Восстановить OLLAMA_BASE_URL и перезапустить billing.

Эксперимент №12 — Депозит администратора через внутренний API.

```
curl -s -X POST http://localhost:1229/api/v1/internal/users/deposit \
  -H "X-Internal-Key: $INTERNAL_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"selfmanagerUserId":1,"amount":50}'
```

Ожидаемый результат: $\text{balance_after} = \text{balance_before} + 50$; транзакция типа DEPOSIT записана в журнале.

Приложение В

Файл docker-compose.yml

```
version: "3.9"

# -----
# PSocial - full stack
#
# Services:
#   selfmanager_db :5432 - PostgreSQL for psocial_selfmanager
#   timer_db       :5434 - PostgreSQL for psocial_timer
#   billing_db     :5433 - PostgreSQL for psocial_billing_service
#   analytics_db   :5435 - PostgreSQL for psocial_analytics
#   selfmanager    :1226 - Ktor self-manager API
#   timer          :1227 - Ktor Pomodoro timer API
#   analytics       :1228 - Ktor analytics API
#   billing        :1229 - FastAPI billing + ML/LLM service
#   dashboard      :1230 - Streamlit admin dashboard
#   user_dashboard :1231 - Streamlit user dashboard
#
# Start: docker-compose up --build
# Stop:  docker-compose down
# Logs:  docker-compose logs -f <service>
# -----

x-jwt: &jwt
  JWT_SECRET: your-super-secret-jwt-secret-key-change-in-production
  JWT_ISSUER: ktor-psocial-app
  JWT_AUDIENCE: ktor-psocial
  JWT_REALM: ktor-psocial

x-internal: &internal
  INTERNAL_API_KEY: psocial-internal-secret

services:

  selfmanager_db:
    image: postgres:16-alpine
    container_name: psocial_selfmanager_db
    environment:
      POSTGRES_DB: psocial_selfmanager
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ""
      POSTGRES_HOST_AUTH_METHOD: trust
    ports:
      - "5432:5432"
    volumes:
      - selfmanager_db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks:
      - psocial_network

  timer_db:
    image: postgres:16-alpine
    container_name: psocial_timer_db
    environment:
      POSTGRES_DB: psocial_timer
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ""
      POSTGRES_HOST_AUTH_METHOD: trust
    ports:
      - "5434:5432"
    volumes:
      - timer_db_data:/var/lib/postgresql/data
    healthcheck:
```

```

    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 10
networks:
  - psocial_network

billing_db:
  image: postgres:16-alpine
  container_name: psocial_billing_db
  environment:
    POSTGRES_DB: psocial_billing
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: ""
    POSTGRES_HOST_AUTH_METHOD: trust
  ports:
    - "5433:5432"
  volumes:
    - billing_db_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 10
  networks:
    - psocial_network

analytics_db:
  image: postgres:16-alpine
  container_name: psocial_analytics_db
  environment:
    POSTGRES_DB: psocial_analytics
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: ""
    POSTGRES_HOST_AUTH_METHOD: trust
  ports:
    - "5435:5432"
  volumes:
    - analytics_db_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 10
  networks:
    - psocial_network

selfmanager:
  image: productivesocial-selfmanager
  container_name: psocial_selfmanager
  ports:
    - "1226:1226"
  environment:
    <<: [*jwt, *internal]
    DATABASE_URL: jdbc:postgresql://selfmanager_db:5432/psocial_selfmanager
    DB_USER: postgres
    DB_PASSWORD: postgres
    PORT: 1226
    HOST: 0.0.0.0
    BILLING_SERVICE_URL: http://billing:1229
    TIMER_SERVICE_URL: http://timer:1227
    ALLOWED_ORIGINS: ""
    USER_REGISTRY_DB_HOST: selfmanager_db
    USER_REGISTRY_DB_PORT: 5432
    USER_REGISTRY_DB_NAME: psocial_selfmanager
    USER_REGISTRY_DB_USER: postgres

```

```

    USER_REGISTRY_DB_PASSWORD: ""
depends_on:
  selfmanager_db:
    condition: service_healthy
networks:
  - psocial_network
restart: unless-stopped

timer:
  image: productivesocial-timer
  container_name: psocial_timer
  ports:
    - "1227:1227"
  environment:
    <<: [*jwt, *internal]
    DATABASE_URL: jdbc:postgresql://timer_db:5432/psocial_timer
    DB_USER: postgres
    DB_PASSWORD: postgres
    PORT: 1227
    HOST: 0.0.0.0
    SELFMANAGER_SERVICE_URL: http://selfmanager:1226
    ALLOWED_ORIGINS: ""
    USER_REGISTRY_DB_HOST: selfmanager_db
    USER_REGISTRY_DB_PORT: 5432
    USER_REGISTRY_DB_NAME: psocial_selfmanager
    USER_REGISTRY_DB_USER: postgres
    USER_REGISTRY_DB_PASSWORD: ""
  depends_on:
    timer_db:
      condition: service_healthy
    selfmanager:
      condition: service_started
  networks:
    - psocial_network
  restart: unless-stopped

analytics:
  image: productivesocial-analytics
  container_name: psocial_analytics
  ports:
    - "1228:1228"
  environment:
    <<: [*jwt, *internal]
    DATABASE_URL: jdbc:postgresql://analytics_db:5432/psocial_analytics
    DB_USER: postgres
    DB_PASSWORD: postgres
    PORT: 1228
    HOST: 0.0.0.0
    BILLING_SERVICE_URL: http://billing:1229
    SELFMANAGER_SERVICE_URL: http://selfmanager:1226
    TIMER_SERVICE_URL: http://timer:1227
    ALLOWED_ORIGINS: ""
    ADMIN_EMAILS: ${ADMIN_EMAILS:-admin@productivesocial.com}
  depends_on:
    analytics_db:
      condition: service_healthy
    billing:
      condition: service_started
    timer:
      condition: service_started
  networks:
    - psocial_network
  restart: unless-stopped

billing:
  image: productivesocial-billing

```

```

container_name: psocial_billing
ports:
  - "1229:1229"
environment:
  APP_ENV: production
  DATABASE_URL: postgresql+asyncpg://postgres:postgres@billing_db:5432/psocial_billing
  DATABASE_URL_SYNC: postgresql://postgres:postgres@billing_db:5432/psocial_billing
  JWT_SECRET_KEY: your-super-secret-jwt-secret-key-change-in-production
  JWT_ALGORITHM: HS256
  JWT_ACCESS_TOKEN_EXPIRE_MINUTES: 1440
  INTERNAL_API_KEY: psocial-internal-secret
  SELFMANAGER_BASE_URL: http://selfmanager:1226
  DEFAULT_CREDITS_ON_REGISTER: 100
  ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY:-}
  OPENAI_API_KEY: ${OPENAI_API_KEY:-}
  DEFAULT_LLM_PROVIDER: ollama
  OLLAMA_BASE_URL: http://host.docker.internal:11434
  OLLAMA_MODEL: llama3.2
volumes:
  - billing_models:/app/models_store
depends_on:
  billing_db:
    condition: service_healthy
  selfmanager:
    condition: service_started
networks:
  - psocial_network
restart: unless-stopped

dashboard:
  image: productivesocial-dashboard
  container_name: psocial_dashboard
  ports:
    - "1230:1230"
  environment:
    PORT: 1230
    BILLING_URL: http://billing:1229
    SELFMANAGER_URL: http://selfmanager:1226
    ANALYTICS_URL: http://analytics:1228
  depends_on:
    - billing
    - selfmanager
    - analytics
  networks:
    - psocial_network
  restart: unless-stopped

user_dashboard:
  image: productivesocial-user-dashboard
  container_name: psocial_user_dashboard
  ports:
    - "1231:1231"
  environment:
    PORT: 1231
    BILLING_URL: http://billing:1229
    SELFMANAGER_URL: http://selfmanager:1226
    ANALYTICS_URL: http://analytics:1228
    TIMER_URL: http://timer:1227
  depends_on:
    - billing
    - selfmanager
    - analytics
    - timer
  networks:
    - psocial_network
  restart: unless-stopped

```

```

networks:
  psocial_network:
    name: psocial_network
    driver: bridge

volumes:
  selfmanager_db_data:
  timer_db_data:
  billing_db_data:
  analytics_db_data:
  billing_models:

```

Диаграмма модели данных



Приложение Д

Экраны мобильного приложения

Источник: интерактивный макет (Figma)

